



**UNIVERSIDAD
DE GRANADA**

TRABAJO FIN DE GRADO
INGENIERÍA EN TECNOLOGÍAS DE TELECOMUNICACIÓN

Optimización de aprendizaje dinámico en conmutadores Ethernet de alto rendimiento

Autor

Luis Galindo Ortuño

Directores

Begoña del Pino Prieto

Carlos Megías Nuñez



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍAS INFORMÁTICA Y DE
TELECOMUNICACIÓN

Granada, Septiembre de 2026

Optimización de aprendizaje dinámico en conmutadores Ethernet de alto rendimiento

Luis Galindo Ortuño

Palabras clave: FPGA, conmutador Ethernet, tabla MAC, memoria caché, Cuckoo Hashing, Verilog, SimPy, TinyLFU.

Resumen

Este Trabajo Fin de Grado aborda el desarrollo de un módulo destinado a mejorar el proceso de aprendizaje dinámico en conmutadores Ethernet de alto rendimiento. El objetivo principal consiste en reducir las escrituras redundantes de direcciones MAC que alcanzan la tabla MAC, disminuyendo la presión sobre esta y mejorando su funcionamiento.

Para comenzar, se ha realizado una revisión del estado del arte sobre la conmutación Ethernet, las tecnologías FPGA y su aplicación en distintos proyectos de telecomunicaciones, así como sobre el aprendizaje dinámico de direcciones MAC en conmutadores Ethernet y los principales modelos utilizados. A partir de este estudio, se utiliza TinyLFU como idea de partida para el diseño de las soluciones de filtrado propuestas.

A continuación, se desarrolla un modelo funcional en Python utilizando SimPy. Sobre este modelo se comparan tres arquitecturas: un modelo base sin filtrado, un modelo con caché y un modelo que combina caché y un estimador de frecuencia (Count-Min Sketch). Para realizar las distintas simulaciones se genera el tráfico de red modelándolo mediante una distribución de Zipf.

A partir de los resultados obtenidos en el modelado funcional, se selecciona la arquitectura más adecuada y se desarrolla una implementación en Verilog HDL, utilizando cocotb para realizar las distintas simulaciones. Posteriormente, la arquitectura se sintetiza e implementa sobre una FPGA Zynq UltraScale+ MPSoC de una placa ZCU102 mediante Vivado.

Finalmente, se realiza una validación del modelo implementado, así como una validación cruzada entre las distintas etapas en Python, HDL y FPGA. Los resultados muestran una notable mejora del modelo con caché implementado respecto al modelo base para distintos escenarios de tráfico. Asimismo, los resultados muestran un comportamiento consistente entre las distintas etapas de implementación (Python, HDL y FPGA), validando tanto el modelo como la metodología empleada.

Optimization of Dynamic Learning in High-Performance Ethernet Switches

Luis Galindo Ortuño

Keywords: FPGA, Ethernet switch, MAC table, cache memory, Cuckoo Hashing, Verilog, SimPy, TinyLFU.

Abstract

This Bachelor's Thesis addresses the development of a module aimed at improving the dynamic learning process in high-performance Ethernet switches. The main objective is to reduce redundant MAC address writes reaching the MAC table, thereby reducing the pressure on it and improving its performance.

To begin with, a review of the state of the art has been carried out, covering Ethernet switching, FPGA technologies and their application in different telecommunications projects, as well as dynamic MAC address learning in Ethernet switches and the main models used. Based on this study, TinyLFU is used as the starting point for the design of the proposed filtering solutions.

Next, a functional model is developed in Python using SimPy. Three architectures are compared using this model: a baseline model without filtering, a cache-based model, and a model combining a cache with a frequency estimator (Count-Min Sketch). For the different simulations, network traffic is generated using a Zipf distribution.

Based on the results obtained from the functional modeling, the most suitable architecture is selected and an implementation is developed in Verilog HDL, using cocotb to perform the different simulations. The architecture is subsequently synthesized and implemented on the Zynq UltraScale+ MPSoC FPGA of a ZCU102 board using Vivado.

Finally, the implemented model is validated, together with a cross-validation between the different Python, HDL, and FPGA stages. The results show a significant improvement of the implemented cache-based model over the baseline model under different traffic scenarios. Furthermore, the results show consistent behavior across the different implementation stages (Python, HDL, and FPGA), validating both the model and the methodology employed.

Yo, **Luis Galindo Ortuño**, alumno de la titulación Ingeniería de Tecnologías de Telecomunicación de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 25619528N, asumo la autoría y originalidad del trabajo, entendida esta última en el sentido de que no ha utilizado fuentes sin citarlas debidamente

Yo, **Luis Galindo Ortuño**, alumno de la titulación Ingeniería de Tecnologías de Telecomunicación de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 25619528N, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: Luis Galindo Ortuño

Granada a 3 de septiembre de 2026.

Dña. **María Begoña del Pino Prieto**, Profesora del Área de Arquitectura y Tecnología de Computadores del Departamento de Ingeniería de Computadores, Automática y Robótica de la Universidad de Granada.

D. **Carlos Megías Núñez**, Profesor en el curso 2025/2026 del Área de Arquitectura y Tecnología de Computadores del Departamento de Ingeniería de Computadores, Automática y Robótica de la Universidad de Granada.

Informan:

Que el trabajo titulado *Optimización de aprendizaje dinámico en conmutadores Ethernet de alto rendimiento*, ha sido realizado bajo su supervisión por D. Luis Galindo Ortuño, y autorizamos la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expiden y firman el presente informe en Granada a 3 de septiembre de 2026.



Índice general

1	Introducción	9
1.1	Motivación	9
1.2	Planteamiento del problema	10
1.3	Objetivos del proyecto	10
1.4	Metodología	11
1.5	Planificación y cronograma	12
2	Estado del Arte y Fundamentos	14
2.1	Conmutación Ethernet	14
2.2	Gestión de Tablas de Direcciones MAC	18
2.3	FPGA	21
2.4	Modelo de tráfico en redes: distribución de Zipf	26
3	Materiales y métodos	28
3.1	Enfoque de modelado	28
3.2	Herramientas de desarrollo software para simulación funcional	29
3.3	Herramientas de desarrollo hardware	30
3.4	Plataforma hardware y protocolos de comunicación	30
3.5	Uso herramientas IA	31
4	Diseño de soluciones de mejora de aprendizaje dinámico	32
4.1	Análisis de técnicas de aprendizaje dinámico	32
4.2	TinyLFU	34
4.3	Modelado funcional y validación del algoritmo	39
5	Implementación hardware e integración	59
5.1	Implementación HDL del módulo de aprendizaje	59
5.2	Integración en el datapath del switch	71
5.3	Implementación y validación en FPGA	75
6	Evaluación de resultados	80
6.1	Parámetros de simulación	80
6.2	Validación cruzada de modelos	82
6.3	Análisis del comportamiento mediante métricas	84
6.4	Caracterización de recursos en la FPGA	91
6.5	Discusión de resultados	92
7	Conclusiones y trabajo futuro	94
7.1	Conclusiones	94
7.2	Trabajo futuro	96
A	Repositorio	97

Índice de figuras

1.1	Cronograma del desarrollo del proyecto.	13
2.1	Distribución Zipf para distintos valores de α	27
4.1	Funcionamiento filtro tipo Bloom	35
4.2	Funcionamiento Count-Min Sketch	37
4.3	Sistema con caché.	40
4.4	Sistema con CMS + caché.	41
4.5	Porcentaje de tráfico filtrado por la caché a 51,2 Gbps	55
4.6	Porcentaje de descartes por saturación en la tabla MAC a 51,2 Gbps	56
4.7	Porcentaje de direcciones MAC aprendidas a 51,2 Gbps	57
4.8	Eficiencia de escritura en la tabla MAC a 51,2 Gbps	57
5.1	Comunicación entre el test en Python y el módulo de aprendizaje implementado en la FPGA.	71
6.1	MACs distintas generadas según el número de MACs posibles para 30.000 tramas generadas	81
6.2	Validación cruzada entre Python, HDL y FPGA para el filtrado de caché y los descartes por saturación de la cola MAC.	82
6.3	Validación cruzada entre Python, HDL y FPGA para el aprendizaje de direcciones MAC y la eficiencia de escritura.	82
6.4	Porcentaje de tráfico filtrado por la caché en función de las direcciones MAC posibles.	84
6.5	Evolución temporal del porcentaje de filtrado de caché en función de las tramas generadas.	85
6.6	Porcentaje de descartes por saturación de la cola MAC en función de las direcciones MAC posibles.	86
6.7	Evolución temporal de los descartes por saturación de la cola MAC.	87
6.8	Porcentaje de direcciones MAC aprendidas en función del número de MACs posibles.	88
6.9	Evolución temporal del porcentaje de direcciones MAC aprendidas.	89
6.10	Eficiencia de escritura en la tabla MAC en función del número de MACs posibles.	90
6.11	Evolución temporal de la eficiencia de escritura en la tabla MAC.	91

Índice de tablas

2.1	Estructura de una trama Ethernet	15
4.1	Parámetros de generación de tráfico utilizados en las simulaciones.	42
5.1	Principales contadores implementados en el módulo HDL. . .	70
5.2	Registros principales implementados en <code>cache_core.v</code>	74
6.1	Diferencias máximas entre Python, HDL y FPGA.	83
6.2	Utilización global de recursos en la FPGA.	91

Capítulo 1

Introducción

Los centros de datos, las redes industriales y los entornos de procesamiento distribuido se implementan sobre redes Ethernet capaces de transportar grandes volúmenes de tráfico con baja latencia y un comportamiento temporal estable. En estos escenarios, los conmutadores Ethernet de capa 2 (*switches*) constituyen un elemento fundamental: deben recibir cada trama, consultar la asociación entre su dirección MAC de destino y el puerto de salida correspondiente, y reenviar la trama a velocidad de línea. Cualquier retraso o mal funcionamiento en este proceso puede aumentar la congestión de la red, introducir latencia excesiva y limitar el rendimiento global de la red.

La eficiencia de este proceso depende, entre otros factores, de la capacidad del *switch* para mantener actualizada su tabla de direcciones MAC y realizar las operaciones de aprendizaje, consulta y reenvío a velocidad de línea de la red. No obstante, dichas tablas MAC disponen de una capacidad limitada, lo que puede provocar situaciones de saturación que afectan tanto al rendimiento del sistema como a la seguridad de la propia red.

Este trabajo aborda el diseño e implementación de un mecanismo de gestión de tablas MAC implementado sobre FPGA.

1.1. Motivación

La evolución de las redes Ethernet ha incrementado las exigencias sobre los dispositivos de conmutación. En centros de datos, redes industriales, sistemas de telecomunicaciones y otras infraestructuras críticas, no basta con ofrecer una elevada capacidad de transmisión: resulta necesario garantizar que las tramas sean procesadas con una latencia baja, estable y compatible con el funcionamiento cada vez a velocidades de línea mayores.

Un *switch* debe ser capaz de procesar y reenviar cada trama de entrada hacia el puerto asociado a su dirección MAC de destino sin interrumpir el flujo de datos. Este requisito adquiere especial relevancia en redes Ethernet

de alta capacidad, cuyas especificaciones abarcan enlaces de 400 y 800 GbE [1]. Para ello, la tabla MAC del *switch* aprende las direcciones MAC de origen de las tramas recibidas y las asocia con el puerto de entrada correspondiente. Cuando posteriormente recibe una trama dirigida a una dirección conocida, consulta la tabla MAC y la reenvía únicamente por el puerto asociado. Este comportamiento reduce el tráfico innecesario y permite aprovechar de forma eficiente los recursos de la red.

Las FPGAs constituyen una plataforma adecuada para implementar este tipo de arquitecturas. Su capacidad de paralelismo y de procesamiento segmentado permite que distintas etapas del procesamiento de una trama se ejecuten de forma concurrente. Frente a soluciones puramente software, esta característica facilita una latencia más determinista y una mayor capacidad de procesamiento. Además, la reconfigurabilidad de la FPGA permite modificar o ampliar la arquitectura sin necesidad de rediseñar un circuito integrado específico, lo que resulta útil en prototipos, sistemas de investigación y redes que requieren mecanismos de control adaptables.

1.2. Planteamiento del problema

El principal problema se origina cuando la tabla MAC de un *switch* no es capaz de procesar todas las direcciones MAC de las tramas de entrada debido a su capacidad limitada, a la alta tasa de llegada de tramas, a la existencia de un elevado número de escrituras redundantes o a escenarios anómalos de la red.

La saturación de la tabla MAC afecta directamente al comportamiento del plano de datos. Si una dirección MAC de destino no dispone de una entrada válida, la trama correspondiente debe difundirse por todos los puertos de salida distintos del puerto de entrada para alcanzar su destino. Aunque este mecanismo permite mantener la conectividad cuando el destino no es conocido, incrementa enormemente el tráfico de la red, empeorando su funcionamiento y en casos extremos, saturándola por completo.

El problema abordado en este trabajo consiste en diseñar un mecanismo hardware de filtrado y aprendizaje capaz de optimizar el funcionamiento de la tabla MAC en diversas situaciones de tráfico de red. La solución implementada debe ser capaz de realizar este trabajo sin penalizar el procesamiento de tramas a la velocidad de línea del *switch*.

1.3. Objetivos del proyecto

El objetivo general de este proyecto es diseñar, implementar e integrar una solución hardware sobre FPGA para la optimización del aprendizaje dinámico de la tabla MAC en conmutadores Ethernet de alto rendimiento.

Para alcanzar este propósito general, se definen los siguientes objetivos específicos:

- **Revisión del estado del arte.** Estudiar los fundamentos de la conmutación Ethernet de capa 2, el funcionamiento de las tablas MAC, los procesos de aprendizaje, consulta y reenvío de entradas, así como los principales mecanismos de aprendizaje dinámico utilizados tanto en *switches* como en otros sistemas de almacenamiento y procesamiento de datos cuyas estrategias resulten aplicables al aprendizaje de direcciones MAC.
- **Modelado del tráfico Ethernet.** Diseñar y parametrizar un modelo de tráfico representativo del comportamiento habitual del tráfico en las redes Ethernet que permita evaluar el rendimiento de la arquitectura propuesta bajo diferentes condiciones de funcionamiento.
- **Diseño de las posibles soluciones.** Diseñar las diferentes arquitecturas objeto de estudio, definiendo su funcionamiento y sus distintos bloques.
- **Validación funcional en Python.** Desarrollar un modelo funcional en Python que permita validar el comportamiento de las arquitecturas propuestas y evaluar su rendimiento bajo diferentes escenarios de tráfico antes de su implementación hardware.
- **Implementación en HDL.** Desarrollar los módulos definidos mediante un lenguaje de descripción hardware, garantizando una arquitectura adecuada para su posterior implementación en FPGA.
- **Implementación en FPGA.** Implementar la arquitectura propuesta en una plataforma FPGA y comprobar su correcto funcionamiento mediante simulación y validación experimental.
- **Validación y evaluación de la solución.** Realizar una validación cruzada entre los modelos desarrollados en Python, HDL y la implementación en la FPGA para verificar la consistencia de los resultados obtenidos. Evaluar el rendimiento de las arquitecturas propuestas y analizar la utilización de recursos hardware en la implementación sobre la FPGA.

1.4. Metodología

La metodología seguida en este trabajo para el desarrollo e implementación del módulo de aprendizaje dinámico puede diferenciarse en tres etapas principales.

En la primera etapa, se desarrollan varios modelos funcionales en Python con las distintas arquitecturas propuestas, utilizando la librería SimPy. Esta librería permite representar el comportamiento del sistema mediante simulación de eventos discretos. Sobre este entorno se realizan distintas pruebas

para evaluar el comportamiento de cada arquitectura y decidir cuáles de ellas deben trasladarse a una descripción hardware en Verilog HDL.

Tras la validación funcional del modelo en Python, la arquitectura seleccionada se describe en Verilog HDL. En esta etapa se desarrollan los distintos módulos hardware que forman el sistema de aprendizaje y filtrado, se definen sus interfaces de entrada y salida, y se conectan mediante señales de control.

La verificación de esta descripción HDL se realiza mediante simulaciones con *cocotb*. Estas pruebas permiten evaluar el diseño ciclo a ciclo, siguiendo la evolución de las señales en cada flanco de reloj, y comparar los contadores obtenidos con los del modelo funcional desarrollado en Python. Esta validación cruzada permite comprobar que ambas implementaciones representan el mismo comportamiento antes de continuar con la síntesis e integración en FPGA.

La última etapa consiste en la implementación de la arquitectura desarrollada sobre una plataforma FPGA. Para ello se sigue el flujo completo de síntesis, implementación y generación del *bitstream* mediante Vivado, obteniendo un diseño funcional en hardware.

Con el fin de validar el comportamiento del sistema en un entorno real, se utiliza un banco de pruebas que permite cargar secuencias de tráfico en una memoria inferida como BRAM y controlar la ejecución desde un ordenador mediante una infraestructura de control basada en UART, XFCP y AXI-Lite. Finalmente, los resultados obtenidos se comparan con los procedentes de las simulaciones en Python y Verilog HDL, verificando la consistencia entre las distintas implementaciones y evaluando tanto el rendimiento alcanzado como la utilización de recursos hardware de la FPGA.

1.5. Planificación y cronograma

El desarrollo del trabajo se ha organizado en distintas fases, siguiendo una evolución progresiva desde el estudio inicial del problema hasta la validación final sobre la FPGA. La metodología descrita en el apartado anterior ha permitido desarrollar el proyecto de forma ordenada e iterativa, validando cada una de las etapas antes de avanzar a la siguiente y reduciendo así el riesgo de propagar errores a lo largo del proceso de desarrollo.

Aunque las distintas fases se presentan de forma secuencial, algunas de ellas se han desarrollado de forma simultánea o discontinua. Es el caso de la documentación del trabajo, que se ha elaborado de manera discontinua, así como la evaluación de resultados, que se ha realizado a medida que las arquitecturas se iban implementando en los modelos en Python, Verilog HDL y FPGA. La Figura 1.1 muestra el cronograma seguido durante el desarrollo del trabajo.

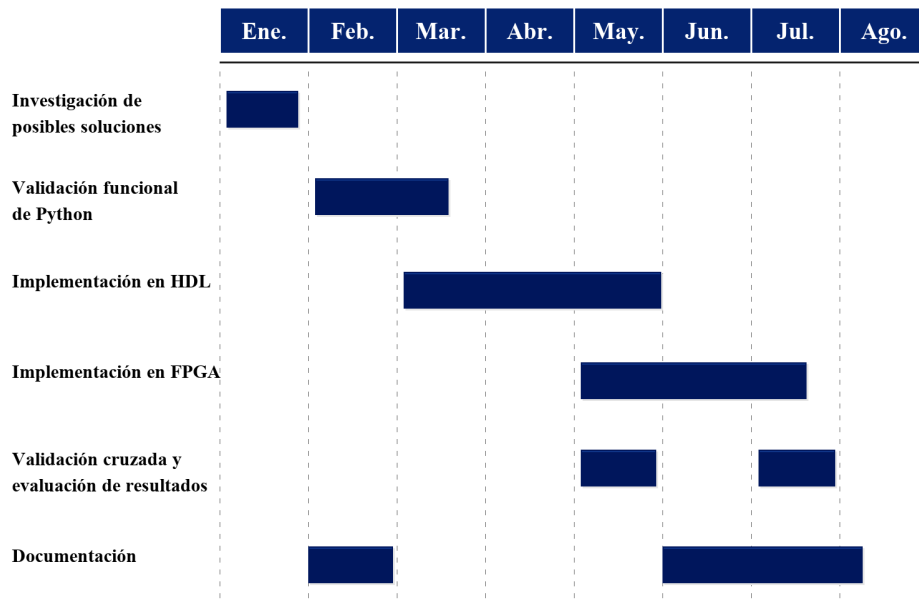


Figura 1.1: Cronograma del desarrollo del proyecto.

Capítulo 2

Estado del Arte y Fundamentos

En este capítulo se explica el funcionamiento de las redes Ethernet. Para ello se definirá tanto su estructura como los distintos elementos fundamentales que conforman estas redes. A continuación, se centrará el estudio en las tablas de direccionamiento MAC y su estructura. También se analizarán distintos mecanismos de aprendizaje dinámico para poder optimizar el funcionamiento de las mismas. Por último, se va a realizar un pequeño análisis de cómo se implementan estos switches en FPGAs.

2.1. Conmutación Ethernet

La conmutación Ethernet es uno de los elementos fundamentales en el funcionamiento de las redes de área local (LAN) actuales. A nivel de la capa 2 del modelo OSI, los conmutadores Ethernet (*switches*) permiten que múltiples dispositivos se comuniquen de forma eficiente, reenviando cada trama únicamente hacia el destino adecuado en lugar de inundar innecesariamente toda la red. Este comportamiento supone una mejora significativa respecto a tecnologías anteriores como los hubs, donde todo el tráfico era difundido a todos los puertos independientemente de su destinatario.

Con el paso del tiempo, los conmutadores Ethernet han evolucionado tanto en capacidad como en complejidad. Estos *switches* han incorporado nuevas funcionalidades y optimizaciones que permiten gestionar grandes volúmenes de tráfico con baja latencia. Para comprender su funcionamiento, es necesario analizar tanto la estructura de las tramas Ethernet como los mecanismos internos de aprendizaje y reenvío que utilizan.

Estructura de la trama Ethernet

En el protocolo Ethernet, la información se transmite fragmentada en unidades denominadas tramas (*frames*). Estas tramas siguen una estructura definida por el estándar IEEE [2], que permite que todos los dispositivos puedan comunicarse independientemente de su fabricante o de su tecnología de red.

A continuación se muestra una tabla donde se define cada uno de los campos que conforman una trama Ethernet, así como su respectivo tamaño.

Campo	Bytes	Descripción
Preámbulo	7	Sincronización de bits
Delimitador de inicio de trama (SFD)	1	Marca el inicio de la trama
MAC de destino	6	Dirección hardware de destino
MAC de origen	6	Dirección hardware de origen
EtherType	2	Indicador de protocolo
Carga útil	46–1500	Datos encapsulados de capas superiores
Secuencia de verificación de trama (FCS)	4	Detección de errores mediante CRC-32

Tabla 2.1: Estructura de una trama Ethernet

El tamaño mínimo de una trama es de 64 bytes, longitud con la que se pueden detectar colisiones en enlaces semi-dúplex. El tamaño máximo es 1518 bytes para tramas estándar, o 1522 bytes cuando está presente una etiqueta VLAN IEEE 802.1Q utilizada para identificar este tipo de redes.

Proceso de aprendizaje y reenvío de tramas

Un switch de capa 2 regula el tráfico de tramas mediante una tabla MAC, cuya función es asociar direcciones MAC con los puertos del propio switch. De este modo, el switch es capaz de reenviar la trama por el puerto adecuado, reduciendo y optimizando el tráfico de la red. El funcionamiento de un switch Ethernet puede dividirse en cuatro procesos principales:

- Aprendizaje (*learning*): cuando una trama llega al switch, este registra la asociación dirección MAC-puerto de entrada de la trama en la tabla

MAC. Una vez que realiza este aprendizaje, ya conoce el puerto por el que tiene que reenviar la información cuando le llegue una trama con la dirección MAC destino conocida.

- **Búsqueda (*lookup*):** al recibir una trama, el switch busca en su tabla MAC por qué puerto necesita reenviar la trama para que llegue a su destino.
- **Reenvío (*forwarding*):** dependiendo del contenido de la trama y del resultado de la etapa de lookup, tenemos tres posibles escenarios:
 - Si la dirección MAC es conocida con direccionamiento unicast, la trama se reenvía por el puerto que indique la tabla MAC.
 - Si la dirección tiene direccionamiento broadcast o multicast, se envía por todos los puertos del switch.
 - Si la dirección MAC es desconocida, se envía por todos los puertos del switch menos el de entrada. A este proceso se le conoce como flooding [3].
- **Envejecimiento (*aging*):** proceso por el cual la tabla MAC se encarga de eliminar las entradas inactivas durante un cierto periodo de tiempo. De este modo, se libera espacio en la tabla MAC para futuras asociaciones MAC-puerto y se mantiene la tabla MAC actualizada.

Arquitectura de switches Ethernet

Un switch Ethernet no se limita únicamente a reenviar tramas entre puertos. Internamente, su funcionamiento se organiza en distintos planos funcionales, cada uno con responsabilidades y requisitos de rendimiento diferentes. En los switches tradicionales, estos planos suelen encontrarse integrados dentro del mismo dispositivo hardware, mientras que en arquitecturas más modernas, como las redes definidas por software (SDN), es habitual separar parte de estas funciones.

Plano de datos (*Data Plane*)

El plano de datos es el encargado de procesar las tramas a velocidad de línea. Su función principal consiste en recibir cada paquete, analizar su información de cabecera y decidir hacia qué puerto debe reenviarse. Debido a las velocidades actuales de las redes Ethernet, este procesamiento debe realizarse en tiempos extremadamente reducidos, por lo que normalmente se implementa mediante hardware especializado, como ASICs o FPGAs.

Entre las principales tareas realizadas por el plano de datos se encuentran:

- **Búsqueda en la tabla MAC:** el switch consulta la dirección MAC destino de la trama para determinar el puerto de salida correspondien-

te.

- **Conmutación interna:** una vez decidido el destino, la trama atraviesa la denominada *switch fabric*, normalmente implementada mediante arquitecturas tipo *crossbar*, que permiten múltiples transferencias simultáneas entre puertos.
- **Gestión de colas y planificación:** cuando varios paquetes compiten por el mismo puerto de salida, el switch debe decidir el orden de transmisión utilizando algoritmos de planificación como FIFO, prioridad estricta o *Weighted Fair Queuing*.

En arquitecturas modernas de altas prestaciones, estas operaciones suelen organizarse mediante pipelines hardware, permitiendo procesar varias tramas de manera concurrente y mantener velocidades de procesamiento de cientos de millones de paquetes por segundo.

Plano de control (*Control Plane*)

El plano de control (*Control Plane*) es responsable de calcular y mantener las reglas que posteriormente utiliza el plano de datos. A diferencia del procesamiento de paquetes, estas tareas operan en escalas temporales mucho mayores y se implementan mediante software ejecutado en un procesador embebido o en otros elementos de control del sistema.

Entre sus funciones más importantes destacan:

- Aprendizaje dinámico de direcciones MAC (también puede integrarse parcial o totalmente en hardware dentro del plano de datos).
- Ejecución de protocolos de prevención de bucles como STP o RSTP.
- Gestión de VLANs y etiquetado IEEE 802.1Q.
- Protocolos de administración y descubrimiento de topología.
- Funciones de encaminamiento IP en switches de capa 3.

La interacción entre el plano de control y el plano de datos se realiza principalmente a través de las tablas de reenvío. El plano de control actualiza estas estructuras, mientras que el plano de datos las consulta continuamente durante el procesamiento de paquetes.

En algunas arquitecturas modernas, como las arquitecturas SDN (*Software Defined Networking*) parte de las funciones de control pueden ejecutarse de forma centralizada en entidades externas al switch. Esta separación permite una gestión más flexible de la red y facilita la implementación de nuevas políticas de encaminamiento, monitorización y control de tráfico mediante interfaces abiertas como OpenFlow o P4Runtime [4].

Plano de gestión (*Management Plane*)

Además del plano de datos y del plano de control, muchos switches incorporan un plano de gestión (*Management Plane*) encargado de la configuración y supervisión del dispositivo. Este plano proporciona mecanismos de administración permitiendo monitorizar el estado del switch y modificar su configuración de forma remota.

En sistemas de altas prestaciones, el plano de gestión suele disponer de canales de comunicación independientes del tráfico de datos, garantizando que el dispositivo continúe siendo administrable incluso en situaciones de congestión o ataques de red.

2.2. Gestión de Tablas de Direcciones MAC

La gestión de las tablas de direcciones MAC constituye uno de los elementos clave en el funcionamiento de los switches de capa 2. Esta gestión es fundamental para la eficiencia del proceso de reenvío y la correcta adaptación a cambios en la topología de red [5].

Arquitecturas centralizadas y distribuidas

Existen dos modelos principales de arquitecturas en las tablas MAC. En una arquitectura centralizada, la tabla MAC, también conocida como *Forwarding Database* (FDB), se encuentra ubicada en un único punto, normalmente en el ASIC principal del switch. Todas las decisiones de forwarding se realizan consultando esta tabla central. La principal ventaja es su sencillez, ya que existe una única copia de la información y es complicado que se produzcan inconsistencias. Sin embargo, cada *line card* debe consultar el módulo central a través del *backplane*, lo que puede convertirse en un cuello de botella a medida que aumenta el número de puertos y la velocidad de transmisión.

Por otro lado, en una arquitectura distribuida, la tabla MAC se replica o se reparte entre las propias tarjetas de línea del switch. De este modo, cada *line card* mantiene su propia copia, ya sea total o parcial, y puede tomar decisiones de reenvío de manera local sin necesidad de acceder a un elemento central. Este enfoque permite mejorar la escalabilidad y reducir la latencia asociada a las consultas. No obstante, introduce un problema fundamental: la sincronización de la información. Cuando una dirección MAC se aprende en una tarjeta, esta información debe propagarse al resto de tarjetas para mantener una visión coordinada en la red.

Implementación en Hardware

Las tablas de direccionamiento MAC de los switches Ethernet comentadas anteriormente suelen implementarse utilizando memorias CAM (Content

Addressable Memory) o estructuras basadas en este tipo de memorias. Esto se debe a que el switch necesita buscar rápidamente si una dirección MAC ya se encuentra almacenada en la tabla y determinar el puerto asociado a dicha dirección [6].

A diferencia de una memoria convencional, donde se accede al dato mediante una dirección, las memorias CAM permiten realizar búsquedas utilizando directamente el contenido. De esta forma, el switch puede comparar una dirección MAC recibida con todas las entradas almacenadas de manera paralela, obteniendo el resultado en un único ciclo de reloj. Esto permite reducir significativamente el tiempo necesario para realizar las operaciones de forwarding y aprendizaje de direcciones.

Modelos CAM

El primer modelo de memoria CAM fueron las memorias BCAM (Binary Content Addressable Memory). Este tipo de memoria CAM realiza comparaciones binarias exactas sobre los datos almacenados. En las tablas MAC, las búsquedas corresponden a coincidencias exactas sobre direcciones de 48 bits, por lo que una BCAM resulta funcionalmente adecuada para este propósito. Sin embargo, este tipo de memorias presenta un elevado coste hardware, un consumo energético superior al de las memorias SRAM y una escalabilidad limitada cuando aumenta el número de entradas.

La evolución para solucionar este problema fue añadir un tercer estado lógico por bit. La TCAM (Ternary CAM) extiende el alfabeto de $\{0, 1\}$ a $\{0, 1, X\}$, donde X funciona como una máscara. De esta forma, un bit almacenado con valor X mantiene el resultado de la comparación independientemente del valor introducido durante la búsqueda, actuando como un comodín a nivel hardware [7]. El principal problema de las TCAMs es que son chips caros: compiten por espacio en PCB, generan calor considerable y su capacidad es limitada, típicamente hasta 72 Mb en chips comerciales. Las tablas de enrutamiento del núcleo de Internet superan con creces esa capacidad.

Algoritmos de hashing

Debido al elevado coste y consumo de las memorias CAM, muchos switches Ethernet implementan las tablas MAC utilizando memorias SRAM junto con algoritmos de hashing. Una tabla hash transforma una clave de longitud arbitraria, como en el caso de este proyecto una dirección MAC, en un índice de tabla mediante una función matemática. El atractivo en hardware es claro: en lugar de comparar en paralelo contra miles de entradas como hace la TCAM, se calcula una posición y se accede directamente. Una sola lectura a SRAM puede devolver el resultado.

El problema fundamental es que cualquier función hash no perfecta produce colisiones: dos claves distintas pueden mapear al mismo índice. En redes Ethernet, las colisiones son costosas no solo en throughput sino en previsibilidad: si el tiempo de búsqueda varía entre 1 acceso y N accesos dependiendo del historial de inserciones, dimensionar el hardware para garantizar la velocidad de la red se vuelve un proceso muy complicado y complejo.

Cuckoo Hashing

El algoritmo de hashing que se va a implementar en este trabajo se conoce como Cuckoo Hashing. Cuckoo Hashing fue propuesto por Rasmus Pagh y Flemming Rodler en 2004 [8]. La idea principal del cuckoo hashing consiste en utilizar dos tablas, T_1 y T_2 , de igual tamaño, cada una con una función hash independiente, h_1 y h_2 . De esta forma, cada clave tiene únicamente dos posibles posiciones de almacenamiento, una en cada tabla.

Para buscar una clave k , se consultan las posiciones $h_1(k)$ en T_1 y $h_2(k)$ en T_2 . La clave se encontrará en una de estas dos posiciones o no existirá en la tabla. Esto permite realizar las búsquedas utilizando siempre un número fijo de accesos a memoria, proporcionando una latencia determinista, característica especialmente interesante en implementaciones hardware.

El proceso de inserción es el mecanismo más característico de este tipo de tablas hash. Primero se intenta almacenar la clave en la posición $h_1(k)$. Si dicha posición está libre, la inserción finaliza. En caso contrario, la clave almacenada en esa posición es desalojada y trasladada a su posición alternativa en la segunda tabla. Si esta nueva posición también se encuentra ocupada, el proceso continúa de forma sucesiva, generando una cadena de desplazamientos entre ambas tablas.

Cuando el número de desplazamientos supera un determinado límite, resulta necesario reconstruir completamente la tabla utilizando nuevas funciones hash.

Posteriormente se mejoró este algoritmo, implementándolo sobre un número d de tablas hash con d funciones hash independientes [9]. Esto aumenta el factor de utilización máxima de la tabla, con 3 tablas se puede alcanzar aproximadamente un 91 %, y reduce la probabilidad de re-hasheo, a costa de que el lookup requiere hasta d accesos. En este proyecto, se va a implementar el algoritmo de Cuckoo Hashing utilizando 4 tablas hash en paralelo.

Aprendizaje dinámico

El aprendizaje dinámico es un mecanismo utilizado por los switches Ethernet para construir automáticamente sus tablas de direccionamiento MAC. Cada vez que un switch recibe una trama, aprende la asociación entre la dirección MAC de origen y el puerto por el que ha sido recibida o la actualiza si la información de la entrada correspondiente ya existía [10]. De

esta forma, el switch aprende progresivamente qué dispositivos se encuentran conectados a cada puerto de la red, pudiendo reenviar posteriormente las tramas únicamente hacia el destino correspondiente y reduciendo así el flooding innecesario.

Las entradas almacenadas en la tabla MAC no son permanentes, sino que cada una de ellas tiene asociado un temporizador de aging. Si durante un determinado periodo de tiempo no se recibe ninguna trama con dicha dirección MAC como origen, la entrada es eliminada de la tabla. En cambio, si la trama aparece se reinicia su temporizador para tener en cuenta que se ha visto esa dirección MAC recientemente. En la mayoría de implementaciones Ethernet este tiempo suele ser de aproximadamente 300 segundos.

Este mecanismo resulta necesario debido a que los dispositivos pueden desconectarse de la red o cambiar de puerto dentro de la topología. Sin un mecanismo de aging, la tabla MAC acabaría almacenando entradas obsoletas, provocando que las tramas fueran reenviadas hacia puertos incorrectos.

Además, las tablas MAC disponen de una capacidad de almacenamiento limitada, por lo que es necesario eliminar periódicamente las asociaciones MAC-puerto que han dejado de estar activas para liberar espacio para nuevos dispositivos. En switches de altas prestaciones, el proceso de aprendizaje dinámico resulta especialmente crítico, ya que la gran cantidad de asociaciones MAC-puerto que se generan y actualizan continuamente puede ejercer una presión significativa sobre las estructuras de almacenamiento. Esta situación motiva el desarrollo de mecanismos capaces de reducir las escrituras sobre la tabla MAC sin comprometer el rendimiento del proceso de aprendizaje dinámico.

2.3. FPGA

Una FPGA (*Field-Programmable Gate Array*) es un circuito integrado digital compuesto por bloques de lógica programable conectados mediante una red de interconexiones también programable. A diferencia de los circuitos integrados de aplicación específica (ASIC), las FPGA permiten implementar diferentes arquitecturas hardware mediante la configuración de estos recursos. En las familias basadas en tecnologías SRAM o memoria Flash, dicha configuración puede modificarse tras la fabricación del dispositivo [11], proporcionando una elevada flexibilidad para adaptar el hardware a distintas aplicaciones.

Esta capacidad de configuración, unida a la posibilidad de ejecutar múltiples operaciones de forma completamente paralela, sitúa a las FPGA en un punto intermedio entre los procesadores de propósito general, muy flexibles pero limitados para aplicaciones altamente paralelas, y los ASIC, que ofrecen el máximo rendimiento y eficiencia energética a costa de un elevado coste de diseño y una escasa flexibilidad.

Arquitectura de una FPGA

Las FPGA modernas se organizan en torno a un tejido programable formado por bloques lógicos configurables (CLB, *Configurable Logic Blocks*) conectados mediante una red de interconexión también programable. Cada bloque lógico está compuesto por elementos básicos como las *Look-Up Tables* (LUT), encargadas de implementar funciones lógicas combinacionales, y biestables, utilizados para construir lógica secuencial. La combinación de estos elementos permite implementar desde circuitos sencillos hasta sistemas digitales de gran complejidad.

Los bloques lógicos se comunican mediante una red jerárquica de interconexiones programables [11], responsable de transportar las señales entre los distintos recursos del dispositivo. Esta red ocupa una parte muy significativa del área del chip y desempeña un papel fundamental en el rendimiento del diseño, ya que el retardo asociado al encaminamiento representa una fracción importante del retardo total del circuito.

Además del tejido lógico, las FPGA actuales incorporan numerosos recursos especializados que serían poco eficientes si se implementaran únicamente mediante LUTs. Entre ellos destacan las memorias embebidas, utilizadas para implementar *buffers*, FIFOs, tablas de consulta y otras estructuras de almacenamiento preparadas para funcionar a alta velocidad. En los dispositivos de AMD/Xilinx, estas memorias incluyen bloques BRAM (*Block RAM*) y, en las familias más recientes, también memorias UltraRAM, que ofrecen una mayor capacidad de almacenamiento y permiten implementar estructuras de mayor tamaño sin necesidad de recurrir a memoria externa [12].

Otro recurso ampliamente utilizado son los bloques de procesamiento digital (DSP), diseñados específicamente para ejecutar operaciones aritméticas intensivas como multiplicaciones, sumas acumuladas o filtrado digital. Gracias a estos bloques es posible realizar este tipo de operaciones con una latencia y un consumo muy inferiores a los obtenidos utilizando únicamente lógica programable.

Las FPGA también incorporan gestores de reloj basados en PLL (*Phase-Locked Loop*), encargados de generar y distribuir las señales de reloj del sistema [13]. Estos bloques permiten, entre otras funciones, multiplicar o dividir la frecuencia de reloj e introducir desplazamientos de fase. En el caso de los dispositivos de AMD/Xilinx, se incorporan además bloques MMCM (*Mixed-Mode Clock Manager*), que proporcionan funciones adicionales para la gestión y sincronización de las señales de reloj.

Finalmente, algunas familias de FPGA incorporan procesadores embebidos, dando lugar a plataformas heterogéneas que combinan lógica programable y procesamiento software dentro de un mismo sistema en chip (SoC). Un ejemplo representativo es la familia Zynq UltraScale+, que integra procesadores ARM junto al tejido programable. En el ámbito de las comu-

nicaciones, esta arquitectura permite implementar sistemas complejos como *switches* Ethernet, donde las funciones del plano de datos pueden implementarse directamente en hardware, mientras que otras tareas de configuración, monitorización y gestión pueden ejecutarse mediante software.

Aplicación al aprendizaje MAC

Las redes Ethernet de alta velocidad requieren procesar cientos de millones de paquetes por segundo con latencias del orden de nanosegundos, lo que hace necesario el empleo de arquitecturas hardware altamente paralelas capaces de procesar paquetes al ritmo del enlace [14].

En los switches comerciales basados en ASIC, las tablas MAC suelen implementarse mediante estructuras especializadas de búsqueda rápida, habitualmente basadas en memorias CAM o en mecanismos propietarios optimizados para este propósito. Sin embargo, estos recursos no suelen estar disponibles como bloques nativos en las FPGA. Por este motivo, resulta habitual implementar las tablas MAC utilizando memorias embebidas (BRAM) junto con algoritmos de hashing, que permiten realizar búsquedas exactas con un reducido coste hardware.

En este trabajo se emplea Cuckoo Hashing [15] como estructura de almacenamiento de la tabla MAC debido a que permite realizar búsquedas mediante un número fijo y reducido de accesos a memoria, proporcionando una latencia de consulta determinista independientemente del nivel de ocupación de la tabla.

Proyectos de FPGA en redes

El uso de FPGA en redes Ethernet no se ha limitado únicamente al ámbito teórico, sino que ha dado lugar a numerosos proyectos y plataformas orientadas al desarrollo de infraestructuras de red de altas prestaciones. Tanto en entornos académicos como industriales, las FPGA se han utilizado para construir switches programables, aceleradores de red y sistemas de procesamiento de paquetes capaces de operar a velocidades cada vez mayores. Estos proyectos han servido no solo para mejorar el rendimiento y reducir la latencia en redes de alta velocidad, sino también como plataformas de experimentación para nuevas arquitecturas de conmutación, mecanismos de aprendizaje y técnicas de procesamiento hardware del tráfico.

Algunos de los proyectos de investigación más importantes en este ámbito son:

- NetFPGA: se ha convertido en una de las plataformas académicas más importantes dentro del ámbito de las redes programables basadas en FPGA [16]. El proyecto nació en la Universidad de Stanford, aproximadamente entre 2006 y 2008, con el objetivo de ofrecer a estudiantes e investigadores una plataforma real sobre la que desarrollar switches

Ethernet y routers IP utilizando Verilog. Una de sus principales ventajas era permitir probar diseños hardware directamente sobre redes funcionales, acercando el desarrollo académico a entornos reales de comunicación. Su arquitectura separa claramente el plano de datos, implementado en la FPGA, del plano de control, que se ejecuta mediante software en el sistema anfitrión o en procesadores embebidos. La evolución más destacada del proyecto fue NetFPGA SUME, presentada en 2014, que permite alcanzar capacidades de procesamiento cercanas a los 100 Gbps. Gracias a ello, puede utilizarse como tarjeta de red de altas prestaciones, switch programable, firewall o plataforma de monitorización y medida de tráfico. Además, NetFPGA SUME proporciona un entorno de desarrollo reutilizable y modular que ha sido ampliamente adoptado en universidades y centros de investigación de distintos países. Sobre esta plataforma se han desarrollado aplicaciones como generadores de tráfico a velocidad de línea, sistemas de captura de paquetes con timestamping de alta precisión y prototipos de procesadores RISC integrados en FPGA.

- White Rabbit: es un proyecto de código abierto desarrollado inicialmente en el CERN junto con otros laboratorios europeos, con el objetivo de proporcionar sincronización temporal de muy alta precisión sobre redes Ethernet estándar. El proyecto surgió por la necesidad de sincronizar miles de dispositivos distribuidos a lo largo de varios kilómetros dentro de las instalaciones del acelerador de partículas del CERN, donde era necesario mantener errores temporales inferiores al nanosegundo.

Para conseguirlo, White Rabbit combina el protocolo IEEE 1588 (PTP) con Ethernet Síncrono (SyncE) e incorpora mecanismos propios de compensación de retardo implementados directamente en FPGA. Uno de sus elementos principales es el White Rabbit PTP Core (WRPC), un núcleo hardware reutilizable que puede integrarse en distintos diseños FPGA y que proporciona señales de sincronización y referencia temporal de alta precisión. Gracias a esta arquitectura, White Rabbit alcanza precisiones inferiores al nanosegundo [17] incluso en enlaces de fibra óptica de varios kilómetros.

Aunque inicialmente fue desarrollado para física de partículas, White Rabbit ha terminado utilizándose en otros ámbitos científicos y tecnológicos donde la sincronización precisa es crítica. Entre sus aplicaciones destacan telescopios astronómicos, detectores de neutrinos, sistemas de adquisición de datos y plataformas de instrumentación científica. Además, investigaciones recientes han demostrado la integración de White Rabbit en módulos FPGA que combinan sincronización temporal, procesamiento digital de señales y transmisión Ethernet dentro de un mismo sistema hardware.

- Corundum: es un proyecto de código abierto iniciado por Alex Forencich *et al.* en la Universidad de California con el objetivo de proporcionar una plataforma FPGA para el desarrollo de interfaces de red de altas prestaciones [18]. Corundum implementa una tarjeta de red (*Network Interface Card*, NIC) completamente programable capaz de operar a velocidades de hasta 100 Gbps y superiores, integrando controladores Ethernet, una interfaz PCI Express Gen3, un motor DMA de altas prestaciones y soporte nativo para sincronización temporal mediante IEEE 1588 PTP.

Una de las principales características de Corundum es su arquitectura modular, que incorpora un sistema escalable de gestión de colas capaz de soportar más de diez mil colas independientes, junto con planificadores de transmisión configurables por hardware. El proyecto incluye también un completo entorno de simulación basado en Python que permite validar tanto el hardware como el controlador software.

A diferencia de las soluciones comerciales desarrolladas por fabricantes como Cisco o Arista, Corundum se ha consolidado como una de las plataformas abiertas de referencia para la investigación en arquitecturas de red programables sobre FPGA. Su implementación completa en Verilog y la disponibilidad de un bloque de aplicación específico permiten desarrollar y evaluar nuevas funciones de red reutilizando toda la infraestructura de comunicación ya implementada. Actualmente soporta numerosas plataformas FPGA de AMD/Xilinx e Intel, además de plataformas de investigación como NetFPGA SUME, facilitando la transferencia de desarrollos entre el ámbito académico y aplicaciones industriales.

Además de los proyectos académicos y de investigación, numerosas empresas del sector privado han incorporado FPGAs en sus infraestructuras de red para mejorar el rendimiento, reducir la latencia y ofrecer soluciones más flexibles y programables. Algunas de las más notorias son:

- Cisco: integra FPGAs en sus plataformas de red de alto rendimiento, principalmente para implementar funciones de plano de datos que requieren latencia determinista y flexibilidad de actualización sin rediseño de ASIC [19]. La motivación comercial es la misma que en el ámbito académico: una FPGA puede reconfigurarse, adaptando su funcionalidad hardware para soportar nuevos protocolos o servicios de red sin necesidad de sustituir el dispositivo.
- Arista Networks: emplea FPGAs en sus switches de la serie 7000 para implementar funciones avanzadas del plano de datos que complementan o sustituyen al ASIC principal en determinados escenarios. Entre los usos documentados figura la generación de sellados de tiempo de alta precisión para análisis de latencia en redes financieras, así como

la implementación de funciones de telemetría a velocidades de línea. Arista ha posicionado el uso de FPGAs como parte de su estrategia de diferenciación frente a la rigidez de los ASICs puros.

2.4. Modelo de tráfico en redes: distribución de Zipf

Para intentar parametrizar un tráfico Ethernet real, se ha empleado un modelo de tráfico basado en una distribución de Zipf. Este tipo de distribución permite representar situaciones en las que unos pocos elementos aparecen con mucha frecuencia, mientras que el resto tienen una presencia mucho menor. Este comportamiento es habitual en numerosos sistemas de comunicaciones, donde una pequeña parte de los recursos concentra gran parte de las solicitudes.

Diversos trabajos han observado este comportamiento en redes de datos. En particular, Breslau et al. demostraron que la popularidad de los contenidos solicitados en diferentes sistemas de caché Web puede aproximarse mediante una distribución de Zipf, donde la probabilidad de acceso disminuye conforme aumenta la posición que ocupa el elemento dentro de un ranking de popularidad [20]. De forma similar, otros autores han utilizado este modelo para representar la distribución de accesos en redes centradas en contenidos y estudiar el comportamiento de mecanismos de encaminamiento y almacenamiento [21].

Matemáticamente, la probabilidad asociada al elemento de rango i viene dada por:

$$P(i) = \frac{\frac{1}{i^\alpha}}{\sum_{j=1}^N \frac{1}{j^\alpha}} \quad (2.1)$$

donde:

- N representa el número total de elementos posibles.
- i corresponde a la posición del elemento dentro del ranking de popularidad.
- α controla el grado de concentración del tráfico.

Cuando α toma valores pequeños, las probabilidades se distribuyen de forma relativamente uniforme entre todos los elementos. Sin embargo, a medida que α aumenta, una pequeña fracción de elementos concentra una proporción cada vez mayor de las solicitudes.

Las medidas realizadas sobre sistemas reales muestran que los patrones de acceso suelen presentar valores del parámetro α próximos a 0.7 [20]. Esto refleja un escenario en el que unos pocos elementos concentran una parte importante del tráfico, aunque sigue existiendo un número elevado de elementos con probabilidades de aparición no despreciables.

Por este motivo, en este trabajo se considera $\alpha = 0.7$ como un escenario representativo de tráfico real, complementándolo con valores superiores para estudiar situaciones donde la concentración del tráfico es más notable.

La Figura 2.1 muestra la distribución obtenida para distintos valores de α considerando un espacio muestral de 1000 elementos posibles.

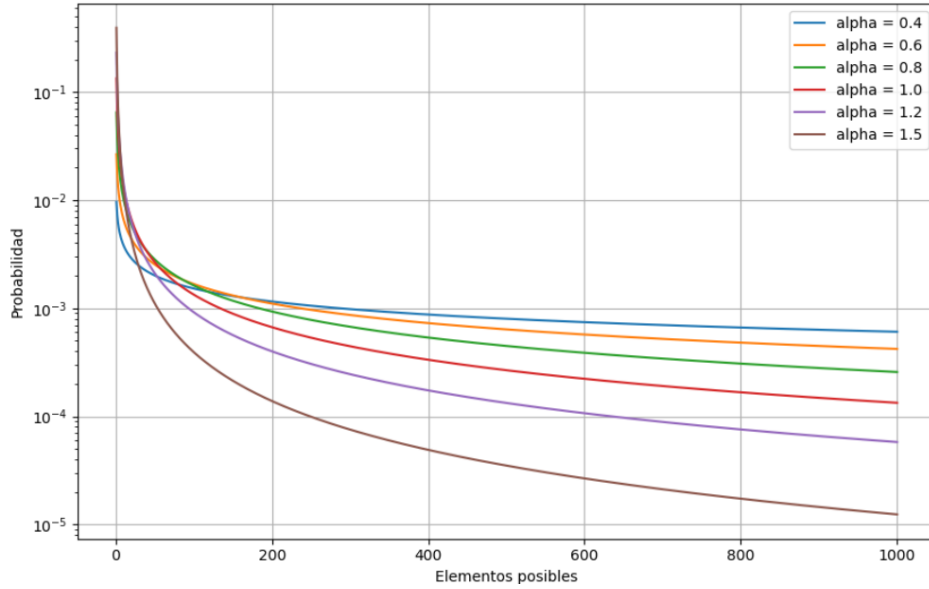


Figura 2.1: Distribución Zipf para distintos valores de α .

Puede observarse que para $\alpha = 0.4$ la distribución presenta una pendiente relativamente suave, permitiendo que un gran número de elementos tenga una probabilidad significativa de aparición. Por el contrario, para valores elevados como $\alpha = 1.2$ o $\alpha = 1.5$, las primeras posiciones concentran la mayor parte del tráfico, mientras que la probabilidad asociada al resto de elementos decrece rápidamente varios órdenes de magnitud.

En el contexto de este trabajo, cada elemento de la distribución se corresponde con una dirección MAC distinta. De esta forma, la distribución Zipf permite modelar escenarios donde algunas direcciones MAC aparecen de manera recurrente, mientras que otras únicamente generan tráfico de forma esporádica.

Capítulo 3

Materiales y métodos

En este capítulo se describen las tecnologías, herramientas y metodología empleadas para llevar a cabo este trabajo. El proceso de desarrollo no se ha basado en un único nivel de abstracción, sino en una metodología de validación progresiva basada en tres etapas: un modelo funcional en Python, una descripción RTL en Verilog HDL y una implementación sobre FPGA.

3.1. Enfoque de modelado

Para comenzar, se desarrolla un modelo funcional en Python utilizando la librería SimPy. Este modelo se emplea como herramienta de pruebas, ya que permite implementar y comparar de forma sencilla las distintas posibles soluciones de mecanismos de aprendizaje dinámico. En esta etapa se estudia cómo se comporta la tabla MAC ante las distintas soluciones. El objetivo principal es decidir qué solución o soluciones tiene sentido trasladar a la etapa hardware.

Una vez analizado el comportamiento del modelo funcional, se desarrolla un modelo RTL en Verilog HDL. Este segundo modelo tiene una finalidad distinta: comprobar que la solución propuesta puede implementarse mediante una descripción RTL sintetizable. En esta etapa se describen explícitamente las señales que conectan los módulos, los registros internos, las colas FIFO y los distintos bloques del sistema. Además, se vuelve a analizar el comportamiento del sistema, lo que permite realizar una validación cruzada entre ambos niveles y comprobar que la descripción HDL mantiene el comportamiento funcional esperado.

Por último, la arquitectura se implementa sobre FPGA. Esta etapa permite comprobar que el diseño sintetizado mantiene el comportamiento observado en simulación y que puede ejecutarse en hardware real. De nuevo, se obtienen resultados equivalentes a partir de los contadores internos del diseño y se comparan con los resultados procedentes de la simulación HDL. Esta validación cruzada permite verificar que la implementación física re-

produce el comportamiento del modelo RTL. Además, esta etapa permite analizar los recursos consumidos en la FPGA por el módulo diseñado y rediseñar la arquitectura si fuera necesario para mejorar su implementación.

De esta forma, cada etapa aporta un papel diferente. El modelo Python facilita la implementación y estudio de las posibles soluciones de forma más sencilla y rápida, el modelo HDL permite verificar su viabilidad mediante una descripción RTL sintetizable y la FPGA permite comprobar su implementación real en hardware. La comparación entre etapas reduce el riesgo de extraer conclusiones a partir de un único entorno de simulación y permite detectar posibles errores mediante la validación cruzada de modelos.

3.2. Herramientas de desarrollo software para simulación funcional

Antes de trasladar las distintas arquitecturas propuestas a una implementación hardware, resulta conveniente disponer de un modelo funcional que permita evaluar rápidamente las distintas alternativas de diseño. Python constituye una herramienta adecuada para este objetivo, ya que es un lenguaje más sencillo que facilita la implementación de los modelos y permite mayor flexibilidad para caracterizar a los modelos propuestos. Además, dispone de un amplio ecosistema de librerías para simulación, tratamiento de datos y representación de resultados.

La principal librería utilizada en este trabajo para la implementación en Python ha sido SimPy, una librería de simulación de eventos discretos [22]. SimPy permite representar un sistema mediante procesos que evolucionan de forma concurrente sobre un mismo entorno de simulación. Cada proceso puede generar eventos, permanecer bloqueado mientras espera a que estos ocurran y reanudar posteriormente su ejecución, reproduciendo de forma sencilla el comportamiento temporal de un sistema sin necesidad de simular cada ciclo de reloj como en Verilog HDL.

Para modelar el paso del tiempo, SimPy proporciona eventos de tipo *timeout*. Mediante instrucciones como `yield env.timeout(n)`, un proceso permanece inactivo durante un determinado intervalo de tiempo simulado antes de continuar su ejecución.

Esta forma de trabajo resulta especialmente adecuada para el estudio y la validación funcional. Además, el desarrollo mediante SimPy facilita su posterior traslado a una implementación hardware en Verilog HDL, ya que cada proceso puede asociarse de forma natural a los distintos bloques del diseño.

3.3. Herramientas de desarrollo hardware

Para el desarrollo hardware del modelo se ha utilizado Verilog HDL, uno de los lenguajes más empleados para el diseño de sistemas digitales [23]. Este lenguaje permite describir el comportamiento del circuito a nivel RTL (*Register Transfer Level*), representando los distintos registros, la lógica combinacional y las conexiones entre módulos.

Para verificar el funcionamiento del diseño se ha empleado *cocotb* [24]. Esta herramienta permite desarrollar el banco de pruebas en Python y comunicarse directamente con el simulador HDL, lo que facilita la reutilización de gran parte del entorno de pruebas desarrollado durante la validación funcional del modelo software. De este modo, las mismas secuencias de direcciones MAC utilizadas en Python pueden aplicarse posteriormente al diseño HDL, simplificando la validación cruzada entre ambas implementaciones.

Como complemento a esta validación se ha utilizado GTKWave [25], una herramienta para la visualización de formas de onda generadas durante la simulación. Esta herramienta resulta especialmente útil, ya que permite ver ciclo a ciclo cómo varían los distintos registros y señales del sistema, facilitando el análisis temporal del diseño y la localización de posibles errores durante el proceso de depuración.

Finalmente, para la síntesis, implementación y generación del *bitstream* se utiliza Vivado [26]. Esta herramienta forma parte del flujo de desarrollo de AMD/Xilinx y permite transformar la descripción RTL en un diseño implementable sobre la FPGA seleccionada. Además de generar el fichero de configuración de la FPGA, Vivado permite analizar la utilización de recursos hardware, como LUTs, registros, BRAM y DSPs, y comprobar el cumplimiento de las restricciones temporales del diseño.

3.4. Plataforma hardware y protocolos de comunicación

La implementación y validación final del diseño se realiza sobre la placa de evaluación ZCU102 de AMD/Xilinx. Esta placa incorpora una FPGA Zynq UltraScale+ MPSoC XCZU9EG y dispone de los recursos necesarios para implementar la arquitectura desarrollada, incluyendo lógica programable, memorias internas e interfaces de comunicación [27]. En este trabajo, la ZCU102 se utiliza tanto para implementar el módulo de aprendizaje dinámico como para almacenar los datos necesarios durante las pruebas y obtener posteriormente los resultados de cada ejecución.

Para realizar las pruebas es necesario establecer una comunicación entre esta FPGA y el ordenador. Esta comunicación se realiza mediante un enlace USB-UART, que permite enviar y recibir información a través de una conexión serie. De esta forma, desde el ordenador es posible cargar los datos

de entrada, configurar los parámetros de la prueba, iniciar la ejecución y recuperar posteriormente los distintos resultados obtenidos.

Sobre este enlace se utiliza XFCP (*Extensible FPGA Control Platform*), desarrollado como una plataforma de control para diseños implementados en FPGA [28]. XFCP proporciona la comunicación entre el software ejecutado en el ordenador y los distintos bloques internos de la FPGA, permitiendo realizar operaciones de lectura y escritura de forma sencilla. En este trabajo se utiliza principalmente como intermediario entre la comunicación serie y la interfaz de control del sistema.

Para acceder a los registros internos se emplea AXI4-Lite, una versión simplificada del protocolo AXI destinada principalmente a la comunicación con periféricos y registros de control [29]. AXI4-Lite permite realizar operaciones individuales de lectura y escritura mediante un conjunto de canales independientes. Cada transferencia se controla mediante señales *VALID* - *READY*, de forma que una operación se completa cuando el emisor presenta información válida y el receptor se encuentra preparado para aceptarla.

La combinación de estos protocolos para establecer la comunicación entre el ordenador y la FPGA es:

$$\text{PC} \rightarrow \text{USB-UART} \rightarrow \text{XFCP} \rightarrow \text{AXI4-Lite} \rightarrow \text{FPGA}$$

3.5. Uso herramientas IA

Durante el desarrollo de este trabajo se han utilizado herramientas de inteligencia artificial como apoyo en distintas tareas como tareas de documentación, consulta técnica y revisión de contenidos.

En concreto, se ha utilizado *Elicit* como herramienta para la búsqueda de documentación científica y la consulta, síntesis y traducción de información relevante para el desarrollo del trabajo.

Por otro lado, *ChatGPT* se ha empleado como apoyo durante la redacción de la memoria, tanto para la creación de algunos diagramas (Figuras 1.1, 4.1, 4.2, 4.3, 4.4 y 5.1) como para revisar y mejorar algunas expresiones escritas y corregir errores ortográficos.

También se ha empleado *ChatGPT* durante el desarrollo de los modelos software y hardware como herramienta de apoyo para consultar información técnica sobre librerías y para facilitar la comprensión de determinados problemas de implementación y depuración.

El uso de estas herramientas ha tenido un carácter auxiliar durante el desarrollo del trabajo. Las decisiones de diseño, la implementación de los modelos, el análisis de los resultados y la elaboración final de la memoria han sido realizados y revisados por el autor.

Capítulo 4

Diseño de soluciones de mejora de aprendizaje dinámico

4.1. Análisis de técnicas de aprendizaje dinámico

El aprendizaje dinámico de direcciones MAC, introducido en el capítulo 2, constituye el mecanismo fundamental mediante el cual un switch construye y mantiene su tabla MAC. Sin embargo, dado que la capacidad de dicha tabla es finita y el número de dispositivos activos en una red puede variar considerablemente a lo largo del tiempo, la decisión de qué entradas debe aprender o priorizar resulta determinante para el rendimiento del sistema. A continuación se analizan las principales políticas de aprendizaje y reemplazo empleadas. A partir de este análisis se justifica la solución implementada, basada en una caché auxiliar y en un estimador de frecuencia (*Count-Min Sketch*).

FIFO (First In, First Out)

La política FIFO es una de las técnicas más simples. Cuando la tabla MAC se llena y se necesita insertar una nueva entrada, se elimina la dirección MAC que fue aprendida en primer lugar. Esta decisión no tiene en cuenta si el dispositivo cuya dirección MAC se ha aprendido sigue activo ni la frecuencia con la que aparece en el tráfico.

Su principal ventaja es la sencillez de implementación. En hardware puede resolverse con un puntero circular que indique la siguiente posición a reemplazar. Esta simplicidad hace que FIFO sea atractiva en diseños donde el coste lógico debe mantenerse bajo. El principal problema es que la política no distingue entre entradas útiles y entradas obsoletas. Por ello, puede expulsar direcciones que siguen generando tráfico de forma frecuente, lo que

aumenta el flooding y reduce la eficiencia del aprendizaje [30].

LRU (Least Recently Used)

La política LRU consiste en reemplazar la entrada que lleva más tiempo sin consultarse. En el contexto de un switch Ethernet, esto implica que las direcciones MAC observadas en los últimos instantes tienen más probabilidad de permanecer en la tabla, mientras que las direcciones inactivas pasan a ser candidatas al desalojo. Para poder implementar esta política, además de almacenar la dirección MAC, es necesario almacenar también el tiempo que lleva una entrada sin consultarse.

Esta idea suele funcionar bien cuando el tráfico presenta localidad temporal, es decir, cuando un dispositivo que acaba de aparecer tiene una probabilidad alta de volver a generar tramas en un intervalo corto. No obstante, LRU también tiene limitaciones. Ante patrones de barrido, ráfagas de direcciones nuevas o tráfico muy disperso, puede terminar sustituyendo entradas todavía relevantes por direcciones que solo se observan una vez. En esos casos, la tabla queda contaminada por accesos puntuales y la eficiencia puede disminuir de forma notable [30].

LFU (Least Frequently Used)

La política LFU utiliza un contador de accesos para cada entrada y reemplaza aquella cuya frecuencia es menor. En escenarios donde el tráfico está muy concentrado en un conjunto reducido de dispositivos, LFU puede resultar más eficaz que LRU, ya que tiende a conservar las direcciones más demandadas.

El inconveniente principal es que la frecuencia acumulada no siempre representa la importancia actual de una entrada. Una dirección antigua puede conservar un contador elevado aunque ya no aparezca en el tráfico. Además, mantener contadores exactos para todas las entradas incrementa el coste de memoria y de lógica, especialmente si se busca una implementación hardware capaz de operar a alta velocidad [31].

SLRU (Segmented LRU) y políticas híbridas

Para combinar información de recencia y frecuencia se propusieron políticas híbridas. Una de ellas es SLRU, que divide la tabla en dos zonas: una región de prueba y una región protegida. Las entradas nuevas se insertan primero en la zona de prueba. Si vuelven a ser accedidas, pasan a la zona protegida. El reemplazo se realiza preferentemente sobre la zona de prueba, de modo que las entradas con evidencia de reutilización quedan mejor preservadas [32].

Esta separación ayuda a filtrar direcciones que aparecen una sola vez y evita que ocupen espacio durante demasiado tiempo. En la misma línea,

ARC (Adaptive Replacement Cache), propuesta por Megiddo y Modha [33], ajusta dinámicamente el peso entre recencia y frecuencia en función del patrón de accesos. Aunque ARC fue diseñada para sistemas de almacenamiento, sus principios también son aplicables a tablas MAC, donde el tráfico puede cambiar con rapidez y no siempre conviene fijar una única política de reemplazo [34].

4.2. TinyLFU

Las políticas anteriores muestran un compromiso común: las soluciones más simples son fáciles de implementar, pero pueden tomar decisiones pobres ante tráfico irregular; las más sofisticadas capturan mejor el comportamiento del sistema, aunque resultan más costosas para una implementación hardware a alta velocidad. En este contexto aparece TinyLFU, una política de admisión basada en frecuencia reciente que intenta conservar las ventajas de LRU y LFU sin mantener contadores exactos para todos los elementos.

Contexto y motivación

TinyLFU [35] fue propuesto por Gil Einziger y Roy Friedman en 2014. Su aplicación original no estaba centrada en switches Ethernet, sino en cachés de servidores y sistemas distribuidos. En estos entornos, el número de objetos distintos suele ser mucho mayor que la capacidad de la caché, y las distribuciones de acceso acostumbra a seguir leyes de potencia, como ocurre con las distribuciones Zipf.

Los autores observaron que políticas como LRU pueden admitir objetos que solo se consultan una vez, desplazando elementos más populares. Como resultado, una parte de la memoria queda ocupada por datos con baja probabilidad de reutilización. TinyLFU aborda este problema introduciendo una decisión de admisión: antes de insertar un elemento, se estima si su frecuencia justifica reservar espacio para él.

Principio de funcionamiento

La idea central de TinyLFU consiste en separar la decisión de admisión de la decisión de expulsión. Las políticas clásicas suelen decidir al mismo tiempo qué elemento sale y cuál entra, es decir, son políticas de reemplazo.

TinyLFU añade un filtro previo que evalúa si el nuevo elemento merece almacenarse. Antes de insertar un objeto en la caché, se compara su frecuencia estimada con la frecuencia estimada del candidato que sería desalojado.

El proceso puede resumirse en los siguientes pasos:

1. Se estima la frecuencia del objeto entrante.
2. Se estima la frecuencia del candidato a ser desalojado.

3. Se comparan ambas frecuencias.
4. Se admite únicamente el elemento con mayor frecuencia estimada.

Con este criterio, la caché deja de comportarse como una memoria que almacena todo lo que observa. En su lugar, actúa como un filtro selectivo que intenta reservar espacio para elementos con mayor probabilidad de reutilización.

Estructuras probabilísticas empleadas

Para estimar las frecuencias de acceso sin mantener un contador exacto para cada elemento, TinyLFU emplea una estructura probabilística de conteo basada en un Count-Min Sketch (CMS) [36]. Además, el algoritmo original incorpora un Bloom Filter [37], denominado Doorkeeper, que actúa como filtro para los elementos observados por primera vez, evitando reservar contadores para aquellos cuya probabilidad de reutilización es muy baja. La combinación de ambas estructuras permite reducir significativamente el consumo de memoria manteniendo una estimación precisa de las frecuencias.

Bloom Filter

Un Bloom Filter es una estructura probabilística que permite comprobar si un elemento ha sido observado previamente. Está formado por un vector de bits inicializado a cero y por varias funciones hash independientes. Cuando se inserta un elemento, cada función hash calcula una posición del vector y el bit correspondiente se pone a 1.

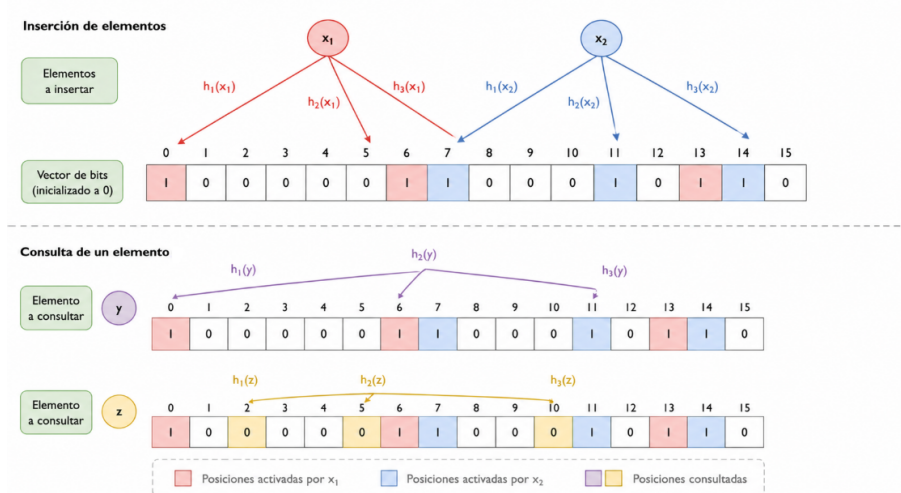


Figura 4.1: Funcionamiento filtro tipo Bloom

La Figura 4.1 muestra un ejemplo. En la parte superior, los elementos x_1 y x_2 se procesan mediante tres funciones hash. Cada una selecciona una posición del vector de bits, que pasa a valer uno. Como varios elementos pueden activar la misma posición, las colisiones forman parte natural del funcionamiento de la estructura.

Para consultar un elemento se aplican de nuevo las mismas funciones hash:

- Si alguna de las posiciones consultadas contiene un cero, el elemento no ha sido insertado.
- Si todas contienen un uno, el elemento probablemente pertenece al conjunto.

La parte inferior de la Figura 4.1 ilustra estos dos casos. El elemento y encuentra todas sus posiciones activadas, por lo que el filtro responde de forma positiva. Esta respuesta puede ser un falso positivo, ya que otros elementos podrían haber activado esos bits. En cambio, si una de las posiciones asociadas al elemento z contiene un cero, puede asegurarse que dicho elemento no fue insertado.

Los Bloom Filters, por tanto, admiten falsos positivos, pero no falsos negativos. Su bajo consumo de memoria y el hecho de que las operaciones tengan coste constante los hacen adecuados para sistemas de red de alta velocidad, donde la latencia y el uso de recursos son factores críticos.

Count-Min Sketch

El Count-Min Sketch (CMS) [36] extiende esta idea al problema de estimar frecuencias dentro de un flujo de datos. La estructura se organiza como una matriz de contadores con varias filas y columnas. Cada fila tiene asociada una función hash distinta.

Cuando se observa un elemento, cada función hash calcula una posición en su fila correspondiente y el contador seleccionado se incrementa. Así, la frecuencia de un elemento no se guarda en una única celda, sino que queda representada por varios contadores en varias filas.

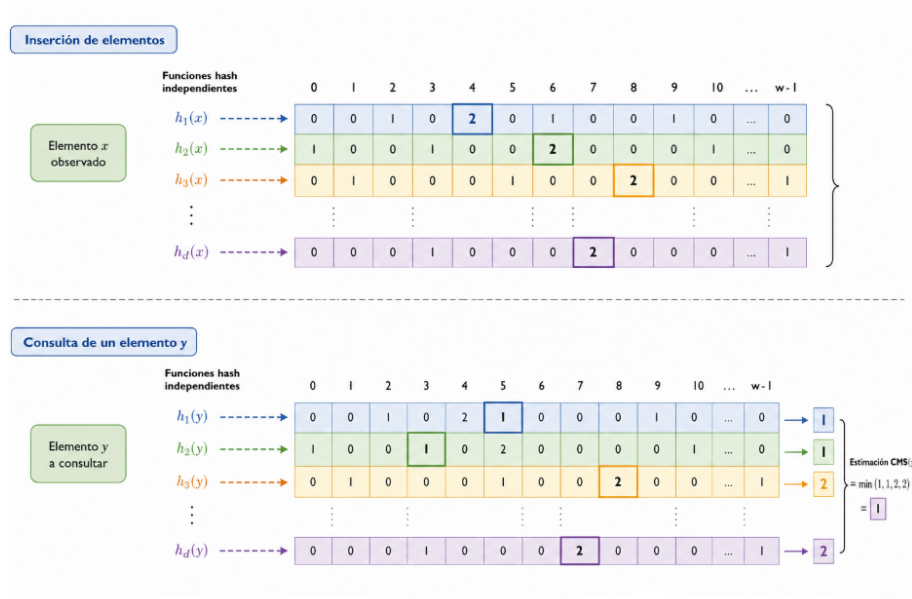


Figura 4.2: Funcionamiento Count-Min Sketch

La Figura 4.2 representa este proceso. En la parte superior, un elemento x actualiza una posición en cada fila. Para estimar después la frecuencia de otro elemento, como y , se aplican de nuevo las funciones hash y se leen los contadores correspondientes.

La estimación se obtiene tomando el mínimo de todos los contadores consultados:

$$\hat{f}(y) = \min_{1 \leq i \leq d} C[i, h_i(y)]$$

donde d es el número de filas, h_i representa la función hash de la fila i y $C[i, h_i(y)]$ es el contador seleccionado.

El uso del mínimo reduce el efecto de las colisiones. Si dos elementos comparten una posición, el contador puede sobreestimar la frecuencia real. Sin embargo, es poco probable que todas las funciones hash colisionen de la misma manera, por lo que el mínimo suele ofrecer una aproximación más cercana al valor real.

El Count-Min Sketch presenta dos propiedades relevantes:

- La frecuencia estimada nunca es inferior a la frecuencia real.
- La estimación puede ser superior a la real debido a colisiones.

Estas propiedades son especialmente útiles en hardware. El CMS requiere poca memoria, permite actualizaciones y consultas en tiempo constante, y sus filas pueden procesarse en paralelo. Por ello, encaja bien en arquitecturas FPGA orientadas a procesar tráfico a velocidad de línea.

Mecanismo de envejecimiento (Aging)

Un problema de los contadores acumulativos es que pueden conservar durante demasiado tiempo la influencia de accesos antiguos. Una dirección muy frecuente en el pasado podría seguir teniendo una estimación elevada aunque ya no aparezca en el tráfico actual.

Para evitarlo, TinyLFU incorpora un mecanismo de envejecimiento (*aging*). Periódicamente, los contadores del Count-Min Sketch se dividen entre dos:

$$f_i \leftarrow \frac{f_i}{2}$$

Esta operación aplica un decaimiento progresivo a las frecuencias almacenadas. De esta manera, la estructura refleja mejor una ventana temporal de accesos recientes sin guardar de forma explícita todo el historial.

Desde el punto de vista hardware, esta operación resulta especialmente eficiente, ya que puede implementarse mediante desplazamientos binarios (*bit shifting*), minimizando el coste computacional y el consumo de recursos.

Aplicación al aprendizaje MAC en switches Ethernet

En este trabajo se toma la idea principal de TinyLFU como punto de partida para diseñar un mecanismo que reduzca las escrituras sobre la tabla MAC durante el aprendizaje dinámico de un conmutador Ethernet. No se implementa el algoritmo TinyLFU de forma completa, sino que se adapta su filosofía al problema del aprendizaje MAC utilizando dos de sus elementos fundamentales: una pequeña caché situada delante de la tabla MAC y un mecanismo de estimación de frecuencia basado en un Count-Min Sketch (CMS).

La idea es sencilla. Antes de acceder a la tabla MAC, cada dirección MAC de origen se consulta en la caché. Si la dirección ya se encuentra almacenada (*cache hit*), se considera que ha sido aprendida recientemente y no es necesario realizar una nueva escritura sobre la tabla MAC. En cambio, cuando la dirección no se encuentra en la caché, se produce un *cache miss*, la dirección continúa el proceso habitual de aprendizaje y se genera la correspondiente petición de escritura sobre la tabla MAC.

De esta forma, la caché actúa como un filtro de direcciones MAC recientemente observadas. El resultado es una reducción del número de escrituras repetitivas y, por tanto, una menor presión sobre las estructuras de almacenamiento.

Para mantener la caché lo más optimizada posible, se incorpora un Count-Min Sketch, encargado de estimar la frecuencia de aparición de cada dirección MAC. A partir de esta estimación, el sistema decide si una dirección merece reemplazar a otro elemento dentro de la caché.

En este punto aparece la principal diferencia respecto a TinyLFU. En el algoritmo original, la política de admisión decide qué elementos se incor-

poran a la caché. En la propuesta desarrollada en este trabajo, la política basada en frecuencia únicamente decide qué direcciones reemplazan a otro elemento en la caché. La tabla MAC mantiene en todo momento el funcionamiento habitual del aprendizaje dinámico: cualquier dirección que produzca un *cache miss* genera una operación de escritura sobre la misma. El CMS únicamente interviene para mantener la caché lo más eficiente posible.

Como consecuencia, la caché tiende a conservar las direcciones MAC más activas del tráfico, evitando que direcciones de aparición esporádica ocupen espacio innecesariamente. Esto incrementa el número de *cache hits* y reduce significativamente las escrituras redundantes sobre la tabla MAC.

Esta aproximación resulta especialmente adecuada para redes Ethernet, donde el tráfico suele seguir distribuciones de tipo Zipf y un pequeño conjunto de direcciones MAC concentra una parte importante de las transmisiones. En estas condiciones, adaptar la idea de TinyLFU al aprendizaje MAC permite reducir la carga sobre la tabla de direcciones sin introducir estructuras hardware de gran complejidad.

4.3. Modelado funcional y validación del algoritmo

Arquitecturas propuestas

Para implementar el modelo en Python, se van a definir tres sistemas distintos para su posterior estudio.

En primer lugar, se implementa el modelo base, que representa el comportamiento de una tabla MAC en un switch Ethernet convencional sin ningún mecanismo de filtrado. En este caso, cada trama generada se envía directamente hacia la cola de escritura de la tabla MAC, de manera que toda dirección MAC observada genera una petición de aprendizaje. Este modelo constituye la referencia frente a la que se comparan el resto de propuestas, permitiendo cuantificar el coste asociado a aprender todas las direcciones MAC presentes en el tráfico.

En segundo lugar, se desarrolla un modelo en el que se introduce una pequeña caché delante de la tabla MAC. Cada dirección MAC generada se consulta inicialmente en la caché. Si la dirección MAC se encuentra en la caché, se produce un hit, la trama queda filtrada y no alcanza la tabla MAC. Por el contrario, si la dirección MAC no se encuentra en la caché, se produce un *miss*. En este caso la trama continúa hacia la cola de escritura de la tabla MAC y hacia la cola de escritura de la propia caché. De esta forma, la caché actúa como un filtro temporal que evita escrituras repetitivas de direcciones recientemente observadas.

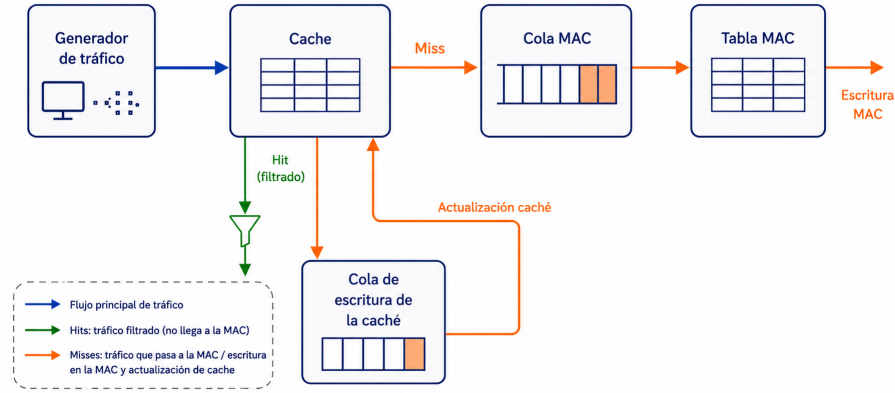


Figura 4.3: Sistema con caché.

Por último, se implementa el modelo completo *CMS + Caché*. Este modelo incorpora un Count-Min Sketch delante de la caché con el objetivo de estimar la frecuencia de aparición de cada dirección MAC. El mecanismo de aprendizaje de la tabla MAC permanece igual respecto al modelo anterior: cualquier dirección MAC que no se encuentre en la caché genera una petición de escritura sobre la tabla MAC.

Sin embargo, la decisión de almacenar una dirección en la caché no depende únicamente de que se produzca un *miss*. En el modelo propuesto se introduce una política de admisión basada en dos criterios: la ocupación actual de la caché y la estimación de frecuencia proporcionada por el CMS.

Cuando una dirección MAC no se encuentra en la caché, la idea principal es admitirla directamente mientras la caché tenga capacidad suficiente. La intervención del CMS solo resulta necesaria cuando la caché alcanza un elevado nivel de ocupación, ya que en ese caso la inserción de una nueva dirección puede provocar reemplazos en la caché.

En una tabla basada en Cuckoo hashing no es necesario alcanzar una ocupación del 100 % para que aparezcan estos efectos. Debido a las colisiones entre posiciones candidatas, una nueva inserción puede requerir reubicaciones aunque todavía existan huecos libres en otras zonas de la caché. Por este motivo, en el modelo se utiliza un umbral de ocupación. Mientras la ocupación se mantiene por debajo de dicho umbral, la dirección se admite directamente. Cuando se alcanza, el CMS actúa como política de admisión y solo permite escribir en caché aquellas direcciones cuya frecuencia estimada supera el umbral fijado.

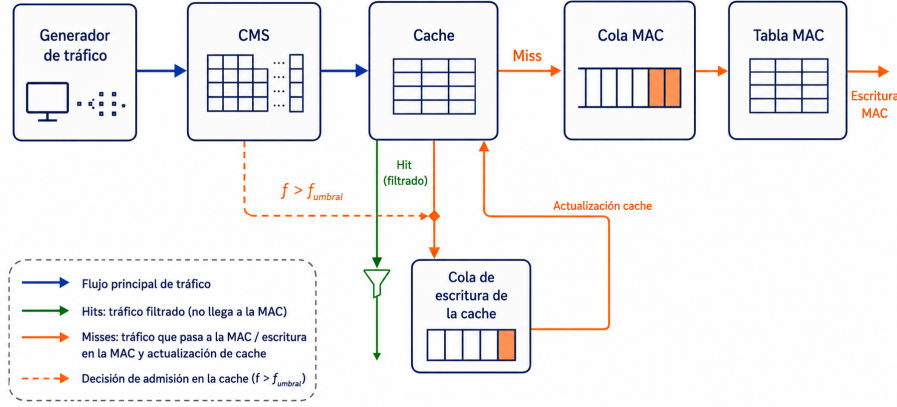


Figura 4.4: Sistema con CMS + caché.

Modelo de tráfico y generación de tramas

Como se explicó en la sección 2.4, la distribución estadística de las direcciones MAC se modela mediante una distribución de Zipf. Además de este comportamiento estadístico, se ha incorporado un modelo temporal para reproducir la llegada de tramas a una velocidad de enlace determinada.

Las simulaciones se han desarrollado considerando una frecuencia de reloj de trabajo fija de 100 MHz, equivalente a un periodo de reloj de 10 ns. A partir de esta frecuencia y de la velocidad del enlace configurada en cada simulación, se calcula el número de ciclos necesarios para transmitir una trama Ethernet de tamaño fijo.

Para aproximar el intervalo entre la generación de dos tramas consecutivas, se considera el tiempo de serialización de una trama Ethernet. Si L representa la longitud de la trama, expresada en bits, y R la velocidad del enlace (bit/s), dicho intervalo puede calcularse como:

$$t_f = \frac{L}{R} \quad (4.1)$$

A partir de este valor, el número de ciclos de reloj necesarios para transmitir una trama puede obtenerse como:

$$N_{ciclos} = t_f \cdot f_{clk} \quad (4.2)$$

donde f_{clk} representa la frecuencia de reloj utilizada en la simulación.

En este trabajo se ha considerado un tamaño de trama constante de 64 bytes, correspondiente al tamaño mínimo permitido en Ethernet. Esta elección permite evaluar el sistema en el escenario más exigente desde el punto de vista de la tasa de paquetes, ya que, para una misma velocidad de enlace, las tramas de menor tamaño generan el mayor número de llegadas por segundo.

En las simulaciones se consideran dos velocidades de enlace con objetivos diferentes. La velocidad de 2 Gbps representa un escenario en el que el sistema dispone de margen suficiente para procesar las peticiones generadas, por lo que apenas aparecen descartes por saturación de la cola de escritura de la tabla MAC. Por otro lado, 51,2 Gbps aproxima el límite de capacidad del modelo, ya que equivale a la llegada de una trama por ciclo de reloj. De esta forma, es posible comparar el comportamiento de las arquitecturas tanto en condiciones de baja presión como en un escenario cercano a la máxima carga a la que es capaz de trabajar el sistema.

Velocidad	Bits/ciclo	Tamaño trama	Ciclos/trama
2 Gbps	20	64 bytes (512 bits)	25.6
51,2 Gbps	512	64 bytes (512 bits)	1

Tabla 4.1: Parámetros de generación de tráfico utilizados en las simulaciones.

Definición de métricas

Para evaluar el comportamiento de las arquitecturas propuestas se han definido varias métricas. Cada una de ellas permite estudiar un aspecto distinto del sistema.

En primer lugar, se analiza la capacidad de la caché para filtrar accesos repetidos. A continuación, se estudian los descartes producidos por la saturación de la cola de la tabla MAC. Estas dos primeras métricas permiten caracterizar el caudal de tráfico que atraviesa el sistema y observar cómo responde cada arquitectura ante distintos patrones de tráfico.

Por último, se evalúan el porcentaje de direcciones aprendidas y la eficiencia de las operaciones de escritura. Estas dos métricas se centran directamente en el comportamiento de la tabla MAC, ya que permiten comprobar cuántas direcciones consigue aprender y cuántas de las escrituras que procesa aportan realmente información nueva.

Estas métricas se calculan a partir de los contadores obtenidos durante cada simulación. De esta forma, es posible comparar los modelos sobre un mismo tráfico de entrada y determinar qué efecto introduce el mecanismo de filtrado.

Porcentaje de filtrado de la caché

El porcentaje de filtrado representa la proporción de tramas cuya dirección MAC origen ya se encuentra almacenada en la caché. Estas tramas producen un *cache hit* y no generan una nueva petición de escritura sobre la tabla MAC.

La métrica se calcula como:

$$P_{\text{filtrado}} = \frac{N_{\text{hits}}}{N_{\text{hits}} + N_{\text{misses}}} \cdot 100 \quad (4.3)$$

donde N_{hits} representa el número de aciertos en la caché y N_{misses} el número de direcciones que no estaban almacenadas en ella.

Un porcentaje elevado indica que la caché consigue eliminar una parte importante de las escrituras repetitivas. Por el contrario, un valor reducido indica que gran parte del tráfico continúa avanzando hacia la tabla MAC.

Porcentaje de descartes por saturación en la tabla MAC

Cuando una trama debe escribirse en la tabla MAC, el módulo de escritura intenta insertar la dirección MAC en la cola correspondiente a la tabla MAC. En el modelo sin caché este proceso tiene lugar para todas las tramas generadas, mientras que en el modelo con caché solo ocurre tras un *cache miss*. Si en ese instante la cola está llena, la solicitud no puede procesarse y se contabiliza como un descarte por saturación.

La fórmula para medir esta métrica es:

$$P_{\text{descartes}} = \frac{N_{\text{descartes MAC}}}{N_{\text{tramas generadas}}} \cdot 100 \quad (4.4)$$

donde $N_{\text{descartes MAC}}$ corresponde al número de solicitudes que deberían avanzar hacia la tabla MAC pero no consiguen entrar en su cola de escritura, y $N_{\text{tramas generadas}}$ es el número total de tramas ofrecidas al sistema.

Esta métrica permite estudiar si la tabla MAC es capaz de procesar todo el tráfico de entrada o se tienen que descartar paquetes por saturación del sistema.

Porcentaje de aprendizaje

El porcentaje de aprendizaje mide qué parte de las direcciones MAC distintas generadas durante la simulación llega a almacenarse correctamente en la tabla MAC.

Se calcula mediante la siguiente expresión:

$$P_{\text{aprendizaje}} = \frac{N_{\text{MAC aprendidas}}}{N_{\text{MAC generadas}}} \cdot 100 \quad (4.5)$$

donde $N_{\text{MAC aprendidas}}$ representa el número de direcciones MAC distintas presentes en la tabla al finalizar la simulación y $N_{\text{MAC generadas}}$ el número de direcciones distintas que han aparecido en el tráfico generado.

Esta métrica permite comprobar si el filtrado previo afecta al funcionamiento normal del aprendizaje dinámico. Un valor cercano al 100 % indica que el sistema ha sido capaz de aprender prácticamente todas las direcciones generadas. En cambio, un valor inferior refleja que algunas direcciones no

han podido almacenarse, normalmente debido a descartes por saturación del sistema.

Eficiencia de escritura

El porcentaje de aprendizaje indica cuántas direcciones distintas termina almacenando la tabla MAC, pero no permite conocer cuántas operaciones de escritura han sido necesarias. Por este motivo, también se define una métrica de eficiencia.

La eficiencia de escritura representa la proporción de intentos que terminan incorporando una nueva dirección a la tabla MAC:

$$P_{\text{eficiencia}} = \frac{N_{\text{escrituras útiles}}}{N_{\text{intentos de escritura}}} \cdot 100 \quad (4.6)$$

En esta expresión, $N_{\text{escrituras útiles}}$ corresponde al número de direcciones nuevas escritas correctamente. Por su parte, $N_{\text{intentos de escritura}}$ incluye las solicitudes que alcanzan el proceso de escritura de la tabla MAC.

Una eficiencia elevada indica que la mayoría de las operaciones procesadas por la tabla aportan una nueva entrada. Una eficiencia baja muestra que existe una proporción importante de intentos redundantes o fallidos. Por tanto, esta métrica permite comprobar si la caché consigue que las escrituras que alcanzan la tabla MAC sean realmente necesarias.

La eficiencia debe analizarse junto con el porcentaje de aprendizaje y los descartes por saturación. En situaciones de alta carga, una eficiencia elevada no implica necesariamente que el sistema haya aprendido todas las direcciones generadas, ya que parte de las solicitudes puede haber sido descartada antes de alcanzar el escritor de la tabla MAC.

Implementación de los modelos funcionales en Python

El modelo funcional utilizado en este trabajo parte de una implementación previa de un switch Ethernet[15] sin ningún mecanismo de filtrado. A partir de este modelo base se han incorporado los nuevos módulos propuestos para reducir las escrituras sobre la tabla MAC. En concreto, se ha añadido una caché situada delante de la tabla de aprendizaje, un Count-Min Sketch para estimar la frecuencia de las direcciones MAC y la lógica de control necesaria para integrar ambos componentes en el funcionamiento del switch. De esta forma, la comparación entre la arquitectura original y la propuesta puede realizarse sobre una misma base de implementación.

En el Apéndice A se encuentra el código completo implementado, así como los test utilizados.

Implementación del generador de tráfico

La primera parte del modelo en Python consiste en la generación de las tramas que posteriormente serán procesadas por el sistema. Cada trama se representa mediante una estructura de datos sencilla, donde se almacenan el instante de generación, el puerto de entrada y las direcciones MAC origen y MAC destino. En el modelo desarrollado, la dirección relevante para el aprendizaje es la MAC origen, ya que es la dirección a partir de la cual el switch organiza el direccionamiento de la trama.

```
1 @dataclass(frozen=True)
2 class Packet:
3     t: int
4     in_port: int
5     src_mac: int
6     dst_mac: int
```

Para generar las direcciones MAC se define inicialmente un espacio muestral de direcciones MAC posibles, denominado `mac_space`. A partir de este valor se construye un conjunto de direcciones MAC aleatorias dentro de este espacio muestral. Este conjunto representa todas las direcciones que pueden aparecer durante una simulación concreta.

```
1 def build_hdl_mac_pool(mac_space: int, rnd: random.Random)
2     -> List[int]:
3     return rnd.sample(range(1 << 48), mac_space)
```

La selección de las direcciones no se realiza de forma uniforme, sino siguiendo la distribución Zipf explicada anteriormente. El parámetro α controla el grado de concentración del tráfico: cuanto mayor es su valor, mayor es la probabilidad de que se repitan las direcciones más frecuentes.

```
1 def build_zipf_weights(mac_space: int, alpha: float) ->
2     List[float]:
3     ranks = list(range(1, mac_space + 1))
4     return [1.0 / (rank** alpha) for rank in ranks]
```

Una vez definidos el conjunto de direcciones posibles y los pesos asociados a cada una, la función `traffic_generator` selecciona una dirección MAC origen mediante `rnd.choices`. Además, se asigna aleatoriamente un puerto de entrada dentro del rango definido por `port_width`.

```
1 src = rnd.choices(
2     population=mac_pool,
3     weights=zipf_weights,
4     k=1,
5 ) [0]
```

```

7 port = rnd.randint(1, (1 << port_width) - 1)
8
9 return Packet(
10     t=t,
11     in_port=port,
12     src_mac=src,
13     dst_mac=0,
14 )

```

La generación temporal de tramas se controla dentro del proceso `source_process`. Para adaptar la generación de paquetes a la velocidad configurada, se utiliza un acumulador de bits. En cada ciclo se añade el número de bits que llegarían al sistema según la tasa de transmisión y la frecuencia de reloj. Cuando el acumulador supera el tamaño de una trama, se genera un nuevo paquete.

```

1 bit_accumulator += self.bits_per_cycle
2
3 if bit_accumulator >= self.packet_bits:
4     bit_accumulator -= self.packet_bits
5     sel_port = self.tdma.next_port()

```

El valor de `bits_per_cycle` se obtiene a partir de la velocidad de transmisión y la frecuencia de reloj del sistema.

```

1 bits_per_packet = self.packet_size_bytes * 8
2 bits_per_cycle = self.bitrate_bps / self.clk_freq_hz
3
4 self.packet_bits = bits_per_packet
5 self.bits_per_cycle = bits_per_cycle
6 self.cycles_per_packet = bits_per_packet / bits_per_cycle

```

Antes de generar definitivamente la trama, el modelo selecciona el puerto de entrada mediante un planificador rotativo (*round-robin*), que asigna de forma secuencial un turno a cada uno de los puertos. Además, cada puerto tiene asociada una probabilidad de llegada, de modo que durante su turno solo se genera una nueva trama si se cumple dicha probabilidad. De esta forma, la carga de tráfico asociada a cada puerto puede ajustarse mediante el vector `p_arrivals`.

```

1 sel_port = self.tdma.next_port()
2
3 if self.rnd.random() <= self.p_arrivals[sel_port - 1]:
4     pkt = traffic_generator(
5         t=t,
6         mac_pool=self.mac_pool,
7         zipf_weights=self.zipf_weights,
8         rnd=self.rnd,
9         port_width=self.port_width,

```

```
10 )
```

Cada vez que se genera una trama, el modelo actualiza los contadores necesarios para calcular las distintas métricas definidas anteriormente.

```
1 generated_packets += 1
2
3 self.st.offered += 1
4 self.seen_src.add(pkt.src_mac)
5 self.src_counts[pkt.src_mac] =
    self.src_counts.get(pkt.src_mac, 0) + 1
```

Finalmente, el proceso avanza un ciclo de simulación mediante `yield self.env.timeout(1)`. Esta instrucción permite a `SimPy` modelar el paso del tiempo y coordinar este generador con el resto de procesos del sistema.

```
1 yield self.env.timeout(1)
2 t += 1
```

Implementación de la caché

La caché se introduce como un filtro previo a la tabla MAC. Su función es evitar que direcciones que ya han aparecido recientemente vuelvan a generar escrituras redundantes en la tabla MAC.

Para mantener una correspondencia funcional con la futura implementación hardware, la caché se modela mediante la clase `CuckooHashingTableParallel`, definida en `Hash_table.py` y reutilizada a partir de la implementación propuesta por Carlos Megías [15]. En este trabajo no se modifica el funcionamiento interno de dicha clase; se utiliza como bloque parametrizable de tabla hash paralela.

```
1 self.cache_table = CuckooHashingTableParallel(
2     key_width=48,
3     cell_count=recent_cache_size,
4     table_count=cache_cuckoo_table_count,
5     bin_size=cuckoo_bin_size,
6     max_loop=cache_cuckoo_max_loop,
7     hash_type="LFSR",
8     insertion_initial_filter=1,
9     insertion_initial_strategy=0,
10    insertion_reallocate_filter=1,
11    insertion_reallocate_strategy=0,
12 )
```

En la configuración utilizada, `key_width` se fija a 48 bits, ya que la clave almacenada es una dirección MAC. El tamaño de la caché se define mediante `recent_cache_size`, mientras que `cache_cuckoo_table_count` establece

el número de tablas paralelas. En las simulaciones se emplea una caché de 4096 posiciones, organizada en 4 tablas paralelas. Esta caché representa únicamente una pequeña fracción de la capacidad de la tabla MAC principal de 16384 posiciones, permitiendo evaluar hasta qué punto una estructura de memoria reducida es capaz de eliminar escrituras redundantes sobre la tabla MAC.

En la política de inserción, los filtros inicial y de reubicación priorizan tablas con posiciones libres, y la estrategia de selección escoge la primera candidata disponible.

Desde el modelo desarrollado solo se necesitan dos operaciones de la tabla: `lookup` para consultar si la dirección está presente e `insert` para almacenar una nueva dirección. La lectura se realiza sobre la dirección MAC origen de la trama:

```
1 cache_hit = (  
2     self.cache_table.lookup(pkt.src_mac)  
3     is not None  
4 )
```

Si `lookup` devuelve un valor distinto de `None`, se considera que la trama se encuentra en la caché y se filtra. Si `lookup` devuelve `None`, la dirección no está almacenada, por lo que la trama continúa hacia la cola MAC. En Python, la consulta a la caché se obtiene de forma instantánea. Sin embargo, para modelar de manera fiel el comportamiento del hardware, se asume que el resultado de dicha consulta no está disponible hasta transcurridos cuatro ciclos de reloj, reproduciendo así la latencia de la implementación hardware del módulo de la tabla hash.

```
1 cache_lookup_latency_cycles: int = 4
```

A diferencia del proceso de lectura, el proceso de escritura en la caché no se realiza en un número fijo de ciclos. Esto es debido a las colisiones y reubicaciones que se producen en el Cuckoo hashing.

Antes de escribir en la caché se calcula el número de reubicaciones que requeriría la inserción. Para ello se realiza una inserción no efectiva, mediante el parámetro `effective=False`. De esta forma se obtiene la latencia asociada a la operación sin modificar todavía el contenido real de la caché.

```
1 total_write_cycles = self._write_latency_from_loops(  
2     loops,  
3     first_write_cycles=self.cache_first_write_cycles,  
4     next_write_cycles=self.cache_next_write_cycles,  
5 )
```

La latencia efectiva utilizada para la escritura de caché es:

$$T_{\text{cache}} = 5 + 4 \cdot N_{\text{loops}} \quad (4.7)$$

El término N_{loops} representa el número de reubicaciones necesarias para realizar la escritura de la dirección MAC. En la versión final del modelo, la latencia de escritura de la caché se ha alineado con la latencia propia de la tabla hash utilizada en HDL. Por tanto, la primera operación de escritura tiene un coste de cinco ciclos y cada reubicación adicional añade cuatro ciclos.

Implementación del Count-Min Sketch

El modelo completo añade un *Count-Min Sketch* (CMS) para estimar la frecuencia de aparición de cada dirección MAC. Esta estructura se utiliza como apoyo a la política de admisión de la caché, no como sustituto de la tabla MAC. Su función es decidir qué direcciones merece la pena almacenar en caché cuando esta se encuentra cerca de su capacidad máxima.

En las simulaciones se emplean cuatro filas, 2048 contadores por fila y contadores de 8 bits. Además, se define un umbral de frecuencia igual a 2 y un umbral de ocupación de la caché del 90 %.

```
1 cms_d: int = 4
2 cms_w: int = 2048
3 cms_counter_bits: int = 8
4 cms_threshold: int = 2
5 cms_cache_admission_fill_threshold: float = 0.90
```

Mientras la ocupación de la caché se mantiene por debajo del umbral fijado, las direcciones MAC que producen un *miss* se admiten directamente, aprovechando el espacio disponible. Una vez superado el umbral de ocupación, aumenta la probabilidad de que se produzcan colisiones debido al funcionamiento de *Cuckoo Hashing*. En ese punto, el CMS comienza a intervenir como política de admisión.

La estimación de frecuencia se lee al comienzo del procesamiento de la trama. Sin embargo, esta lectura no produce todavía ninguna decisión, ya que el sistema debe esperar a conocer si la dirección produce un *hit* o un *miss* en la caché.

```
1 cms_estimate_before = (
2     self.cms.read_estimate(pkt.src_mac)
3     if self.cms is not None
4     else 0
5 )
```

Solo tras confirmar que la dirección MAC no se encuentra en la caché se actualiza el CMS y se calcula la frecuencia estimada que se utilizará para

decidir la admisión. De esta forma, el CMS se actualiza únicamente con direcciones que han producido un *miss*, manteniendo la caché centrada en las direcciones que realmente generan nuevas solicitudes hacia la ruta de aprendizaje.

```

1  cms_estimate_after = (
2      reg_s4.cms_estimate_before + 1
3      if self.cms is not None
4      else 0
5  )
6
7  self.cms.update(key)
8  self.st.cms_updates += 1

```

La decisión de escritura en caché depende de dos condiciones. Si la ocupación de la caché es inferior al 90 %, la dirección se admite directamente. Si la ocupación ya ha superado ese valor, solo se admite cuando la frecuencia estimada por el CMS alcanza el umbral configurado.

```

1  if self._cache_fill_ratio() <
2      self.cms_cache_admission_fill_threshold:
3      should_update_cache = True
4  elif cms_estimate_after >= self.cms_threshold:
5      should_update_cache = True
6  else:
7      should_update_cache = False

```

Este criterio permite combinar dos comportamientos. Al principio, la caché se llena sin aplicar restricciones, ya que todavía existe espacio disponible. Cuando la ocupación es elevada, el CMS evita que direcciones de aparición esporádica desplacen a otras que se repiten con mayor frecuencia.

Implementación de la tabla MAC

La tabla MAC es el bloque encargado de almacenar las distintas direcciones MAC para gestionar el direccionamiento del switch.

La implementación de la tabla MAC también se realiza con `CuckooHashingTableParallel`. La tabla MAC utilizada contiene 16384 entradas distribuidas en 4 tablas paralelas. Esta configuración proporciona una capacidad suficientemente elevada para que las colisiones y los problemas de almacenamiento no tengan impacto durante las simulaciones. Así, los resultados obtenidos reflejan principalmente el comportamiento del mecanismo de filtrado propuesto, y no las limitaciones de capacidad de la propia tabla MAC.

```

1  self.mactable = CuckooHashingTableParallel(
2      key_width=48,
3      cell_count=mac_cap,

```

```

4     table_count=mac_cuckoo_table_count,
5     bin_size=cuckoo_bin_size,
6     max_loop=mac_cuckoo_max_loop,
7     hash_type="LFSR",
8     insertion_initial_filter=1,
9     insertion_initial_strategy=0,
10    insertion_reallocate_filter=1,
11    insertion_reallocate_strategy=0,
12 )

```

La cola de la tabla MAC tiene espacio para 64 elementos. La función de esta cola es desacoplar la llegada de escrituras de la tabla para prevenir descartes por ráfagas de tráfico. En el modelo final, esta cola se representa mediante una memoria interna y un registro de salida, modelados como `mac_fifo_mem_store` y `mac_fifo_out_store`. El escritor de la tabla MAC trabaja en paralelo y consume tramas desde el registro de salida mediante el proceso `mac_writer_process`.

```

1  pkt: Packet = yield self.mac_fifo_out_store.get()
2
3  self.st.mac_write_attempts += 1
4
5  lookup_before = self.mactable.lookup(
6      pkt.src_mac
7  )

```

La consulta previa con `lookup` no forma parte de la decisión de admisión, sino de la clasificación de métricas. Permite distinguir si una inserción fallida corresponde a una dirección ya presente o a una dirección que no ha podido aprenderse.

La duración de la escritura depende del número de reubicaciones que devuelve `insert`.

```

1  insert_result = self.mactable.insert(
2      key=pkt.src_mac,
3      effective=True,
4  )
5
6  success, loops, returned_key = (
7      self._decode_insert_result(
8          insert_result,
9          pkt.src_mac,
10     )
11 )

```

```

1  total_write_cycles = self._write_latency_from_loops(
2      loops
3  )

```

Para la tabla MAC se utiliza el mismo criterio temporal que en la caché: cinco ciclos para la primera operación de escritura y cuatro ciclos adicionales por cada reubicación:

$$T_{\text{MAC}} = 5 + 4 \cdot N_{\text{loops}} \quad (4.8)$$

Implementación del pipeline

El modelo completo se organiza como un pipeline de procesos **SimPy**. Esta elección permite describir el paso de las tramas por varias etapas temporales sin convertir la simulación en una ejecución puramente secuencial. Cada etapa recibe una estructura con la trama y las señales calculadas hasta ese punto, espera los ciclos correspondientes y entrega el resultado a la siguiente etapa.

```

1 self.s1_store = simpy.Store(self.env)
2 self.s2_store = simpy.Store(self.env)
3 self.s3_store = simpy.Store(self.env)
4 self.s4_store = simpy.Store(self.env)
5 self.s5_store = simpy.Store(self.env)

```

Estas colas intermedias funcionan como registros de pipeline. Además de la trama, transportan la información necesaria para resolver la consulta de caché y, en el modelo con CMS, la estimación de frecuencia leída al inicio.

En la etapa 0 se crea la trama y se lee el CMS si está habilitado. La consulta efectiva de la caché no se utiliza en esta etapa, ya que su respuesta se modela con la latencia configurada y solo está disponible más adelante.

Las etapas 1, 2 y 3 introducen la latencia de propagación de la consulta. Cada una consume un registro, espera un ciclo y lo entrega a la etapa siguiente.

```

1 def stage1_process(self):
2     reg_s1 = yield self.s1_store.get()
3     yield self.env.timeout(1)
4     yield self.s2_store.put(...)

```

```

1 def stage2_process(self):
2     reg_s2 = yield self.s2_store.get()
3     yield self.env.timeout(1)
4     yield self.s3_store.put(...)

```

```

1 def stage3_process(self):
2     reg_s3 = yield self.s3_store.get()
3     yield self.env.timeout(1)
4     yield self.s4_store.put(reg_s3)

```

La etapa 4 es el punto en el que se completa la respuesta de la lectura de la caché. Si la dirección se encuentra en la caché, la trama se filtra. En caso contrario, se introduce en la cola de la tabla MAC si su capacidad lo permite.

```
1 cache_hit = (  
2     self.cache_table.lookup(key)  
3     is not None  
4 )  
5  
6 if cache_hit:  
7     self.st.cache_hits += 1  
8     self.st.suppressed += 1  
9 else:  
10    self.st.cache_misses += 1
```

En el modelo sin CMS, la petición de escritura de caché se genera en esta misma etapa 4, siempre que la dirección MAC a escribir haya sido admitida por la cola de la tabla MAC. En el modelo con CMS, en cambio, la decisión de admisión en caché se retrasa a una etapa adicional.

La etapa 5 solo se utiliza cuando el CMS está activo. En ella se actualiza el estimador de frecuencia y se decide la admisión en caché a partir de dos condiciones: la ocupación actual de la caché y la frecuencia estimada por el CMS.

```
1 cms_estimate_after = (  
2     reg_s4.cms_estimate_before + 1  
3     if self.cms is not None  
4     else 0  
5 )  
6  
7 self.cms.update(key)
```

```
1 if self._cache_fill_ratio() <  
2     self.cms_cache_admission_fill_threshold:  
3     should_update_cache = True  
4 elif cms_estimate_after >= self.cms_threshold:  
5     should_update_cache = True  
6 else:  
7     should_update_cache = False  
8  
9 if should_update_cache and reg_s4.admitted_to_mac:  
10    yield self.cache_fifo_mem_store.put(key)
```

El resumen de etapas queda de la siguiente forma:

- **Etapa 0:** generación de la trama y lectura del CMS si está habilitado.
- **Etapa 1:** primer ciclo de latencia.

- **Etapa 2:** segundo ciclo de latencia.
- **Etapa 3:** tercer ciclo de latencia.
- **Etapa 4:** procesado del resultado de lectura de la caché. Se genera, si es necesario, la solicitud de escritura en la tabla MAC. Para el modelo sin CMS, en esta etapa se genera también la petición de escritura en la caché.
- **Etapa 5:** etapa exclusiva del modelo con CMS. En esta etapa se actualiza, si es necesario, el CMS y se genera la petición de escritura en la caché.

Las etapas del pipeline no escriben directamente en la caché o en la tabla MAC. Solo generan peticiones hacia las colas correspondientes. Las escrituras reales se ejecutan en dos procesos independientes: `mac_writer_process` para la tabla MAC y `cache_writer_process` para la caché.

```

1 self.env.process(self.stage1_process())
2 self.env.process(self.stage2_process())
3 self.env.process(self.stage3_process())
4 self.env.process(self.stage4_process())
5 self.env.process(self.stage5_process())
6
7 self.env.process(self.mac_fifo_prefetch_process())
8 self.env.process(self.mac_writer_process())
9 self.env.process(self.cache_fifo_prefetch_process())
10 self.env.process(self.cache_writer_process())

```

Esta separación es importante para el funcionamiento del sistema: el pipeline puede seguir procesando tramas mientras la tabla MAC o la caché continúan ocupadas con escrituras anteriores. Además, los procesos de pre-lectura de las colas modelan el retardo de salida de las FIFOs antes de que el escritor correspondiente pueda consumir el dato. Gracias a esta separación, el pipeline se puede implementar de forma correcta, ya que la escritura en la tabla MAC y en la caché no tiene un número fijo de ciclos.

Validación funcional de los modelos

Una vez implementados los modelos en Python, es necesario comprobar que su comportamiento es coherente antes de abordar la implementación en HDL. El objetivo de esta validación no es evaluar todavía una arquitectura hardware concreta, sino verificar que los modelos propuestos proporcionan una mejora respecto al modelo base ante distintas condiciones de tráfico. Una vez validados, estos modelos servirán como referencia para comprobar posteriormente la implementación hardware.

Para realizar esta validación se utiliza un barrido del número de direcciones MAC posibles a una velocidad de 51,2 Gbps. Esta velocidad representa el escenario más exigente de los estudiados, ya que se genera una trama por

ciclo de reloj a 100 MHz. De este modo, el efecto del módulo de filtrado puede apreciarse con mayor claridad.

En todas las simulaciones se generan 30.000 tramas. El número de direcciones MAC posibles varía entre 1000 y 20000 direcciones, mientras que el parámetro de la distribución Zipf toma los valores $\alpha = 0.7$ y $\alpha = 1.2$. El primer caso representa un tráfico más repartido entre las distintas direcciones MAC, mientras que el segundo concentra una mayor parte de las tramas sobre un conjunto reducido de direcciones.

Análisis de resultados

El primer objetivo a analizar es el efecto que produce la caché sobre la carga de la tabla MAC. Para ello se estudian el porcentaje de tráfico filtrado por la caché y el porcentaje de descartes producidos por saturación de la cola de escritura.

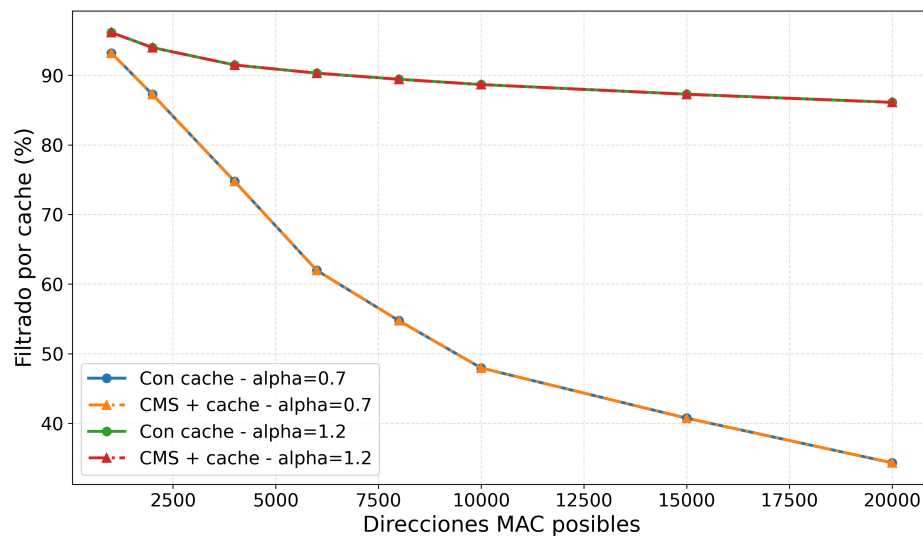


Figura 4.5: Porcentaje de tráfico filtrado por la caché a 51,2 Gbps

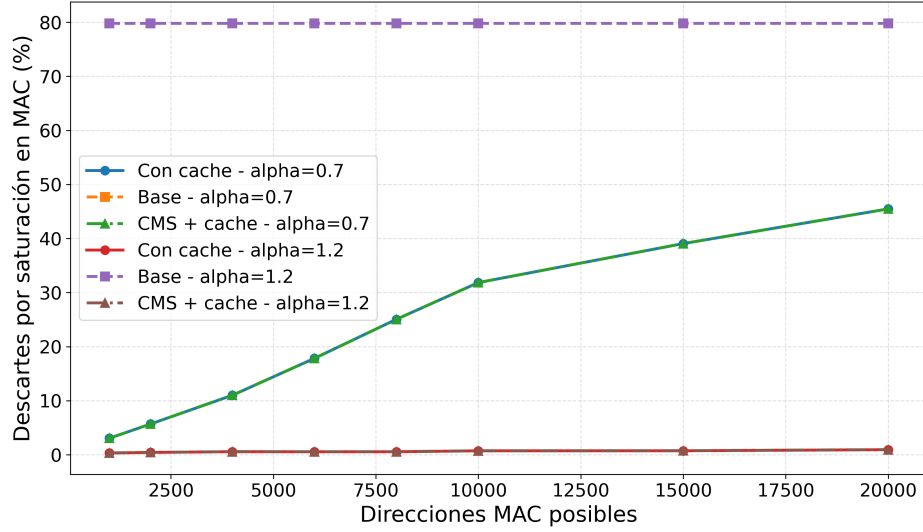


Figura 4.6: Porcentaje de descartes por saturación en la tabla MAC a 51,2 Gbps

Como puede observarse en la Figura 4.5, al aumentar el número de direcciones MAC posibles, el porcentaje de filtrado de la caché disminuye. Este comportamiento resulta coherente, ya que la probabilidad de que una dirección MAC se repita disminuye a medida que aumenta el número de direcciones MAC posibles. Como consecuencia, aumenta el número de solicitudes que alcanzan la tabla MAC y crece la presión sobre su cola de escritura.

Este comportamiento puede observarse directamente en la Figura 4.6. A medida que disminuye el porcentaje de tráfico filtrado, aumenta el porcentaje de descartes por saturación en la tabla MAC. Sin embargo, los modelos con caché y con CMS más caché presentan una reducción muy significativa de estos descartes respecto al modelo base. Esta diferencia se acentúa conforme aumenta la repetición de direcciones MAC en el tráfico, llegando a ser prácticamente nulos para $\alpha = 1.2$.

Una vez analizada la reducción de la congestión y de los descartes en la tabla MAC, resulta interesante estudiar cómo afecta este comportamiento a la capacidad de aprendizaje de la propia tabla MAC. Para ello se analizan el porcentaje de direcciones MAC aprendidas y la eficiencia de escritura.

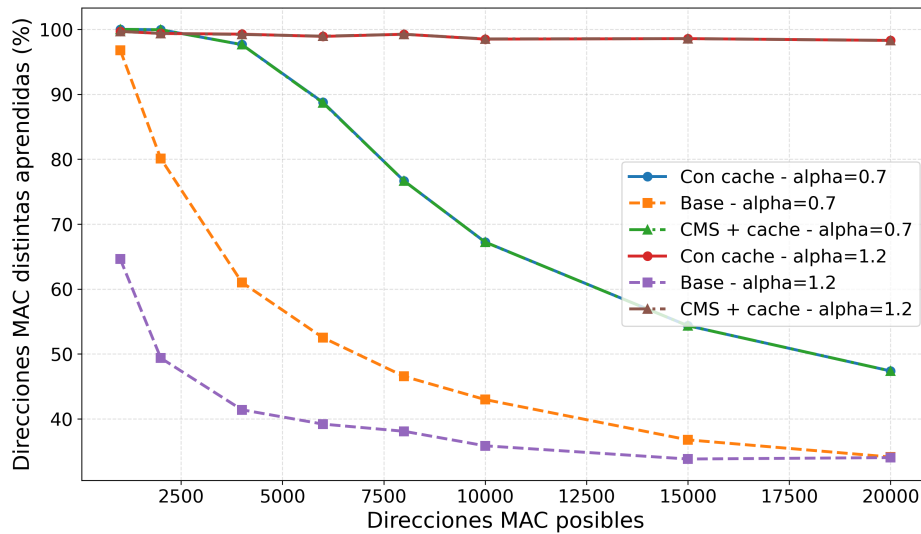


Figura 4.7: Porcentaje de direcciones MAC aprendidas a 51,2 Gbps

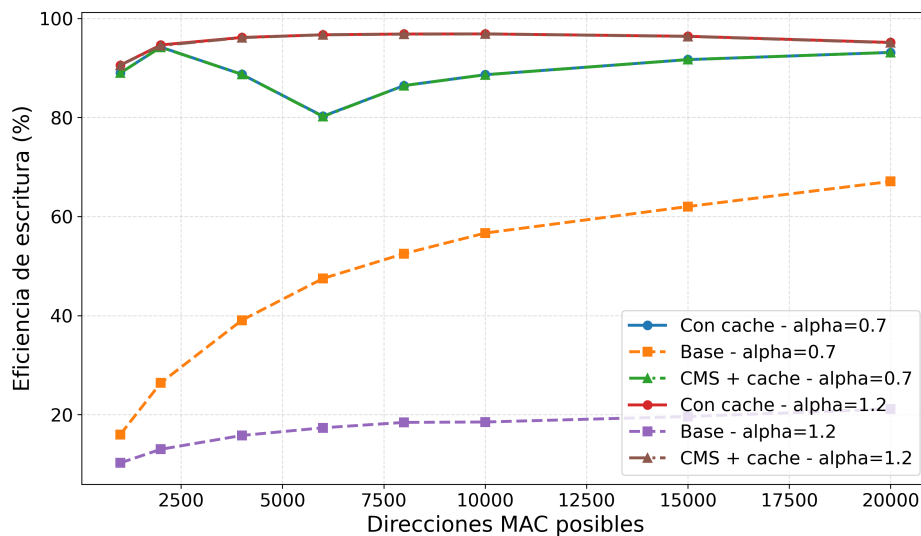


Figura 4.8: Eficiencia de escritura en la tabla MAC a 51,2 Gbps

La Figura 4.7 muestra un comportamiento coherente con las gráficas analizadas anteriormente. La primera observación clara es que, independientemente del tipo de tráfico, los modelos con solo caché y con CMS más caché presentan una notable mejora respecto al modelo base. Al igual que en la Figura 4.6, cuanto mayor es el número de direcciones MAC posibles, menor es el porcentaje de aprendizaje.

Se puede apreciar cómo los modelos implementados reducen el número de escrituras redundantes y permiten que una mayor proporción de las solicitudes procesadas correspondan a direcciones nuevas. Como consecuencia, el porcentaje de aprendizaje aumenta de forma significativa respecto al modelo base.

Este comportamiento también se refleja en la Figura 4.8. La eficiencia de escritura es claramente superior en los modelos con caché, ya que una mayor parte de las operaciones que alcanzan la tabla MAC corresponden realmente a nuevas direcciones MAC y no a escrituras repetidas.

Finalmente, puede observarse que los modelos con solo caché y con CMS más caché presentan resultados muy similares. Este resultado es coherente con la política implementada. Mientras la caché dispone de espacio libre, ambos modelos se comportan de forma equivalente. El CMS únicamente comienza a intervenir cuando la ocupación de la caché supera el umbral establecido, limitando la admisión de direcciones poco frecuentes sin modificar el funcionamiento del aprendizaje dinámico de la tabla MAC.

En conjunto, todas las métricas evolucionan de acuerdo con el comportamiento esperado del sistema. Al aumentar la diversidad de direcciones MAC disminuye el porcentaje de filtrado, aumenta la presión sobre la tabla MAC y se reduce el porcentaje de aprendizaje. Del mismo modo, la incorporación de la caché mejora claramente el comportamiento respecto al modelo sin filtrado, mientras que el CMS mantiene prácticamente las mismas prestaciones introduciendo una política de admisión más selectiva cuando la caché alcanza ocupaciones elevadas.

Los resultados obtenidos muestran que la incorporación de la caché mejora de forma significativa el proceso de aprendizaje de la tabla MAC respecto al modelo base, reduciendo los descartes por saturación y aumentando tanto el porcentaje de direcciones MAC aprendidas como la eficiencia de escritura.

En consecuencia, el modelo funcional desarrollado en Python justifica su implementación en HDL y se utilizará como referencia para verificar posteriormente su comportamiento. Además, dado que el modelo con solo caché presenta resultados prácticamente idénticos al modelo con CMS más caché en todas las métricas analizadas, se ha decidido implementar en HDL únicamente la arquitectura con caché. Esta decisión permite simplificar el diseño hardware manteniendo prácticamente las mismas mejoras observadas en el modelo funcional.

Capítulo 5

Implementación hardware e integración

Una vez comprobado en Python que la incorporación de una caché mejora el proceso de aprendizaje de la tabla MAC, el siguiente paso consiste en trasladar esta arquitectura a una implementación hardware. El objetivo de esta etapa no es únicamente desarrollar el diseño en HDL, sino disponer de un modelo que pueda sintetizarse y ejecutarse sobre una FPGA.

Como se justificó en el capítulo anterior, la implementación hardware se centra únicamente en el modelo base y el modelo con caché. Estas arquitecturas se describen en HDL mediante los módulos `modulo_hdl_sin_cache.v` y `modulo_hdl_cache.v`, respectivamente.

En este capítulo se describe cómo se ha trasladado la implementación del modelo funcional a una versión completamente hardware y cómo se ha creado el entorno necesario para poder implementar esta versión en la FPGA utilizada en este proyecto.

En el Apéndice A se encuentra el código completo implementado, tanto para su implementación HDL como su posterior implementación en FPGA. También se incluyen distintos ficheros y reportes de su implementación y síntesis con Vivado.

5.1. Implementación HDL del módulo de aprendizaje

A partir del modelo seleccionado tras la validación funcional en Python se ha desarrollado su implementación en HDL, trasladando el funcionamiento de los distintos bloques a una arquitectura síncrona basada en señales de control, registros, colas FIFO y tablas hash.

Para esta implementación se han reutilizado los módulos `hash_table.v` y `switch_simple_fifo.v`, proporcionados por Carlos Megías [15].

`hash_table.v` proporciona una implementación de tabla hash basada en Cuckoo Hashing, mientras que `switch_simple_fifo.v` implementa una cola FIFO parametrizable. Estos módulos se utilizan como bloques básicos para construir las estructuras de almacenamiento necesarias en la arquitectura.

Utilizando estos bloques se ha desarrollado el sistema completo. Este desarrollo incluye la lógica de control e interconexión entre los distintos módulos, la interfaz de entrada, el mecanismo de filtrado mediante caché, la generación y gestión de las peticiones de escritura hacia la caché y la tabla MAC y el sistema de monitorización utilizado para obtener las métricas de funcionamiento. Por tanto, la reutilización de los módulos proporcionados se limita a las estructuras básicas de almacenamiento, mientras que la lógica que determina el funcionamiento conjunto de la arquitectura ha sido desarrollada como parte de este proyecto.

Arquitectura del sistema

La Figura 4.3 muestra la arquitectura general del modelo con caché seleccionado tras la validación funcional en Python y utilizado como base para su implementación en HDL. El módulo global de esta arquitectura es `modulo_hdl_cache`.

La arquitectura está formada por cuatro módulos principales:

- **Caché.** Se implementa mediante una instancia del módulo reutilizado `hash_table`, denominada `hash_inst`. Su función dentro de la arquitectura es comprobar si una dirección MAC ha sido observada recientemente. Cuando se produce un *hit*, la lógica desarrollada filtra la petición y evita que continúe hacia la tabla MAC. En caso de *miss*, la dirección continúa por la ruta de aprendizaje.
- **Cola de escritura de la caché.** Se implementa mediante una instancia del módulo reutilizado `switch_simple_fifo`, denominada `cache_write_fifo_inst`. Esta cola almacena temporalmente las direcciones que deben escribirse en la caché después de producirse un *miss*, desacoplando la consulta de la caché de su proceso de escritura.
- **Cola de escritura de la tabla MAC.** Se implementa mediante otra instancia de `switch_simple_fifo`, denominada `mac_write_fifo_inst`. Su función es almacenar los *misses* de la caché que pasan al proceso de escritura de la tabla MAC.
- **Tabla MAC.** Se implementa mediante una segunda instancia del módulo reutilizado `hash_table`, denominada `mac_table_inst`, y almacena las direcciones MAC finalmente aprendidas.

Además de estos cuatro bloques principales, se han desarrollado elementos auxiliares necesarios para controlar y supervisar el funcionamiento del sistema:

- **Interfaz de entrada.** Recibe las direcciones MAC mediante las señales `learn_request_valid_in`, `learn_request_ready_out` y `learn_request_mac_in`. Esta interfaz utiliza un mecanismo `valid/ready` para controlar la entrada de nuevas peticiones.
- **Lógica de control.** Coordina el funcionamiento de los distintos bloques de la arquitectura. Se encarga de generar las consultas a la caché, interpretar sus respuestas, aplicar el mecanismo de filtrado, generar las peticiones de escritura y controlar el avance de las direcciones MAC a través de las distintas etapas del sistema.
- **Sistema de monitorización.** Está formado por un conjunto de contadores internos desarrollados para registrar el comportamiento del módulo durante cada prueba. Entre ellos se encuentran las peticiones aceptadas, los *hits* y *misses* de caché, los descartes producidos y las entradas aprendidas por la tabla MAC. Estos valores se utilizan posteriormente para analizar los resultados obtenidos.

De esta forma, los módulos proporcionados `hash_table` y `switch_simple_fifo` se utilizan como bloques básicos de almacenamiento, mientras que la arquitectura que los integra y coordina, junto con los mecanismos de filtrado, control y monitorización, constituye el desarrollo realizado en este trabajo. A continuación se describe con mayor detalle la implementación de los distintos bloques y su funcionamiento conjunto.

Señales de reloj y reset

Todo el módulo `modulo_hdl_cache` trabaja sincronizado con una señal de reloj común, denominada `clk`. Las transferencias entre registros, la actualización de contadores y el control de las escrituras se realizan en los flancos de subida de esta señal. De esta forma, el comportamiento del sistema queda descrito a nivel RTL como una serie de transferencias sincronizadas ciclo a ciclo.

Además del reloj, el módulo recibe una señal de reinicio `rst`. Esta señal permite reiniciar el circuito a un estado inicial antes de lanzar una simulación o una prueba sobre la FPGA. En los procesos secuenciales principales se utiliza un reset síncrono, por lo que la reinicialización del estado interno solo se produce en un flanco de subida de `clk`.

```
1 always @(posedge clk) begin
2     if (rst) begin
3         ...
4     end else begin
5         ...
6     end
7 end
```

Por tanto, cuando `rst` está activo durante un flanco de subida de `clk`, los registros de control se inicializan a cero y se eliminan las escrituras pendientes. Cuando `rst` está desactivado, el módulo actualiza su estado siguiendo la lógica normal del circuito.

Las instancias de `hash_table.v` y `switch_simple_fifo.v` que conforman los distintos bloques reciben las mismas señales `clk` y `rst`. Por tanto, la caché, la tabla MAC y las colas de escritura avanzan en el mismo dominio de reloj. Esto simplifica la integración, ya que no es necesario introducir sincronizadores adicionales entre estos bloques.

Implementación de la caché

La caché se implementa como una instancia del módulo `hash_table.v`. Este módulo tiene dos interfaces principales: una interfaz de lectura, empleada para comprobar si la dirección MAC ya está almacenada, y una interfaz de escritura, utilizada para insertar nuevas direcciones que se quieran escribir en la tabla hash.

La entrada de la caché utiliza una interfaz de tipo `valid/ready`. La señal `learn_request_valid_in` indica que existe una nueva dirección MAC disponible para consultar, mientras que `learn_request_ready_out` se activa cuando la caché está lista para realizar una lectura. Cuando ambas señales se activan, se produce el *handshake* y la caché acepta la dirección MAC.

```

1 assign learn_request_ready_out = query_ready;
2
3 assign input_hs =
4     learn_request_valid_in &&
5     learn_request_ready_out;
6
7 assign query_data      = learn_request_mac_in;
8 assign query_request_id = {
9     learn_request_port_in,
10    learn_request_mac_in
11 };
12 assign query_valid     = input_hs;

```

La señal `input_hs` representa el *handshake* de entrada. En ese mismo ciclo, la dirección MAC se conecta a `query_data` y se activa `query_valid`. Además de la dirección, se envía un identificador de petición mediante `query_request_id`. Este identificador contiene el puerto y la MAC original, de forma que ambos valores puedan recuperarse cuando la tabla hash devuelva la respuesta de lectura 4 ciclos después.

La interfaz de lectura se conecta mediante las señales `query_request_*` y `query_response_*`. La señal `query_request_data` recibe la dirección MAC que se desea consultar. La señal `query_request_valid` indica que la petición es válida y `query_request_ready` indica que la tabla hash puede

aceptarla. Por tanto, la lectura solo se inicia cuando ambas señales están activas.

```

1 .query_request_data(query_data),
2 .query_request_id(query_request_id),
3 .query_request_valid(query_valid),
4 .query_request_ready(query_ready),
5
6 .query_response_id(query_response_id),
7 .query_response_valid(query_response_valid),
8 .query_response_ready(query_response_ready),
9 .query_response_error(query_response_error),

```

La señal `query_response_error` se utiliza para distinguir si la dirección MAC se encuentra o no en la caché. En esta implementación no se utiliza el dato devuelto por `query_response_data`, ya que para el filtrado solo interesa saber si la clave existe o no. Por este motivo, dicho puerto se deja sin conectar.

La salida de la lectura se interpreta a partir de `query_response_valid` y `query_response_error`. Si la respuesta es válida y no hay error, la dirección ya estaba almacenada en la caché y se considera un *hit*. En cambio, si ambas señales están activas, se genera un *miss*. Ese *miss* se convierte en la salida válida del bloque de caché hacia la ruta de aprendizaje.

```

1 assign miss_event =
2     query_response_valid &&
3     query_response_error;
4
5 assign learn_request_mac_out    = response_mac;
6 assign learn_request_port_out  = response_port;
7 assign learn_request_valid_out = miss_event;

```

De esta forma, las direcciones que producen *hit* quedan filtradas dentro del bloque de caché, mientras que las direcciones que producen *miss* se propagan hacia la cola de la tabla MAC.

La misma instancia de `hash_table` dispone también de una interfaz de escritura. Esta interfaz recibe los datos procedentes de la FIFO de escritura de la caché:

```

1 .write_request_data(write_request_data),
2 .write_request_active(1'b1),
3 .write_request_valid(write_valid_reg),
4 .write_request_ready(write_ready),
5
6 .write_response_valid(write_response_valid),

```

La señal `write_request_data` contiene la dirección MAC y el puerto que se quieren insertar. La señal `write_request_active` se fija a uno, ya que

todas las entradas insertadas se consideran válidas. El *handshake* de escritura se produce cuando `write_request_valid` y `write_request_ready` están activas. Una vez terminada la inserción, la tabla genera `write_response_valid`.

La instancia utilizada se parametriza para que la estructura tenga el mismo tamaño y la misma política de inserción que el módulo caché estudiado en Python en la sección 4.3:

```

1  hash_table #(
2      .DEPTH(HASH_DEPTH),
3      .TABLE_COUNT(TABLE_COUNT),
4      .STORE_WIDTH(STORE_WIDTH),
5      .KEY_WIDTH(KEY_WIDTH),
6      .LOOP_COUNT(LOOP_COUNT),
7      .INSERTION_INITIAL_FILTER(1),
8      .INSERTION_INITIAL_STRATEGY(0),
9      .INSERTION_REALLOCATE_FILTER(1),
10     .INSERTION_REALLOCATE_STRATEGY(0),
11     .CONFIG_ENABLE(0),
12     .REGISTER_STORE_STATE(2),
13     .ID_WIDTH(ID_WIDTH)
14 )
15 hash_inst (...);

```

Cola de escritura de la caché

La cola de escritura de la caché se implementa mediante una instancia del módulo `switch_simple_fifo.v`. Como ya se ha explicado anteriormente, su función no es solo desacoplar múltiples procesos de escritura que lleguen en ciclos continuados, sino también separar en procesos independientes la lectura y escritura de la caché.

La FIFO se parametriza con una profundidad de 64 posiciones y cada posición dispone de un número de bits igual a `FIFO_DATA_WIDTH`. Este valor se define como la suma de `KEY_WIDTH` y `STORE_WIDTH`. En la configuración utilizada, `KEY_WIDTH` vale 48 bits, ya que representa la dirección MAC, y `STORE_WIDTH` vale 8 bits, correspondientes al puerto asociado. Por tanto, `FIFO_DATA_WIDTH` queda fijado a 56 bits.

```

1  switch_simple_fifo #(
2      .DEPTH(WRITE_FIFO_DEPTH),
3      .DATA_WIDTH(FIFO_DATA_WIDTH),
4      .FRAME_FIFO_WR(0),
5      .FRAME_FIFO_RD(0)
6  )
7  cache_write_fifo_inst (...);

```

La escritura en la FIFO se rige mediante los puertos `wr_enable` y `wr_`

ready de la instancia `cache_write_fifo_inst`. En el módulo superior, estos puertos se conectan respectivamente a las señales `write_fifo_wr_enable` y `write_fifo_wr_ready`. El dato que se almacena en la FIFO se conecta a través de `write_fifo_wr_data` y corresponde a la concatenación del puerto y la dirección MAC recuperados de la respuesta de lectura de la caché:

```
1 assign write_fifo_wr_data = {
2     response_port,
3     response_mac
4 };
5
6 assign write_fifo_wr_enable =
7     output_hs &&
8     write_fifo_wr_ready;
```

La señal `output_hs` indica que el *miss* ha sido aceptado por la cola de la tabla MAC. Por tanto, la FIFO de escritura de caché solo intenta almacenar direcciones que han podido avanzar hacia la cola de escritura de la tabla MAC. Además, se comprueba `write_fifo_wr_ready` para no intentar guardar un elemento en la FIFO cuando no queda espacio disponible.

La lectura de la FIFO se conecta con la interfaz de escritura de la tabla hash. La señal `write_fifo_rd_valid` indica que existe un elemento disponible en la cola, mientras que `write_fifo_rd_enable` ordena extraer dicho elemento. Esta extracción solo se realiza cuando la petición puede enviarse realmente a la tabla hash.

A diferencia de la FIFO, que puede almacenar varios elementos pendientes, la interfaz de escritura de `hash_table.v` no admite múltiples operaciones concurrentes. Por este motivo, se incorpora un mecanismo de control basado en el registro `write_wait_response_reg`, encargado de garantizar que únicamente exista una escritura pendiente de respuesta en cada instante.

```
1 assign write_valid_reg =
2     (!write_wait_response_reg ||
3     write_response_valid) &&
4     write_fifo_rd_valid;
5
6 assign write_hs =
7     write_valid_reg &&
8     write_ready;
9
10 assign write_fifo_rd_enable = write_hs;
11 assign write_request_data = write_fifo_rd_data;
```

La señal `write_valid_reg` solo se activa si la FIFO contiene un dato y la tabla hash no tiene una escritura pendiente, o si la respuesta de la escritura anterior llega en ese mismo ciclo. De esta forma, el módulo puede encadenar una nueva escritura justo cuando termina la anterior, sin introducir ciclos

de latencia adicionales.

El *handshake* de escritura hacia la tabla hash se representa mediante `write_hs`. Esta señal se activa cuando la petición es válida y la tabla hash indica disponibilidad mediante `write_ready`. En ese mismo ciclo se activa `write_fifo_rd_enable`, por lo que el dato se extrae de la FIFO y se conecta a la entrada de escritura de la tabla hash mediante `write_request_data`.

```

1 if (write_response_valid) begin
2     write_wait_response_reg <= 1'b0;
3 end
4
5 if (write_hs) begin
6     write_wait_response_reg <= 1'b1;
7 end

```

Cuando se lanza una escritura hacia la tabla hash, el registro `write_wait_response_reg` pasa a uno. Mientras permanece activo, no se genera una nueva petición. Cuando `write_response_valid` se activa, significa que la escritura anterior ha finalizado y el registro vuelve a cero.

Este mecanismo adapta la FIFO a la interfaz de escritura de `hash_table.v`. La cola puede acumular solicitudes, pero la tabla hash solo recibe una nueva escritura cuando no existe otra pendiente de respuesta o cuando la anterior termina en ese mismo ciclo.

Tabla MAC y su cola de escritura

Al igual que ocurre en la caché, las operaciones de escritura sobre la tabla MAC presentan una latencia variable debido a las reubicaciones asociadas al algoritmo Cuckoo Hashing. Para desacoplar la llegada de solicitudes de aprendizaje de la ejecución efectiva de las inserciones, se introduce una cola FIFO de escritura situada delante de la tabla MAC.

La estructura es similar a la utilizada para la escritura de la caché, pero con algunas diferencias importantes en la interfaz y en los contadores asociados. La cola MAC se implementa mediante otra instancia de `switch_simple_fifo`, también con una profundidad de 64 posiciones. La escritura en esta cola se produce cuando existe un *miss* válido y la FIFO indica que puede aceptar un nuevo elemento.

```

1 assign output_hs =
2     learn_request_valid_out &&
3     mac_fifo_wr_ready;
4
5 assign learn_request_accepted_out = output_hs;
6 assign learn_request_dropped_out =
7     learn_request_valid_out &&
8     !mac_fifo_wr_ready;
9

```

```

10 assign mac_fifo_wr_data = {
11     response_port,
12     response_mac
13 };
14
15 assign mac_fifo_wr_enable = output_hs;

```

La señal `output_hs` representa el handshake asociado a la interfaz valid/ready de entrada de la cola MAC. Se activa únicamente cuando existe una petición válida procedente de la caché y la FIFO dispone de espacio para aceptarla. Por este motivo, si la FIFO de la tabla MAC está llena, `output_hs` no se activa y la petición queda contabilizada como descarte. En cambio, si la FIFO dispone de espacio, el dato se almacena y posteriormente será leído por el escritor de la tabla MAC.

Las señales `learn_request_accepted_out` y `learn_request_dropped_out` permiten contabilizar ambas situaciones y serán utilizadas posteriormente para evaluar el comportamiento del sistema.

La lectura de esta cola se controla de forma equivalente a la FIFO de la caché. La señal `mac_fifo_rd_valid` indica que hay una dirección pendiente de escribir y `mac_fifo_rd_enable` se activa cuando la tabla MAC acepta la petición de escritura:

```

1 assign mac_write_valid_reg =
2     (!mac_write_wait_response_reg ||
3     mac_write_response_valid) &&
4     mac_fifo_rd_valid;
5
6 assign mac_write_hs =
7     mac_write_valid_reg &&
8     mac_write_ready;
9
10 assign mac_fifo_rd_enable = mac_write_hs;
11 assign mac_write_request_data = mac_fifo_rd_data;

```

La señal `mac_write_valid_reg` cumple la misma función que en la escritura de la caché: solo permite iniciar una nueva escritura si existe un dato pendiente en la FIFO y la tabla MAC no está esperando la respuesta de una escritura anterior.

La tabla MAC se implementa mediante una segunda instancia del módulo `hash_table`. En esta instancia no se utiliza la interfaz de lectura, ya que el objetivo del bloque es modelar el aprendizaje de direcciones mediante escrituras. Por ello, las señales de consulta se conectan a cero o se dejan sin utilizar, mientras que la interfaz de escritura se conecta a la salida de su cola de escritura.

```

1 hash_table #(
2     .DEPTH(MAC_HASH_DEPTH),

```

```

3      .TABLE_COUNT(MAC_TABLE_COUNT),
4      .STORE_WIDTH(STORE_WIDTH),
5      .KEY_WIDTH(KEY_WIDTH),
6      .LOOP_COUNT(MAC_LOOP_COUNT),
7      .INSERTION_INITIAL_FILTER(1),
8      .INSERTION_INITIAL_STRATEGY(0),
9      .INSERTION_REALLOCATE_FILTER(1),
10     .INSERTION_REALLOCATE_STRATEGY(0),
11     .CONFIG_ENABLE(0),
12     .REGISTER_STORE_STATE(2),
13     .LFSR_POLY(MAC_LFSR_POLY),
14     .ID_WIDTH(ID_WIDTH)
15 )
16 mac_table_inst (...);

```

La configuración utilizada mantiene la misma política de inserción que la empleada en la caché y en el modelo funcional desarrollado en Python. En este caso, la tabla dispone de 16384 entradas distribuidas en cuatro tablas paralelas.

La interfaz de respuesta de escritura de la tabla MAC sí se utiliza para generar métricas internas. Las señales `mac_write_response_valid` y `mac_write_response_error` permiten saber cuándo ha terminado una escritura y si se ha completado correctamente. El proceso secuencial encargado de actualizar estos contadores se describe en el siguiente apartado.

Además, la salida `utilization_table` de esta instancia se conecta a `mac_table_learned_entries`. Esta señal representa el número de entradas ocupadas en la tabla MAC y se utiliza posteriormente para calcular el porcentaje de direcciones aprendidas.

Procesos secuenciales de control

Además de las conexiones combinacionales entre módulos, el diseño incluye varios procesos secuenciales que se ejecutan en cada flanco de subida de la señal de reloj `clk`. Estos procesos son los encargados de recordar el estado de las escrituras en curso y de actualizar los contadores internos.

A diferencia del modelo funcional, donde el comportamiento se describe mediante procesos gestionados por SimPy, en HDL el estado del sistema se almacena en registros actualizados de forma síncrona con la señal de reloj. Estos procesos secuenciales permiten controlar las operaciones cuya ejecución se extiende durante varios ciclos y mantener la información necesaria para coordinar los distintos bloques del diseño.

El primer proceso secuencial controla la escritura de la caché. Su registro principal es `write_wait_response_reg`. Este registro indica si ya se ha enviado una petición de escritura a la tabla hash y todavía no se ha recibido su respuesta. Si se activa `write_response_valid`, significa que la escritura anterior ha terminado y el registro vuelve a cero. Si se produce un nuevo *handshake* de escritura, `write_hs`, el registro pasa a uno.

```

1 always @(posedge clk) begin
2     if (rst) begin
3         write_wait_response_reg <= 1'b0;
4     end else begin
5         if (write_response_valid) begin
6             write_wait_response_reg <= 1'b0;
7         end
8
9         if (write_hs) begin
10            write_wait_response_reg <= 1'b1;
11        end
12    end
13 end

```

Este proceso no realiza la inserción directamente. La inserción la ejecuta internamente el módulo `hash_table.v`. El proceso únicamente controla cuándo puede enviarse una nueva petición desde la FIFO hacia esa tabla. Por tanto, actúa como una pequeña lógica de arbitraje entre la cola de escritura y la interfaz de escritura de la caché.

El segundo proceso secuencial controla la escritura de la tabla MAC. Su estructura es parecida, pero además actualiza los contadores que permiten conocer cuántas escrituras se han solicitado, cuántas han terminado y cuántas han producido error.

```

1 always @(posedge clk) begin
2     if (rst) begin
3         mac_write_wait_response_reg <= 1'b0;
4         mac_table_write_requests_reg <= 32'd0;
5         mac_table_write_responses_reg <= 32'd0;
6         mac_table_write_success_reg <= 32'd0;
7         mac_table_write_failed_reg <= 32'd0;
8     end else begin
9         if (mac_write_response_valid) begin
10            mac_write_wait_response_reg <= 1'b0;
11            mac_table_write_responses_reg <=
12                mac_table_write_responses_reg + 1;
13
14            if (mac_write_response_error) begin
15                mac_table_write_failed_reg <=
16                    mac_table_write_failed_reg + 1;
17            end else begin
18                mac_table_write_success_reg <=
19                    mac_table_write_success_reg + 1;
20            end
21        end
22
23        if (mac_write_hs) begin
24            mac_write_wait_response_reg <= 1'b1;
25            mac_table_write_requests_reg <=
26                mac_table_write_requests_reg + 1;

```

```

27         end
28     end
29 end

```

El funcionamiento de este proceso puede resumirse en dos acciones. En primer lugar, cuando se acepta una nueva escritura hacia la tabla MAC, se incrementa `mac_table_write_requests_reg` y se marca que existe una operación pendiente. En segundo lugar, cuando la tabla MAC genera una respuesta, se incrementa `mac_table_write_responses_reg` y se clasifica la operación como correcta o fallida dependiendo de `mac_write_response_error`.

De esta forma, los procesos secuenciales no definen el flujo completo de datos, sino el estado asociado a las operaciones que no terminan en un único ciclo. Las señales combinacionales generan los *handshakes* y conectan las FIFOs con las tablas hash, mientras que los procesos síncronos se encargan de regular los procesos de escritura tanto en la caché como en la tabla MAC.

Contadores internos del módulo

Además de implementar el proceso de aprendizaje, el módulo incorpora una serie de contadores que permiten registrar los eventos más relevantes durante la ejecución de cada experimento. Estos contadores no intervienen en el funcionamiento del circuito, sino que proporcionan la información necesaria para analizar posteriormente el comportamiento del sistema.

Cada uno de ellos se incrementa automáticamente cuando se produce un determinado evento, como un miss en la lectura de la caché, una escritura en la tabla MAC o un descarte de una dirección MAC por saturación.

Contador	Evento medido
<code>cache_miss_outputs</code>	Lecturas no encontradas en la caché
<code>frames_forwarded_to_mac</code>	Misses aceptados por la cola MAC
<code>frames_dropped_by_mac_backpressure</code>	Misses descartados por saturación de la cola MAC
<code>mac_table_write_requests</code>	Escrituras solicitadas a la tabla MAC
<code>mac_table_write_responses</code>	Respuestas de escritura recibidas
<code>mac_table_write_success</code>	Escrituras completadas correctamente
<code>mac_table_write_failed</code>	Escrituras que no han podido completarse
<code>mac_table_learned_entries</code>	Entradas almacenadas en la tabla MAC

Tabla 5.1: Principales contadores implementados en el módulo HDL.

A partir de estos contadores pueden obtenerse las mismas métricas utilizadas durante la validación funcional en Python, como el porcentaje de direcciones filtradas, los descartes provocados por la saturación de la cola MAC, el porcentaje de direcciones aprendidas o la eficiencia de escritura.

5.2. Integración en el datapath del switch

Una vez desarrollado el módulo HDL, el último paso es integrarlo dentro de un entorno que permita ejecutarlo sobre una FPGA. El objetivo no es implementar el módulo en un switch Ethernet completo, sino disponer de un entorno controlado que permita evaluar exclusivamente el comportamiento del módulo de aprendizaje.

Para implementar este entorno sobre la FPGA, se han reutilizado dos módulos proporcionados por Carlos Megías: `fpga.v` y `fpga_core.v` [15].

El módulo `fpga.v` se ha reutilizado como nivel superior de la arquitectura, simplemente adaptando sus parámetros y conexiones para incorporar el diseño desarrollado. Por otro lado, se ha desarrollado el módulo `cache_core.v` basado en el módulo proporcionado `fpga_core.v`. Este nuevo módulo se ha modificado para que sea capaz de instanciar el módulo principal `modulo_hdl_cache.v`, así como para incluir la carga de trazas en BRAM y la lectura de los contadores utilizados durante la simulación.

A continuación se muestra un diagrama de bloques que enseña los distintos bloques que componen este entorno de comunicación y los distintos protocolos que utilizan para comunicarse.

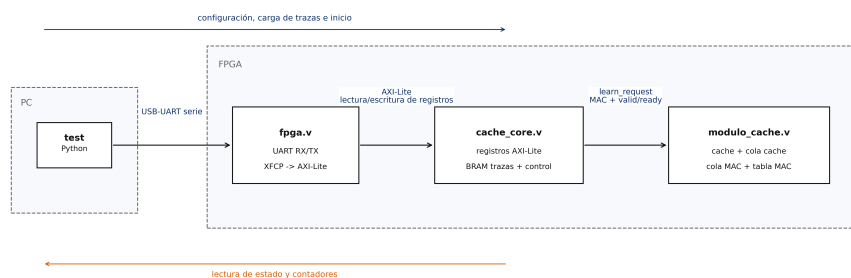


Figura 5.1: Comunicación entre el test en Python y el módulo de aprendizaje implementado en la FPGA.

La Figura 5.1 muestra la estructura utilizada para comunicar el test ejecutado en el PC con el módulo hardware. El bloque situado en el PC corresponde al script de prueba en Python. Este script no genera señales hardware directamente, sino que realiza operaciones de lectura y escritura sobre registros. Para ello utiliza el puerto serie de la placa, representado en la figura mediante el enlace USB-UART.

Dentro de la FPGA, el primer bloque de la cadena es `fpga.v`. Este módulo actúa como nivel superior del diseño y agrupa las conexiones con los pines físicos, el reloj, el reset y la comunicación serie. En su interior se recibe el tráfico UART procedente del PC y se utiliza XFCP para interpretar las peticiones de acceso a registros. Estas operaciones se convierten posteriormente en transacciones AXI-Lite.

El bloque `cache_core.v` recibe esas transacciones mediante una interfaz esclava AXI-Lite. Este módulo contiene el mapa de registros utilizado por el script Python, la memoria BRAM donde se almacena la secuencia de direcciones MAC y la lógica de control encargada de iniciar y detener cada prueba. Por tanto, `cache_core.v` es el bloque que adapta las operaciones de control procedentes del PC al funcionamiento interno del experimento.

Finalmente, `cache_core.v` se comunica con `modulo_hdl_cache.v`, que contiene la lógica de aprendizaje y filtrado descrita en el apartado anterior. Esta conexión ya no se realiza mediante registros, sino mediante una interfaz de datos basada en señales `valid/ready`. A través de ella se entregan las direcciones MAC de la traza y se reciben los eventos y contadores asociados al procesamiento.

La flecha superior de la figura representa el sentido de configuración del experimento. En esta fase, el test Python escribe en los registros de la FPGA los parámetros de ejecución, carga la traza en la BRAM e inicia la prueba. La flecha inferior representa el camino inverso, empleado para consultar el estado del sistema y leer los contadores una vez finalizado el procesamiento. Esta organización permite separar claramente el software de control de la lógica hardware: el PC solo accede a registros, mientras que el procesamiento de las direcciones MAC se realiza íntegramente dentro de la FPGA.

Conexión entre los bloques

La comunicación entre los bloques se realiza en dos pasos principales. En primer lugar, el módulo `fpga.v` recibe las señales físicas del puerto serie y las conecta con la interfaz XFCP. Esta interfaz transforma los bytes recibidos por UART en paquetes de control que posteriormente son interpretados como accesos AXI-Lite.

```
1  xfcf_interface_uart_inst (
2      .clk(clk_core_int),
3      .rst(rst_core),
4      .uart_rxd(uart_rxd_int),
5      .uart_txd(uart_txd),
6      ...
7  );
8
9  xfcf_mod_axil
10 xfcf_mod_axil_inst (
11     .clk(clk_core_int),
```



```

12     .rst(rst_core),
13     .m_axil_awaddr(axil_xfcp_awaddr),
14     .m_axil_wdata(axil_xfcp_wdata),
15     .m_axil_araddr(axil_xfcp_araddr),
16     .m_axil_rdata(axil_xfcp_rdata),
17     ...
18 );

```

De esta forma, `fpga.v` actúa como puente entre la comunicación serie externa y el bus de registros interno. Las señales `axil_xfcp_*` generadas por `xfcp_mod_axil` se conectan directamente a la interfaz esclava AXI-Lite de `cache_core.v`.

```

1  core_inst (
2      .clk(clk_core_int),
3      .rst(rst_core),
4
5      .s_axil_awaddr(axil_xfcp_awaddr),
6      .s_axil_wdata(axil_xfcp_wdata),
7      .s_axil_araddr(axil_xfcp_araddr),
8      .s_axil_rdata(axil_xfcp_rdata),
9      ...
10 );

```

El segundo paso se realiza dentro de `cache_core.v`. Este módulo interpreta las lecturas y escrituras sobre registros, almacena las direcciones MAC en la BRAM y genera las señales de entrada hacia el módulo de aprendizaje. La conexión con `modulo_hdl_cache.v` se realiza mediante la interfaz `learn_request`, formada por la dirección MAC, el puerto y las señales de control `valid/ready`.

```

1  modulo_hdl_cache_inst (
2      .clk(clk),
3      .rst(dut_reset),
4
5      .learn_request_mac_in(dut_mac_in),
6      .learn_request_port_in(dut_port_in),
7      .learn_request_valid_in(dut_valid_in),
8      .learn_request_ready_out(dut_ready_out),
9
10     .learn_request_accepted_out(
11         dut_learn_request_accepted
12     ),
13     .learn_request_dropped_out(
14         dut_learn_request_dropped
15     ),
16     ...
17 );

```

Con esta estructura, el módulo de aprendizaje no necesita conocer cómo

se comunica la FPGA con el PC. Únicamente recibe direcciones MAC cuando `cache_core.v` activa `dut_valid_in` y acepta una nueva entrada cuando responde con `dut_ready_out` produciéndose el **handshake** de entrada. Los contadores generados por el módulo se devuelven a `cache_core.v`, donde quedan disponibles como registros para que el test Python pueda leerlos al finalizar la prueba.

Mapa de registros

El intercambio de información entre el script Python y el módulo hardware se realiza mediante un conjunto de registros de control y estado implementados en `cache_core.v`. A través de ellos es posible configurar los distintos parámetros de la prueba, cargar las trazas de direcciones MAC y consultar los contadores generados durante la ejecución.

Dirección	Registro	Función
0x00	<code>control</code>	Inicio y reinicio del experimento
0x04	<code>status</code>	Estado de ejecución
0x10	<code>n_frames_target</code>	Número de tramas a procesar
0x14	<code>trace_count</code>	Número de direcciones cargadas en la BRAM
0x18	<code>frame_gap_cycles</code>	Separación temporal entre tramas
0x40	<code>cycles_total</code>	Ciclos totales de ejecución
0x44	<code>frames_generated</code>	Tramas enviadas al módulo
0x48	<code>frames_accepted_by_cache</code>	Tramas aceptadas por la entrada del módulo
0x50	<code>cache_miss_outputs</code>	Fallos de caché producidos
0x58	<code>frames_dropped_by_mac_backpressure</code>	Descartes por saturación de la cola MAC
0x90	<code>mac_table_learned_entries</code>	Entradas aprendidas en la tabla MAC

Tabla 5.2: Registros principales implementados en `cache_core.v`.

El registro de control situado en la dirección 0x00 concentra las principales operaciones de gestión del módulo. Mediante sus distintos bits es posible iniciar una nueva ejecución o reiniciar el estado interno del módulo antes de comenzar una nueva prueba. Por su parte, el registro de estado permite comprobar en todo momento si el experimento continúa en ejecución o si ya ha finalizado.

El resto de registros se utiliza para configurar los parámetros del experimento y consultar las métricas generadas durante la ejecución. Entre ellos se encuentran el número de tramas que se desea procesar, la separación temporal entre direcciones consecutivas o los distintos contadores internos del módulo, necesarios para calcular las métricas analizadas en el capítulo de resultados.

5.3. Implementación y validación en FPGA

Una vez integrado el módulo dentro del entorno de la FPGA, el siguiente paso consiste en preparar un sistema que permita comunicarle las distintas trazas a la velocidad deseada para poder medir las distintas métricas.

En las primeras versiones del diseño, las direcciones MAC se generaban directamente dentro de la FPGA mediante un generador interno. Aunque esta aproximación simplificaba el funcionamiento del núcleo de pruebas, impedía comparar los resultados obtenidos con los de las simulaciones, ya que cada ejecución utilizaba una secuencia diferente de direcciones.

Para evitar este problema, se decidió modificar el banco de pruebas incorporando una memoria interna donde cargar previamente la traza completa. Durante la ejecución, el test únicamente debe recorrer esta memoria y suministrar las direcciones MAC al módulo de aprendizaje respetando la velocidad configurada. Gracias a este cambio, tanto la simulación HDL como la FPGA procesan exactamente la misma secuencia de direcciones.

Carga de traza en BRAM

Las distintas direcciones MAC de una traza se cargan en una BRAM implementada dentro del módulo `cache_core.v`. Cada posición contiene una dirección MAC de 48 bits correspondiente a cada una de las direcciones MAC que se enviarán posteriormente al módulo de aprendizaje.

Para facilitar tanto la inferencia de memoria de bloque por parte de Vivado como la escritura desde la interfaz AXI-Lite de 32 bits, cada dirección MAC se divide internamente en tres palabras de 16 bits, almacenadas en bancos independientes.

```
1 (* ram_style = "block" *)
2 reg [15:0] trace_mem_low0 [0:TRACE_DEPTH-1];
3
4 (* ram_style = "block" *)
5 reg [15:0] trace_mem_low1 [0:TRACE_DEPTH-1];
6
7 (* ram_style = "block" *)
8 reg [15:0] trace_mem_high [0:TRACE_DEPTH-1];
```

El atributo `ram_style = "block"` indica al sintetizador que esta memoria debe implementarse utilizando bloques BRAM de la FPGA. Esta decisión resulta necesaria debido al tamaño de las trazas utilizadas durante los experimentos, que pueden contener decenas de miles de direcciones MAC. Una implementación basada únicamente en registros incrementaría considerablemente el consumo de recursos.

La carga de la traza se realiza desde el script Python escribiendo sucesivamente las distintas partes de cada dirección MAC. En primer lugar se selecciona la posición de memoria, después se escriben los 32 bits menos significativos y, finalmente, los 16 bits restantes. Esta última escritura actúa como confirmación de que la entrada se ha recibido completamente y puede almacenarse en la memoria.

Una vez cargada la traza, el funcionamiento del núcleo resulta muy sencillo. Un contador interno va recorriendo secuencialmente todas las posiciones de memoria y entrega cada dirección MAC al módulo de aprendizaje cuando este está preparado para aceptarla.

```
1 if (trace_issue) begin
2     input_valid_reg <= 1'b1;
3     current_mac_reg <= trace_playback_data_reg;
4     frames_generated_reg <= frames_generated_reg + 1;
5     trace_playback_index_reg <=
6         trace_playback_index_reg + 1;
7 end
```

La velocidad a la que se presentan las direcciones MAC al módulo de aprendizaje viene determinada por el parámetro `frame_gap_cycles`, que establece el número de ciclos de reloj entre dos tramas consecutivas. Cuando este parámetro toma el valor cero, el núcleo intenta enviar una nueva dirección MAC en cada ciclo, siempre que la caché pueda aceptarla.

Lectura de registros

Una vez finalizada la ejecución de una traza, los resultados del experimento se obtienen leyendo los registros de estado y los contadores implementados en `cache_core.v`. Estos registros están conectados a la interfaz AXI-Lite, por lo que el test Python puede acceder a ellos con las mismas funciones empleadas para configurar la prueba.

En el código HDL, las lecturas se gestionan mediante un bloque `case` asociado a la dirección solicitada. Cada dirección del mapa de registros devuelve un contador concreto del experimento. Por ejemplo, las direcciones situadas a partir de `0x40` permiten consultar el número de ciclos ejecutados, las tramas procesadas, los fallos de caché y los descartes por saturación de la cola MAC.

```

1 case ({ctrl_reg_rd_addr >> 2, 2'b00})
2   RBB+16'h40: ctrl_reg_rd_data_reg <=
3     cycles_total_reg;
4   RBB+16'h44: ctrl_reg_rd_data_reg <=
5     frames_generated_reg;
6   RBB+16'h50: ctrl_reg_rd_data_reg <=
7     cache_miss_outputs_reg;
8   RBB+16'h58: ctrl_reg_rd_data_reg <=
9     frames_dropped_by_mac_backpressure_reg;
10  RBB+16'h90: ctrl_reg_rd_data_reg <=
11    dut_mac_table_learned_entries;
12  default: ctrl_reg_rd_ack_reg <= 1'b0;
13 endcase

```

El registro de estado, situado en la dirección 0x04, se utiliza para saber cuándo ha terminado la prueba. Mientras el bit **running** permanece activo, el módulo continúa procesando la traza o drenando las escrituras pendientes. Cuando el bit **done** se activa y **running** vuelve a cero, el test puede leer los contadores finales.

Tras detectar el final de la ejecución, el script lee los registros asociados a las métricas principales. Estos valores no son todavía porcentajes, sino contadores hardware. A partir de ellos se calculan posteriormente el filtrado de caché, los descartes por saturación de la cola MAC, el porcentaje de aprendizaje y la eficiencia de escritura.

```

1 results = {
2   "frames_generated":
3     read_reg(node, REG_FRAMES_GENERATED),
4   "cache_miss_outputs":
5     read_reg(node, REG_CACHE_MISS_OUTPUTS),
6   "frames_dropped_by_mac_backpressure":
7     read_reg(node, REG_FRAMES_DROPPED_BY_MAC),
8   "mac_table_write_success":
9     read_reg(node, REG_MAC_TABLE_WRITE_SUCCESS),
10  "mac_table_learned_entries":
11    read_reg(node, REG_MAC_TABLE_LEARNED_ENTRIES),
12 }

```

El programa Python únicamente actúa como lector de registros y como herramienta para almacenar los resultados en un archivo CSV, manteniendo el cálculo de métricas coherente con el utilizado en la simulación HDL.

Script de ejecución en Python

La carga de las tramas y la ejecución de los experimentos se realizan mediante el script `load_trace_and_run.py`. Este programa actúa como interfaz entre el usuario y la FPGA, automatizando todo el proceso necesario para ejecutar una prueba. Su función es leer una traza de direcciones MAC

desde un archivo CSV, cargarla en la memoria de la FPGA, iniciar la ejecución y recuperar los resultados al finalizar.

El primer paso consiste en leer el archivo que contiene la traza y establecer la comunicación con la placa mediante la interfaz serie.

```
1 trace = read_trace_csv(Path(args.trace_in),
2                          args.n_frames)
3
4 interface = xfcplib.interface.SerialInterface(
5     args.uart_port,
6     args.baud_rate
7 )
8 node = interface.enumerate()
```

El script carga la traza completa en la BRAM y configura los parámetros necesarios para el experimento, como el número de tramas que se desea procesar o la separación entre ellas.

```
1 write_reg(node, REG_N_FRAMES_TARGET, n_frames)
2 write_reg(node, REG_FRAME_GAP_CYCLES,
3           frame_gap_cycles)
4
5 write_reg(node, REG_CONTROL, 0x1)
6
7 while True:
8     status = read_reg(node, REG_STATUS)
9     running = status & 0x1
10    done = (status >> 1) & 0x1
11
12    if done and not running:
13        break
```

Tras activar el experimento, el script espera hasta que el módulo indique que la ejecución ha finalizado. En ese momento se leen los distintos contadores implementados en hardware, que posteriormente servirán para calcular las métricas utilizadas durante la evaluación.

Síntesis, implementación y lanzamiento de test

Para sintetizar el diseño se utiliza una estructura de compilación incluida en un `Makefile`. La generación del bitstream se realiza ejecutando el comando `make` en el directorio del proyecto.

Este proceso crea automáticamente el proyecto de Vivado, incorpora todas las fuentes HDL, ejecuta la síntesis y la implementación del diseño y, finalmente, genera el archivo `fpga.bit`, que será utilizado posteriormente para programar la FPGA.

Una vez generado el bitstream, la programación de la placa se realiza mediante el comando `make program-only`.

Este objetivo verifica previamente la existencia del archivo `fpga.bit`, inicia el servidor hardware cuando es necesario y ejecuta Vivado en modo batch para cargar el diseño sobre la FPGA.

Una vez programada la placa, las pruebas pueden ejecutarse individualmente mediante `make run` indicando el punto y los parámetros con los que se quiere lanzar la simulación o bien realizar un barrido completo utilizando `make sweep`.

Gracias a este flujo de trabajo es posible automatizar todo el proceso, desde la generación del bitstream hasta la obtención de los resultados experimentales, facilitando la repetición de las pruebas y garantizando que todas ellas se ejecutan siguiendo el mismo procedimiento.

Setup de pruebas utilizado

Las pruebas experimentales se realizaron sobre una FPGA ZCU102 conectada a un servidor accesible mediante SSH. El acceso remoto se realiza a través de una VPN configurada con WireGuard, lo que permite programar la placa y ejecutar los experimentos sin necesidad de acceder físicamente al laboratorio.

La comunicación entre el servidor y la FPGA se establece mediante el puerto serie USB, identificado en el sistema como un dispositivo `/dev/ttyUSB`. A través de esta conexión se envían las tramas y se recuperan los contadores generados durante la ejecución.

Es importante señalar que las direcciones MAC no se transmiten continuamente desde el ordenador durante el experimento. Antes de iniciar la prueba, la traza completa se copia a la memoria BRAM del núcleo y, a partir de ese momento, es la propia FPGA la encargada de reproducir la secuencia de direcciones a la velocidad configurada. De esta forma se evita que la comunicación serie limite el rendimiento del sistema o introduzca variaciones entre distintas ejecuciones.

Con la implementación hardware completada y el entorno de pruebas preparado, el sistema dispone de todos los elementos necesarios para validar experimentalmente la solución propuesta. En el siguiente capítulo se presentan los resultados obtenidos tanto en simulación HDL como sobre la FPGA, evaluando el comportamiento del mecanismo de filtrado propuesto y comparándolo con el modelo funcional desarrollado en Python.

Capítulo 6

Evaluación de resultados

En este capítulo se analizan los distintos resultados obtenidos por los distintos modelos desarrollados durante el trabajo. Para comenzar el estudio se comprueba que el comportamiento observado en el modelo funcional en Python se mantiene al pasar a una descripción HDL y, posteriormente, a una ejecución real sobre FPGA. Una vez validada esta correspondencia, se analiza el efecto que introduce la caché sobre el proceso de aprendizaje dinámico de direcciones MAC. Para cuantificar la mejora que introduce el módulo de aprendizaje respecto al modelo base, se estudian las métricas definidas en el capítulo 4: porcentaje de filtrado de caché, descartes por saturación de la cola MAC, porcentaje de aprendizaje y eficiencia de escritura en la tabla MAC.

Finalmente, se analiza el rendimiento de la implementación hardware y el consumo de recursos obtenido tras la síntesis e implementación en Vivado.

6.1. Parámetros de simulación

Para poder estudiar los distintos modelos, el primer paso es definir los distintos parámetros que van a caracterizar el tráfico, así como el entorno y tipos de test que se van a utilizar.

Tal y como se explicó en la sección 2.4, la distribución utilizada para asemejar el tráfico simulado a un tráfico Ethernet real será la distribución de Zipf.

Con el objetivo de analizar distintos grados de concentración del tráfico, en este trabajo se realizarán simulaciones para valores de α de 0,7 y 1,2. Estos valores permiten abarcar desde escenarios con una distribución cercana a la realidad hasta situaciones donde una pequeña fracción de direcciones MAC concentra la mayor parte de las tramas generadas, cubriendo así un amplio rango de comportamientos observados en sistemas reales.

En la siguiente figura se muestra el número de direcciones MAC distintas que se generan en cada tipo de tráfico en función de su espacio muestral.

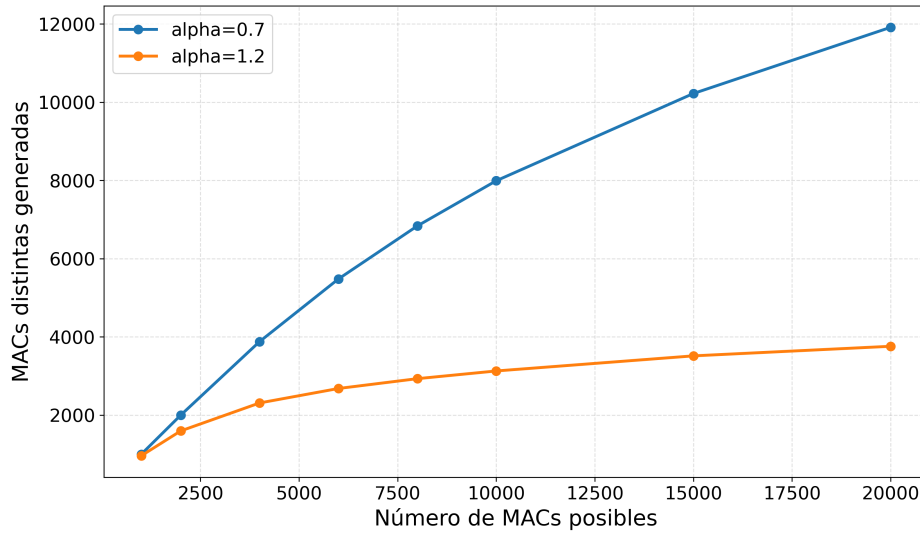


Figura 6.1: MACs distintas generadas según el número de MACs posibles para 30.000 tramas generadas

Además del parámetro α , se fijan dos velocidades de línea: 2 Gbps y 51,2 Gbps. La primera permite estudiar un escenario en el que el sistema trabaja de forma holgada y apenas se descarta ninguna trama por saturación. La segunda representa el caso de máxima carga considerado en este trabajo, equivalente a recibir una trama por ciclo que es la velocidad máxima a la que puede recibir la caché en su proceso de lectura.

Para medir el sistema en distintos entornos, se plantean dos tipos de pruebas diferentes. La primera es un barrido del número de direcciones MAC posibles. En este test se fija el número de tramas generadas a 30.000 y se varía el tamaño del espacio muestral de direcciones MAC. El objetivo es estudiar cómo cambia el comportamiento del sistema cuando aumenta la diversidad del tráfico. Aunque el número de direcciones MAC posibles sea mayor que el tamaño de la tabla MAC, el número de direcciones MAC distintas generadas siempre va a ser menor que el tamaño de la tabla MAC para facilitar el estudio de las distintas métricas. Esta relación entre direcciones MAC posibles y direcciones MAC distintas se muestra en la Figura 6.1.

La segunda prueba es un estudio transitorio. En este caso se fija $\alpha = 0,7$ y se varía el número de tramas generadas. Este valor de α se utiliza porque representa el escenario más parecido a un tráfico Ethernet real. El objetivo del transitorio es observar cómo evolucionan las métricas desde que arranca el sistema hasta que llega a un régimen estacionario.

6.2. Validación cruzada de modelos

Antes de analizar el comportamiento del sistema con las distintas métricas, es importante comprobar que los distintos modelos describen el mismo comportamiento. Este proceso es importante ya que afianza la correcta implementación del módulo de aprendizaje en los distintos modelos y valida la metodología de trabajo que se ha seguido en este trabajo.

Esta comparación se realiza mediante el barrido de direcciones MAC posibles, así se puede comprobar el comportamiento similar en varios puntos de la simulación para distintos valores de α . Para todos los modelos se utiliza una velocidad de línea de 51,2 Gbps para realizar esta validación cruzada en el caso más exigente para el sistema.

La Figura 6.2 agrupa las dos métricas relacionadas con el caudal de tráfico: el filtrado de la caché y los descartes por saturación de la cola MAC. La Figura 6.3 recoge las métricas asociadas al comportamiento de la tabla MAC: el porcentaje de direcciones aprendidas y la eficiencia de escritura.

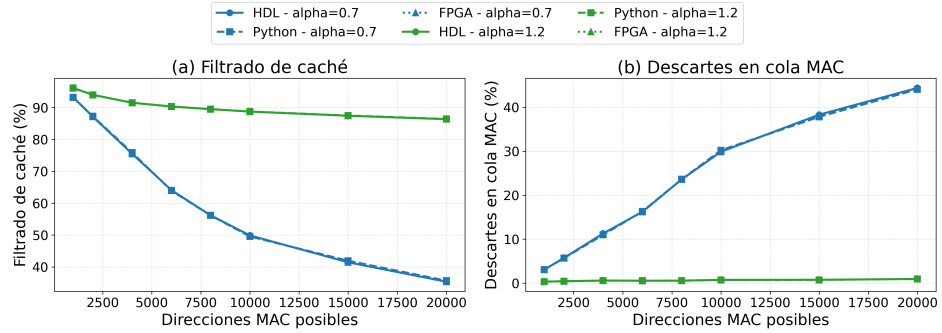


Figura 6.2: Validación cruzada entre Python, HDL y FPGA para el filtrado de caché y los descartes por saturación de la cola MAC.

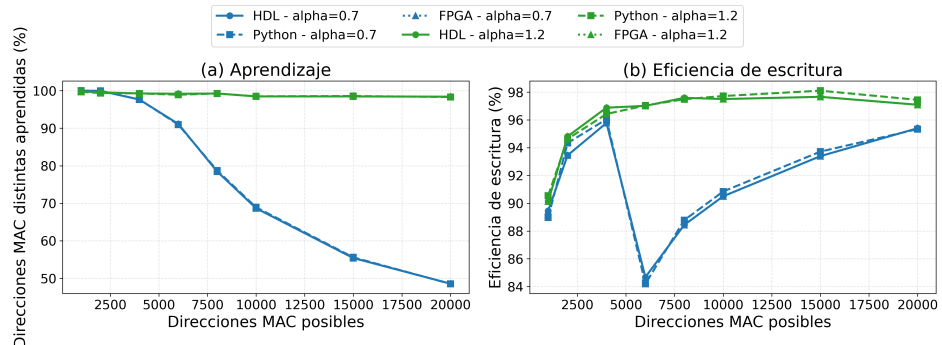


Figura 6.3: Validación cruzada entre Python, HDL y FPGA para el aprendizaje de direcciones MAC y la eficiencia de escritura.

La Tabla 6.1 muestra las diferencias observadas para las distintas métricas recién mostradas. Los valores de la tabla son diferencias porcentuales y muestran la diferencia máxima observada entre los distintos puntos representados en las figuras.

Métrica	Python-HDL	HDL-FPGA
Filtrado de caché	0,48	0,00
Descartes por saturación en MAC	0,47	0,00
Direcciones MAC aprendidas	0,26	0,00
Eficiencia de escritura	0,88	0,00

Tabla 6.1: Diferencias máximas entre Python, HDL y FPGA.

En primer lugar, se observa que la simulación HDL y la ejecución sobre FPGA mantienen exactamente el mismo comportamiento. Para esta comparación se utilizan las trazas de direcciones MAC generadas previamente y cargadas en la BRAM del diseño, de forma que ambos modelos procesan la misma secuencia de entrada. Las curvas coinciden en todos los puntos del barrido. Esto confirma que la lógica sintetizada, la carga de trazas y la lectura de contadores mediante AXI-Lite reproducen correctamente el comportamiento observado en simulación.

Una vez validada la equivalencia entre HDL y FPGA, se comparan estos resultados con el modelo funcional en Python. En este caso aparecen pequeñas diferencias, siempre inferiores al 1 % en las métricas estudiadas. Las pequeñas diferencias observadas entre el modelo funcional en SimPy y la simulación HDL se deben a la distinta semántica temporal de ambos entornos. En HDL, todos los registros del diseño se actualizan de forma simultánea en cada flanco de reloj, utilizando los valores existentes antes de dicho flanco. En cambio, SimPy modela el sistema mediante procesos de eventos discretos que, cuando coinciden en el mismo instante de simulación, deben ejecutarse siguiendo un orden determinado por el planificador.

Esto puede producir diferencias puntuales cuando dos operaciones ocurren en el mismo ciclo. Por ejemplo, si una escritura en la caché finaliza en el mismo instante en el que llega una nueva trama con la misma dirección MAC, el modelo Python puede evaluar antes la escritura o la nueva consulta según el orden de los procesos. En la simulación HDL, por el contrario, la visibilidad de esa escritura queda determinada por la lógica síncrona del circuito y por los registros de la tabla hash.

Aunque existen pequeñas diferencias entre Python y HDL, todas ellas se mantienen por debajo del 1 %. La evolución de las métricas es prácticamente idéntica en los tres modelos, por lo que las conclusiones obtenidas a partir del modelo funcional siguen siendo válidas para la implementación hardware.

6.3. Análisis del comportamiento mediante métricas

Una vez validada la equivalencia entre modelos, se analiza el comportamiento del sistema utilizando las métricas definidas anteriormente. Las métricas analizadas no son independientes entre sí, sino que describen distintas partes del funcionamiento del sistema interconectadas. En primer lugar, el porcentaje de filtrado de la caché determina qué proporción del tráfico queda suprimida antes de alcanzar la tabla MAC. Este valor incide directamente en la carga que llega a la cola de escritura de la tabla MAC y, por tanto, en el número de descartes por saturación. A su vez, una reducción de los descartes permite que más direcciones MAC nuevas puedan ser procesadas por la tabla MAC, lo que repercute sobre el porcentaje de aprendizaje alcanzado. Finalmente, la eficiencia de escritura permite evaluar si las operaciones que llegan a la tabla MAC aportan realmente nuevas direcciones o si siguen existiendo escrituras redundantes después del proceso de filtrado.

Por este motivo, las métricas se presentan siguiendo el orden natural del sistema: primero el filtrado realizado por la caché, después los descartes en la cola de la tabla MAC y, finalmente, las métricas asociadas al aprendizaje y a la eficiencia de escritura de la tabla MAC.

El estudio se realiza mediante dos tipos de simulaciones. El primer análisis consiste en un barrido del número de direcciones MAC posibles, manteniendo fijo el número de tramas generadas. El segundo analiza el transitorio temporal, manteniendo fijo el espacio de direcciones y variando el número de tramas procesadas.

Filtrado por caché

La primera métrica a analizar es el porcentaje de tramas filtradas por la caché.

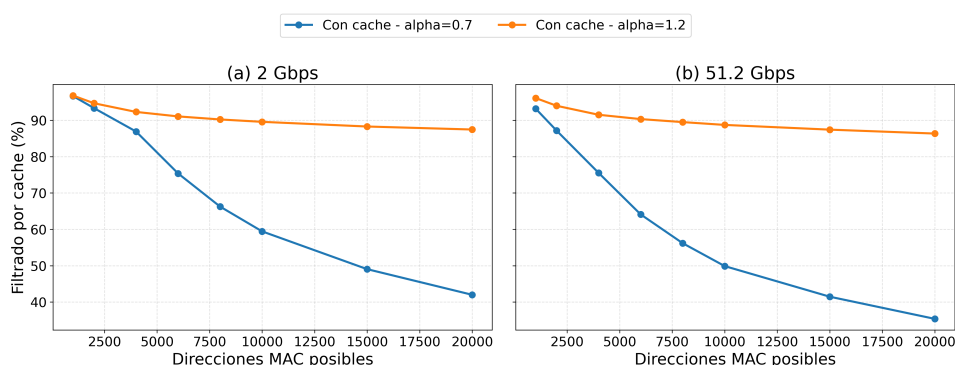


Figura 6.4: Porcentaje de tráfico filtrado por la caché en función de las direcciones MAC posibles.

La Figura 6.4 muestra el porcentaje de tráfico filtrado por la caché para distintas velocidades y distinto tipo de tráfico. En ambos casos se observa que el filtrado disminuye a medida que aumenta el número de direcciones MAC posibles, siendo lógico esto ya que disminuye la probabilidad de que una dirección MAC recién llegada se encuentre ya en la caché.

Asimismo, el parámetro α tiene una influencia significativa en los resultados. Para valores elevados, como $\alpha = 1, 2$, el tráfico se concentra sobre un conjunto reducido de direcciones MAC, lo que favorece los aciertos en caché y permite mantener porcentajes de filtrado cercanos al 90 %. Para este valor elevado de α , apenas aumenta el tráfico filtrado aunque aumenten las direcciones MAC posibles, ya que apenas aumentan las direcciones MAC distintas generadas como se mostró en la Figura 6.1.

Por el contrario, para valores de tráfico estándar, como $\alpha = 0, 7$, el tráfico se distribuye de forma más uniforme entre las distintas direcciones, reduciendo notablemente la efectividad de la caché.

También puede apreciarse una ligera disminución del filtrado al aumentar la velocidad de enlace de 2 Gbps a 51,2 Gbps. Esto se debe a que el tiempo disponible entre la llegada de tramas consecutivas es menor, por lo que en determinadas situaciones la caché no dispone de tiempo suficiente para almacenar una dirección MAC antes de que vuelva a aparecer una nueva trama asociada a ella. Como consecuencia, aumenta el número de no coincidencias en la caché y se reduce ligeramente el porcentaje de tráfico filtrado.

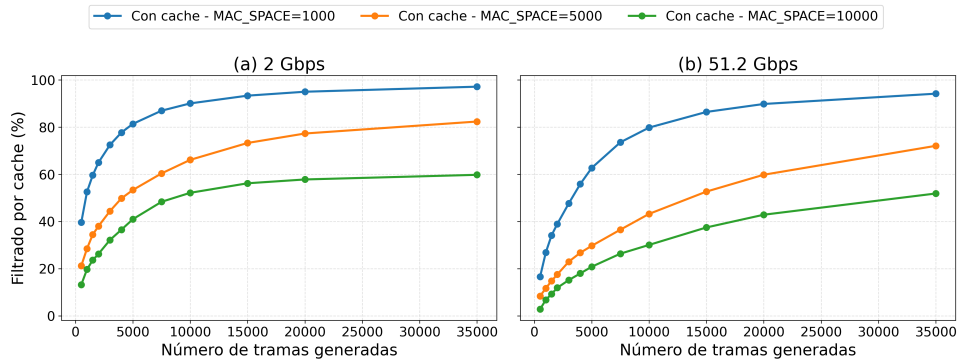


Figura 6.5: Evolución temporal del porcentaje de filtrado de caché en función de las tramas generadas.

La Figura 6.5 muestra que independientemente de la velocidad y de las direcciones MAC posibles, el porcentaje de elementos que filtra la caché aumenta a medida que aumenta el tiempo de simulación. Esto es debido a que a medida que se van generando mayor número de tramas, la caché va acumulando direcciones previamente observadas, aumentando progresivamente la probabilidad de acierto hasta alcanzar un régimen estacionario.

Este régimen estacionario se alcanza en menor número de tramas a una velocidad de 2 Gbps ya que el sistema procesa un mayor número de tramas para el mismo número de tramas generadas debido a que a 2 Gbps apenas existen descartes y todas las inserciones disponen de tiempo suficiente para completarse.

Descartes por saturación en la cola MAC

Los descartes por saturación indican qué parte del tráfico no puede entrar en la cola de escritura de la tabla MAC. Esta métrica complementa al filtrado, ya que permite comprobar si la reducción de tráfico conseguida por la caché es suficiente para evitar pérdidas cuando la velocidad de línea aumenta.

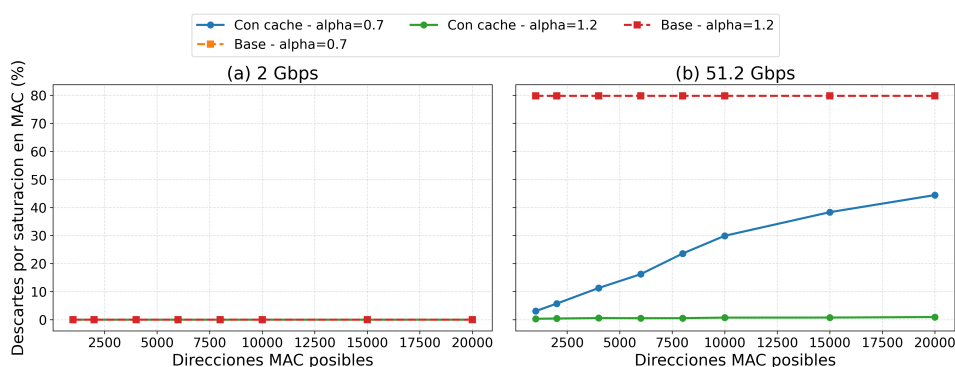


Figura 6.6: Porcentaje de descartes por saturación de la cola MAC en función de las direcciones MAC posibles.

Como se observa en la Figura 6.6, a 2 Gbps prácticamente no aparecen descartes por saturación, ya que la separación temporal entre tramas permite que la cola MAC y el escritor de la tabla MAC absorban las solicitudes generadas. En cambio, a 51,2 Gbps la presión sobre la cola MAC aumenta de forma notable. El modelo base, al enviar todas las tramas hacia la tabla MAC, alcanza porcentajes de descarte muy elevados. El modelo con caché reduce estos descartes porque una parte importante de las repeticiones queda filtrada antes de llegar a la tabla MAC.

La mejora depende de la concentración del tráfico. Con $\alpha = 1, 2$, la caché filtra mejor y los descartes se mantienen bajos. Con $\alpha = 0, 7$, al aumentar el número de direcciones MAC posibles aparecen más fallos de caché y, por tanto, más solicitudes hacia la cola MAC. En ese caso los descartes crecen, aunque siguen siendo claramente inferiores a los del modelo base.

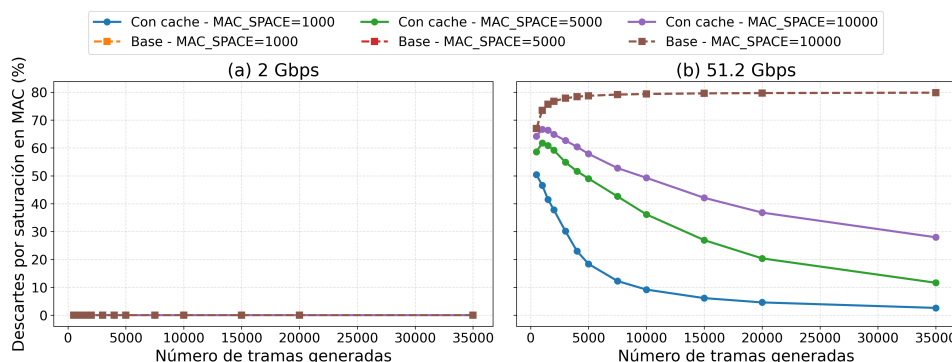


Figura 6.7: Evolución temporal de los descartes por saturación de la cola MAC.

La Figura 6.7 muestra la evolución temporal de los descartes por saturación de la tabla MAC. A 2 Gbps no se observan descartes, ya que la separación temporal entre tramas es suficientemente grande para que la cola MAC se vacíe antes de la llegada de nuevas solicitudes. En cambio, a 51,2 Gbps la tasa de llegada es mucho mayor y la cola MAC se satura durante el transitorio, especialmente para espacios de direcciones grandes. En el modelo sin caché los descartes aumentan de forma continuada con el número de tramas, ya que todas las solicitudes compiten directamente por el acceso a la cola MAC. Por el contrario, en el modelo con caché los descartes crecen inicialmente, pero posteriormente tienden a reducirse o estabilizarse, debido a que la caché va acumulando direcciones ya observadas y filtra una parte creciente del tráfico repetido.

Porcentaje de aprendizaje

El porcentaje de aprendizaje permite estudiar qué parte de las direcciones MAC distintas generadas termina almacenada correctamente en la tabla MAC. Esta métrica es importante porque indica si el filtrado previo reduce escrituras redundantes permitiendo aprender a la tabla MAC el mayor número de direcciones MAC distintas posibles.

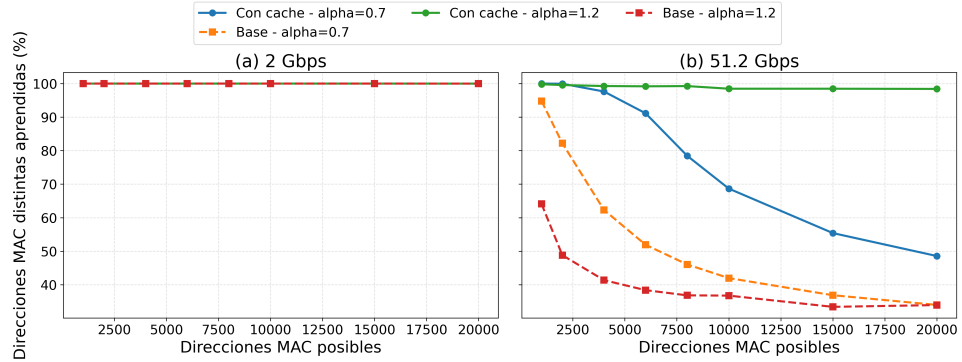


Figura 6.8: Porcentaje de direcciones MAC aprendidas en función del número de MACs posibles.

A 2 Gbps se mantiene el 100 % de aprendizaje para ambos modelos independientemente del valor de α , ya que la separación temporal entre tramas permite que la cola y la tabla MAC procesen las tramas sin pérdidas. En cambio, a 51,2 Gbps aparecen diferencias claras entre los distintos modelos. Para $\alpha = 0,7$, donde el tráfico está menos concentrado y se generan más direcciones distintas, el porcentaje de aprendizaje disminuye al aumentar el espacio de direcciones, debido a la mayor presión sobre la cola de la tabla MAC. Para $\alpha = 1,2$, el tráfico está más concentrado en pocas direcciones frecuentes, por lo que la caché consigue filtrar eficazmente las repeticiones y el aprendizaje se mantiene cercano al 100 % incluso a una tasa de llegada de una trama por ciclo de reloj.

Independientemente del tipo de tráfico, se aprecia una mejora clara en el modelo con caché, especialmente cuando el tráfico presenta una mayor repetición de direcciones. Esta mejora no se debe a un aumento de la capacidad de la tabla MAC, que permanece constante, sino a la reducción de escrituras redundantes que compiten por el acceso a la cola y el escritor de la tabla MAC. Como consecuencia, una mayor proporción de las solicitudes procesadas corresponde a direcciones MAC nuevas, aumentando el porcentaje de aprendizaje.

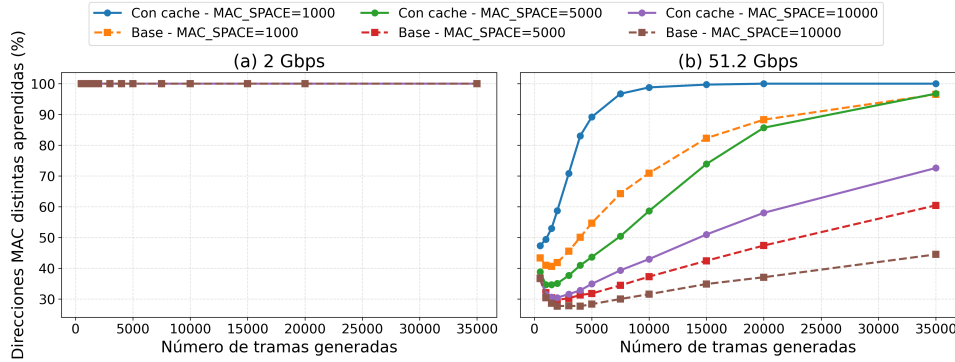


Figura 6.9: Evolución temporal del porcentaje de direcciones MAC aprendidas.

La Figura 6.9 representa la evolución temporal del porcentaje de direcciones MAC distintas aprendidas para diferentes tamaños muestrales de las posibles direcciones MAC. A 2 Gbps todas las configuraciones se mantienen en el 100 % de aprendizaje durante toda la simulación. Esto es acorde al resto de gráficas estudiadas a esta velocidad. En este régimen, la caché no resulta crítica para evitar pérdidas, ya que el propio sistema sin caché es capaz de absorber todo el tráfico generado.

A 51,2 Gbps se puede observar un comportamiento distinto. Al principio de la simulación, se obtiene un porcentaje de aprendizaje entre el 30 % y el 50 %. Esto es debido a que cuando comienza la simulación, el número de direcciones MAC distintas generadas crece de una forma mucho mayor ya que no se ha procesado ninguna trama y la mayor parte de las primeras tramas van a ser direcciones MAC nuevas. Esto provoca una saturación de la cola de escritura como se ha visto anteriormente. A medida que transcurre la simulación, la probabilidad de que una nueva dirección MAC sea nueva disminuye considerablemente, lo que reduce la saturación del sistema y mejora el porcentaje de aprendizaje.

Eficiencia de escritura

La eficiencia de escritura mide qué proporción de las solicitudes que alcanzan el escritor de la tabla MAC terminan aportando una dirección nueva. Por tanto, permite distinguir entre aprender muchas direcciones y hacerlo con un número reducido de escrituras redundantes.

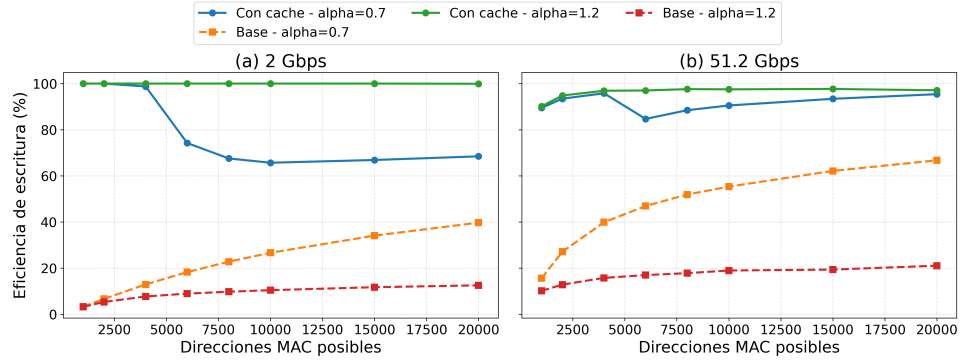


Figura 6.10: Eficiencia de escritura en la tabla MAC en función del número de MACs posibles.

La Figura 6.10 muestra la eficiencia de escritura en la tabla MAC para los modelos con y sin caché. A 2 Gbps no se producen pérdidas por saturación, por lo que todos los intentos válidos alcanzan la tabla MAC. En este caso, el modelo sin caché presenta una eficiencia baja para espacios muestrales de direcciones MAC pequeños, ya que gran parte de las tramas corresponden a direcciones ya aprendidas y generan escrituras repetidas. Al aumentar el número de direcciones MAC posibles, crece la diversidad del tráfico y una mayor proporción de los accesos corresponde a direcciones nuevas, por lo que la eficiencia mejora. En el modelo con caché, para $\alpha = 1,2$ la eficiencia se mantiene cercana al 100 %, ya que la alta repetición de direcciones permite que la caché filtre eficazmente los accesos redundantes. Para $\alpha = 0,7$, en cambio, la distribución es menos concentrada y aparecen más fallos de caché repetidos, reduciendo la eficiencia aunque el porcentaje de aprendizaje se mantenga elevado.

A 51,2 Gbps la interpretación de la eficiencia debe realizarse teniendo en cuenta los descartes previos a la tabla MAC. En este régimen aumenta la presión sobre la cola de entrada de la tabla, de modo que no todos los fallos de caché llegan a convertirse en intentos de escritura. Por ello, algunas curvas pueden mostrar valores de eficiencia superiores a los observados a 2 Gbps, aunque el aprendizaje global sea menor. Esto ocurre porque la métrica se calcula únicamente sobre los intentos que alcanzan el escritor MAC, y no sobre la totalidad de tramas generadas ni sobre todos los fallos de caché. Así, la saturación elimina parte del tráfico antes de que forme parte del denominador de la eficiencia, modificando la composición de los intentos procesados en el proceso de aprendizaje.

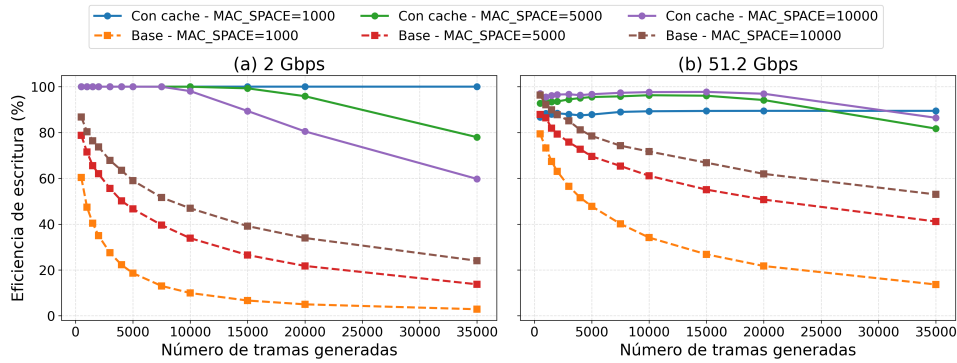


Figura 6.11: Evolución temporal de la eficiencia de escritura en la tabla MAC.

La Figura 6.11 complementa el análisis anterior. En el modelo base, la eficiencia cae a medida que aparecen más direcciones MAC repetidas, ya que muchas escrituras intentan insertar direcciones que la tabla ya conoce. En el modelo con caché, la eficiencia se mantiene más alta durante más tiempo porque una parte de esas repeticiones no llega a la tabla MAC.

6.4. Caracterización de recursos en la FPGA

Además del análisis funcional realizado mediante las distintas métricas de diseño, es fundamental realizar un estudio sobre los recursos que emplea la arquitectura propuesta sobre la FPGA para analizar si es adecuado su diseño. A continuación se muestra en la siguiente tabla los recursos de utilización global tras la implementación de la arquitectura propuesta en la FPGA ZCU102.

Recurso	Utilizado	Disponible	Uso
CLB LUTs	3701	274080	1,35 %
CLB Registers	2934	548160	0,54 %
Block RAM Tile	125	912	13,71 %
DSPs	0	2520	0,00 %

Tabla 6.2: Utilización global de recursos en la FPGA.

Los datos de la Tabla 6.2 muestran que el consumo de lógica es muy reducido. El diseño utiliza un 1,35 % de las LUTs disponibles y un 0,54 % de los registros. Esto indica que la lógica de control, los contadores y las distintas interfaces apenas consumen recursos de la FPGA.

El recurso más utilizado es la memoria BRAM, con un 13,71 % de ocupación. Para interpretar correctamente este valor conviene separar la memoria utilizada por el entorno de pruebas de la memoria propia del módulo de

aprendizaje. Como se ha explicado anteriormente, para realizar las pruebas en la FPGA, las trazas a simular se cargan previamente en la BRAM descrita en `cache_core.v`, en lugar de comunicarse trama a trama durante la ejecución como en la versión HDL.

Esta carga de trazas consume 90 bloques RAMB36, lo que representa un 9,87 % de la BRAM total disponible en la FPGA. Por otro lado, el propio módulo de aprendizaje utiliza 32 bloques RAMB36 y 4 bloques RAMB18. Esto corresponde a un 3,73 % de la BRAM total disponible en la FPGA. Por lo tanto, aunque el consumo de la BRAM pueda parecer algo elevado, la mayor parte del mismo se debe al entorno de pruebas y no al propio módulo de aprendizaje desarrollado.

La ausencia de uso de DSPs confirma que el diseño no depende de bloques aritméticos dedicados. Las operaciones necesarias para el aprendizaje y filtrado se resuelven mediante lógica combinatorial, registros, colas FIFO y memorias internas. Por tanto, la implementación deja disponible la práctica totalidad de los recursos de cálculo de la FPGA para otros bloques del sistema.

Para validar temporalmente la implementación se utilizan los márgenes de *setup* y *hold* reportados por Vivado. Estas métricas indican cuánto margen temporal queda antes de incumplir las restricciones impuestas al reloj del diseño; por tanto, valores positivos implican que no existen violaciones temporales.

El núcleo del sistema trabaja con un reloj interno de 100 MHz, correspondiente a un periodo de 10 ns. El informe de *timing* de Vivado indica que se cumplen todas las restricciones temporales especificadas. En concreto, el peor margen de *setup* obtenido es de 3,307 ns, mientras que el peor margen de *hold* es de 0,003 ns.

6.5. Discusión de resultados

Los resultados obtenidos permiten extraer varias conclusiones sobre el modelo desarrollado. En primer lugar, la validación cruzada entre Python, HDL y FPGA confirma que los tres modelos representan el mismo sistema y valida la metodología seguida en este trabajo. El modelo Python permite estudiar de forma flexible el comportamiento de la arquitectura y justificar su posterior implementación hardware. La simulación HDL verifica que dicho comportamiento puede describirse mediante lenguaje hardware y sirve como paso intermedio antes de la ejecución sobre la FPGA. Por último, la prueba realizada en la FPGA demuestra que el diseño sintetizado puede ejecutarse en hardware real manteniendo las mismas condiciones estudiadas en las etapas anteriores.

En segundo lugar, la caché demuestra ser eficaz para reducir escrituras redundantes sobre la tabla MAC. Su efecto es especialmente importante

a altas velocidades de línea, donde la tabla MAC no puede absorber una solicitud de escritura por cada trama generada. Al filtrar las direcciones MAC repetidas, la caché reduce la presión sobre la cola MAC, disminuye los descartes y permite que una mayor parte de las escrituras procesadas correspondan a direcciones MAC nuevas. Esto aumenta considerablemente el porcentaje de direcciones MAC que la tabla MAC puede aprender.

Desde el punto de vista hardware, la solución presenta un coste bajo. El uso de LUTs y registros es reducido, no se emplean DSPs y el consumo principal aparece en BRAM, como cabía esperar al implementar estructuras de almacenamiento basadas en tablas hash y una memoria de trazas. Además, el diseño cumple temporización a 100 MHz, por lo que resulta viable para el entorno de pruebas utilizado.

Capítulo 7

Conclusiones y trabajo futuro

7.1. Conclusiones

Los resultados obtenidos permiten considerar alcanzado el objetivo general y los objetivos específicos planteados en la Sección 1.3.

El principal objetivo de este trabajo era diseñar e implementar una solución hardware capaz de optimizar el proceso de aprendizaje dinámico de la tabla MAC de un *switch* Ethernet. Esta solución debía ser capaz de reducir las escrituras redundantes que alcanzan la tabla MAC, siendo capaz de implementarse sobre FPGA.

Para poder diseñar y evaluar esta solución, uno de los primeros objetivos planteados fue caracterizar el tráfico Ethernet y definir un modelo que permitiera reproducir diferentes condiciones de funcionamiento. Para ello, se ha utilizado una distribución de Zipf, con la que es posible representar distintos grados de repetición de las direcciones MAC mediante la variación del parámetro α . De esta forma, se ha conseguido disponer de un modelo de tráfico parametrizable sobre el que evaluar y comparar las distintas arquitecturas propuestas.

La solución propuesta consiste en incorporar una memoria caché con su respectiva cola FIFO antes de la tabla MAC. Su función es detectar aquellas direcciones que aparecen de forma repetida en el tráfico y evitar que todas ellas generen una nueva operación de escritura sobre la tabla MAC. Esta arquitectura ha sido propuesta después de comparar diferentes posibles soluciones evaluadas mediante un modelo funcional en Python. Aunque también se ha estudiado una solución basada en CMS y caché, más fiel a la idea original de TinyLFU en la que se ha basado el proyecto, los resultados obtenidos son muy similares a los de la arquitectura que utiliza únicamente caché. Debido a esto, se utiliza la arquitectura que utiliza solo la caché ya que permite obtener prácticamente la misma mejora respecto al modelo base

con una complejidad mucho menor.

Los resultados obtenidos muestran que la incorporación de una caché antes de la tabla MAC reduce significativamente las escrituras redundantes asociadas al aprendizaje dinámico. Esta mejora se ha evaluado mediante distintas métricas, como el porcentaje de tráfico filtrado por la caché, los descartes por saturación de la tabla MAC, el porcentaje de direcciones MAC aprendidas y la eficiencia de escritura en la tabla MAC. El análisis conjunto de estas métricas muestra que el filtrado de direcciones MAC repetidas reduce la presión sobre la tabla MAC y permite que una mayor proporción de las nuevas direcciones sea finalmente aprendida.

Esta mejora es especialmente importante en los escenarios de mayor presión sobre el sistema, donde el modelo sin caché descarta un mayor número de direcciones MAC por saturación. Esta mejora también depende del grado de repetición de las direcciones MAC en el tráfico de red, ya que cuanto mayor sea la concentración del tráfico sobre un conjunto reducido de direcciones, mayor será la capacidad de la caché para filtrar escrituras redundantes.

La arquitectura seleccionada se ha implementado posteriormente en Verilog HDL y se ha integrado en una FPGA Zynq UltraScale+ XCZU9EG. La validación cruzada de los resultados obtenidos en Python, en la simulación HDL y en la ejecución sobre FPGA muestra un comportamiento equivalente entre las distintas implementaciones. Por tanto, se puede considerar cumplido el objetivo de trasladar la solución desde un modelo funcional hasta una implementación hardware real, manteniendo el comportamiento esperado durante las distintas etapas de simulación.

La implementación sobre FPGA también ha permitido analizar el coste hardware del modelo propuesto. Los resultados de utilización muestran un consumo reducido de recursos lógicos y la ausencia de bloques DSP. El principal recurso utilizado es la memoria BRAM, debido tanto a las estructuras de almacenamiento necesarias para el funcionamiento del sistema como a las memorias utilizadas para almacenar las trazas durante las pruebas. Estos resultados muestran que la incorporación del mecanismo de filtrado no requiere una cantidad elevada de recursos de procesamiento de la FPGA.

En conclusión, la utilización de una caché previa a la tabla MAC constituye una solución válida para reducir la presión sobre el proceso de aprendizaje dinámico de un *switch* Ethernet. La arquitectura desarrollada proporciona una solución sencilla, de bajo coste hardware y capaz de mejorar el funcionamiento del sistema en aquellos escenarios donde la tasa de escritura de la tabla MAC puede convertirse en un factor limitante.

7.2. Trabajo futuro

Aunque el modelo desarrollado ha sido validado tanto mediante simulación como sobre FPGA, aún existen varias líneas que permitirían continuar y ampliar el trabajo realizado.

La principal línea de trabajo futuro consiste en integrar el módulo dentro de la arquitectura completa de un *switch* Ethernet. En este trabajo, la implementación sobre FPGA se ha probado mediante un entorno controlado cargando trazas de direcciones MAC sobre una BRAM. Una integración completa permitiría evaluar el mecanismo de filtrado directamente sobre el tráfico real recibido directamente por los puertos del *switch* y estudiar su interacción con los procesos completos de lectura, escritura y reenvío del *switch*.

Otra posible línea consiste en continuar el desarrollo de la arquitectura basada en TinyLFU. Los resultados obtenidos muestran que la incorporación del CMS no aporta una mejora significativa frente al uso exclusivo de la caché en los escenarios estudiados. Sin embargo, sería interesante analizar distintas configuraciones de la arquitectura de TinyLFU completa, ya que diversas configuraciones podrían ser útiles para algunos escenarios de tráfico no analizados en este trabajo o escenarios concretos donde un filtro *doorkeeper* como el que contiene TinyLFU puede ser útil.

En conjunto, estas líneas de trabajo permitirían evolucionar la arquitectura desarrollada hacia una solución completamente integrada en un *switch* Ethernet y ampliar su aplicación a nuevos escenarios, manteniendo como objetivo principal la optimización del aprendizaje dinámico de la tabla MAC.

Apéndice A

Repositorio

Se ha creado un repositorio en GitHub para almacenar el código, las librerías utilizadas y los reportes generados por Vivado durante el desarrollo de este proyecto. Todo el contenido se encuentra disponible en la siguiente dirección: <https://github.com/luisgalindoort/Optimizacion-aprendizaje-dinamico-conmutadores-Ethernet-alto-rendimiento>.

El repositorio se ha organizado en tres carpetas principales, separando los distintos modelos desarrollados durante el proyecto:

- **Python:** contiene los modelos funcionales desarrollados en Python, incluyendo el modelo base, el modelo con caché y el modelo con CMS y caché. También se incluyen los test utilizados para estudiar las distintas métricas.
- **HDL:** contiene la implementación RTL desarrollada en Verilog HDL, junto con los módulos utilizados para implementar la caché, la tabla MAC y las colas FIFO. También se incluyen las distintas librerías utilizadas en esta fase y los test utilizados.
- **FPGA:** contiene los archivos necesarios para la implementación y validación del diseño sobre la FPGA. Se incluyen los módulos utilizados para integrar el diseño, los test en Python utilizados en la carga y procesamiento de trazas y los ficheros relacionados con la síntesis, utilización de recursos e implementación de Vivado.

Bibliografía

- [1] John D'Ambrosia, Kent Lusted, Gary Nicholl, Dave Ofelt, Mark Nowell, Matt Brown, and Rob Stone. Project Overview – IEEE P802.3df: 200 Gb/s, 400 Gb/s, 800 Gb/s, and 1.6 Tb/s Ethernet. IEEE 802.3 Beyond 400 Gb/s Ethernet Study Group, 2021. Disponible en: https://grouper.ieee.org/groups/802/3/B400G/public/21_1028/index.html.
- [2] IEEE. IEEE Standard for Ethernet. IEEE Std 802.3-2018 (Revision of IEEE Std 802.3-2015), 2018. DOI: <https://doi.org/10.1109/IEEESTD.2018.8457469>.
- [3] Radia Perlman. Interconnections: Bridges, Routers, Switches, and Internetworking Protocols. Addison-Wesley, 2000. <https://www.pearson.com/en-us/subject-catalog/p/interconnections-bridges-routers-switches-and-internetworking-protocols/P200000003321>.
- [4] Joaquín Álvarez Horcajo, Isaias Martinez-Yelmo, Diego Lopez-Pajares, J. A. Carral, and Marco Savi. A Hybrid SDN Switch Based on Standard P4 Code. IEEE Communications Letters, vol. 25, no. 3, 2021. URL: <https://ieeexplore.ieee.org/document/9276311>.
- [5] IEEE. IEEE Standard for Local and Metropolitan Area Networks: Media Access Control (MAC) Bridges. IEEE Std 802.1D, 2004. DOI: <https://doi.org/10.1109/IEEESTD.2004.94569>.
- [6] David E. Taylor and Edward W. Spitznagel. On Using Content Addressable Memory for Packet Classification. Washington University in St. Louis, Technical Report WUCSE-2005-9, 2005. https://openscholarship.wustl.edu/cse_research/979/.
- [7] Sneha Lata Murotiya and Anu Gupta. CNTFET based design of content addressable memory cells. 2013 4th International Conference on Computer and Communication Technology (ICCCCT), 2013. pp. 1–4. DOI: <https://doi.org/10.1109/ICCCCT.2013.6749593>.
- [8] Rasmus Pagh and Flemming Friche Rodler. Cuckoo Hashing. European Symposium on Algorithms (ESA), 2001. DOI: https://doi.org/10.1007/3-540-44676-1_12.

- [9] Salvatore Pontarelli, Pedro Reviriego, and Juan Antonio Maestro. Parallel d-Pipeline: A Cuckoo Hashing Implementation for Increased Throughput. *IEEE Transactions on Computers*, 2016. DOI: <https://doi.org/10.1109/TC.2015.2417524>.
- [10] A. Gopalan and S. Ramasubramanian. Fast recovery from link failures in Ethernet networks. *International Workshop on the Design of Reliable Communication Networks (DRCN)*, 2013. <https://ieeexplore.ieee.org/document/6578334>.
- [11] Clive Maxfield. *FPGAs: Instant Access*. Newnes, 2008. <https://shop.elsevier.com/books/fpgas-instant-access/maxfield/978-0-7506-8974-8>.
- [12] Xilinx, Inc. UltraRAM: Breakthrough Embedded Memory Integration on UltraScale+ Devices (WP477). Xilinx White Paper, 2016. <https://docs.amd.com/v/u/en-US/wp477-ultraram>.
- [13] Meinhard Kissich. *Architecture of FPGAs*. Graz University of Technology, 2021. <https://www.isec.tugraz.at/wp-content/uploads/2021/08/p1.pdf>.
- [14] Tamás Tóthfalusi and Peter Orosz. Line-rate packet processing in hardware: the evolution towards 400 Gbit/s. *Proceedings of the 9th International Conference on Applied Informatics*, vol. 1, pp. 259–268, 2015. URL: https://www.researchgate.net/publication/300661776_Line-rate_packet_processing_in_hardware_the_evolution_towards_400_Gbits.
- [15] Carlos Megías, Alex Forencich, Eduardo Ros, and Javier Díaz. Minimizing Cuckoo Hashing Insertion Time for Networking Applications in FPGAs. *IEEE Access*, vol. 13, pp. 216832–216841, 2025. DOI: <https://doi.org/10.1109/ACCESS.2025.3647975>.
- [16] John W. Lockwood, Nick McKeown, Gregory Watson, Glen Gibb, Paul Hartke, Jad Naous, Ramanan Raghuraman, and Jianying Luo. NetFPGA—An Open Platform for Gigabit-Rate Network Switching and Routing. *International Conference on Microelectronic Systems Education (MSE)*, 2007. URL: https://yuba.stanford.edu/~grg/docs/netfpga_open_platform_short.pdf.
- [17] Grzegorz Daniluk and Tomasz Włostowski. White Rabbit: Sub-Nanosecond Synchronization for Embedded Systems. *Proceedings of the 43rd Annual Precise Time and Time Interval Systems and Applications Meeting*, pp. 45–60, 2011. URL: <https://www.ion.org/publications/abstract.cfm?articleID=10773>.

- [18] Alex Forencich, Alex C. Snoeren, George Porter, and George Papen. Corundum: An Open-Source 100-Gbps NIC. 28th IEEE International Symposium on Field-Programmable Custom Computing Machines (FCCM), 2020. DOI: <https://doi.org/10.1109/FCCM48280.2020.00015>.
- [19] Cisco Systems. Cisco Nexus 3550-T Programmable Network Platform Data Sheet. Cisco Data Sheet, 2021. <https://www.cisco.com/c/en/us/products/collateral/switches/nexus-3550-series/datasheet-c78-744762.html>.
- [20] Lee Breslau, Pei Cao, Li Fan, Graham Phillips, and Scott Shenker. Web Caching and Zipf-like Distributions: Evidence and Implications. Proceedings of IEEE INFOCOM, 1999. DOI: <https://doi.org/10.1109/INFOCOM.1999.749260>.
- [21] Christos Tsilopoulos, George Xylomenos, and Yannis Thomas. Reducing Forwarding State in Content-Centric Networks with Semi-Stateless Forwarding. Proceedings of IEEE INFOCOM, 2014. DOI: <https://doi.org/10.1109/INFOCOM.2014.6848039>.
- [22] SimPy Developers. SimPy Documentation: Basic Concepts. SimPy Documentation, 2026. Disponible en: https://simpy.readthedocs.io/en/stable/simpy_intro/basic_concepts.html.
- [23] IEEE. IEEE Standard for Verilog Hardware Description Language. IEEE Std 1364-2005, 2006. DOI: <https://doi.org/10.1109/IEEESTD.2006.99495>.
- [24] cocotb Developers. cocotb Documentation. cocotb Project Documentation, 2025. Disponible en: <https://docs.cocotb.org/en/stable/>.
- [25] GTKWave Authors. GTKWave Documentation: Quick Start. GTKWave Documentation, 2023. Disponible en: <https://gtkwave.github.io/gtkwave/quickstart/sample.html>.
- [26] Advanced Micro Devices, Inc. Vivado Design Suite User Guide: Implementation. User Guide UG904, 2024. Disponible en: <https://docs.amd.com/r/en-US/ug904-vivado-implementation>.
- [27] Advanced Micro Devices, Inc. ZCU102 Evaluation Board User Guide. User Guide UG1182, 2023. Disponible en: <https://docs.amd.com/v/u/en-US/ug1182-zcu102-eval-bd>.
- [28] Alex Forencich. Extensible FPGA Control Platform. GitHub Repository, 2025. Disponible en: <https://github.com/alexforencich/xfcp>.

- [29] Xilinx, Inc. AXI Reference Guide. User Guide UG761, 2011. Disponible en: https://www.xilinx.com/support/documents/ip_documentation/axi_ref_guide/latest/ug761_axi_reference_guide.pdf.
- [30] Faishal Zharfan, Larastya Devindira Hasna, Ridha Muldina Negara, and N. Syambas. Comparison of Caching Replacement Policies in Changing the Number of Interest Packets on Named Data Networks Using Mininet-NDN. International Conference on Telecommunication Systems, Services, and Applications (TSSA), 2021. DOI: <https://doi.org/10.1109/TSSA52821.2021.9768514>.
- [31] Jie Li, Jinlong Wu, György Dán, Åke Arvidsson, and Maria Kihl. Performance Analysis of Local Caching Replacement Policies for Internet Video Streaming Services. International Conference on Software, Telecommunications and Computer Networks (SoftCOM), 2014. URL: <https://doi.org/10.1109/SoftCOM.2014.7039112>.
- [32] Ramakrishna Karedla, J. Spencer Love, and Bradley G. Wherry. Caching Strategies to Improve Disk System Performance. IEEE Computer, 1994. DOI: <https://doi.org/10.1109/2.268884>. Disponible en: <https://www.computer.org/csdl/magazine/co/1994/03/r3038/13rRUyYjKdx>.
- [33] Nimrod Megiddo and Dharmendra S. Modha. Outperforming LRU with an Adaptive Replacement Cache Algorithm. Computer, 2004. DOI: <https://doi.org/10.1109/MC.2004.1297303>.
- [34] Sixto J. Castro, Edwin F. Boza, and Cristina L. Abad. An Evaluation of Cache Management Policies Under Workloads with Malicious Requests. IEEE 2nd Ecuador Technical Chapters Meeting (ETCM), 2017. URL: <https://doi.org/10.1109/ETCM.2017.8247467>.
- [35] Gil Einziger and Roy Friedman. TinyLFU: A Highly Efficient Cache Admission Policy. 22nd Euromicro International Conference on Parallel, Distributed, and Network-Based Processing (PDP), 2014. URL: <https://doi.org/10.1109/PDP.2014.34>.
- [36] Graham Cormode and S. Muthukrishnan. An Improved Data Stream Summary: The Count-Min Sketch and Its Applications. Journal of Algorithms, vol. 55, no. 1, 2005. URL: <https://doi.org/10.1016/j.jalgor.2003.12.001>.
- [37] Burton H. Bloom. Space/Time Trade-Offs in Hash Coding with Allowable Errors. Communications of the ACM, vol. 13, no. 7, 1970. URL: <https://doi.org/10.1145/362686.362692>.